Pearson
BTEC

شهادة Pearson BTEC
International من المستوى 2 في

تكنولوجيا المعلومات

الوحدة 5

مقدمة في البرمجة

المؤلف: روب كادويل

المستوى
2

كتاب الطالب

تم النشر بواسطة شركة بيرسون إديوكيشن ليمتد، 80 ستراند، لندن، WC2R 0RL.

www.pearsonschoolsandfecolleges.co.uk

يمكن العثور على نسخ من المواصفات الرسمية لجميع شهادات بيرسون التأهيلية على الموقع الإلكتروني: qualifications.pearson.com

© حقوق التأليف والنشر لمتن النصوص لعام 2023 محفوظة لشركة بيرسون إديوكيشن ليمتد

افتتاحية بقلم شركة فلورنس برودكشن ليمتد

تم الطباعة بواسطة شركة فلورنس برودكشن ليمتد

© حقوق التأليف والنشر للرسومات التوضيحية الأصلية محفوظة لشركة بيرسون إديوكيشن ليمتد

الرسومات التوضيحية من إعداد شركة فلورنس برودكشن ليمتد

نُشرت هذه الطبعة عام 2023

فهرسة المكتبة البريطانية في بيانات النشر

يتوفر سجل كتالوج لهذا الكتاب من المكتبة البريطانية

إشعار حقوق التأليف والنشر

جميع الحقوق محفوظة. لا يجوز إعادة إنتاج أي جزء من هذا المنشور بأي شكل أو بأي وسيلة (بما في ذلك نسخه أو تخزينه في أي وسيط باستخدام الوسائل الإلكترونية، سواء بشكل عابر أو عرضي لبعض الاستخدامات الأخرى لهذا المنشور) دون إذن كتابي من مالك حقوق الطبع والنشر، يُتوقع ذلك وفقًا لأحكام قانون حقوق التأليف والنشر والتصاميم وبراءات الاختراع لعام 1988 أو بموجب شروط ترخيص صادر عن وكالة ترخيص حقوق التأليف والنشر، برناردز إن، 86 فيتر لين، لندن www.cla.co.uk) EC4A 1EN). يجب توجيه طلبات الحصول على إذن كتابي لمالك حقوق التأليف والنشر إلى الناشر.

مواقع الويب

لا تتحمل بيرسون إديوكيشن ليمتد المسؤولية عن المحتوى الخاص بأي مواقع إنترنت خارجية. ومن الضروري أن يقوم المعلمون بمعاينة كل موقع ويب قبل استخدامه في الفصل للتأكد من أن عنوان URL لا يزال دقيقًا وملائمًا ومناسبًا. ونقترح أن يقوم المعلمون بوضع إشارة مرجعية على مواقع الويب المفيدة والنظر في تمكين الطلاب من الوصول إليها من خلال الشبكة الداخلية للمدرسة/ الكلية.

ملاحظة من الناشر تنفذ بيرسون عمليات تحرير قوية لضمان دقة المحتوى في هذا المنشور، ويتم بذل كل جهد ممكن لضمان خلو هذا المنشور من الأخطاء. ومع ذلك، ما نحن إلا بشر، وأحيانًا تحدث أخطاء. ولا تتحمل بيرسون المسؤولية عن أي سوء فهم ينشأ نتيجة أخطاء في هذا المنشور، ولكن من أولوياتنا ضمان دقة المحتوى. إذا لاحظت وجود خطأ، فيُرجى التواصل معنا عبر resourcescorrections@pearson.com حتى نتأكد من تصحيحه.

في حين بذل الناشرون قصارى جهدهم لضمان دقة المشورة بشأن التأهيل وتقييمه، فإن المواصفات الرسمية ومواد إرشادات التقييم المرتبطة بها هي المصادر الموثوقة الوحيدة للمعلومات ويجب الرجوع إليها دائمًا للحصول على إرشادات نهائية.

شكروتقدير

مفتاح تفسيري:

l = يسار، r = يمين، c = وسط، b = أسفل

الصور:

123RF: بيوتركت، ص 1 (الشكل 5.1)؛ **بيرسون إديوكيشن ليمتد**: يورغ كارستنسن، ص 11 ص (الشكل 5.11)؛ **شوترستوك**: غريبيشكوف مكسيم، ص 9 (الشكل 5.9)، بيرتل 123، ص 11 ل (الشكل 5.11)، ريكافوتو، ص 11 ج (الشكل 5.11)، بوها 21 (الشكل 5.20).

© جميع حقوق طبع ونشر الصور الأخرى محفوظة لصالح شركة بيرسون إديوكيشن

الوحدة 05

مقدمة في البرمجة

مقدمة

في هذه الوحدة، ستستكشف الميزات الرئيسية للغات البرمجة وتستخدمها لتطوير برنامج حاسوبي واختباره. وتبدأ بتعلم كيفية استخدام نماذج البرمجة بأساليبها (المبنية على الأحداث، الإجرائية، والبرمجة الكائنية) وتحديد أنواع البرامج الأنسب لها. كما تكتسب مجموعة من مهارات البرمجة من خلال تعلم مختلف التركيبات والتقنيات في لغة برمجة واحدة. وستستخدم هذه المهارات لتطوير برنامج لغرض معين. وبالتالي ستبحث عن طرق مختلفة لاختبار برنامج حاسوبي وتحسينه وتطبيق هذه التقنيات على برنامجك للتأكد من موثوقيته وقوته. بالإضافة إلى ذلك، ستتعرف على العوامل التي ينبغي مراعاتها أثناء مراجعة البرنامج وتطبيقها على برنامجك للتأكد من تلبيته لمتطلبات المستخدم ومناسبته للغرض.

أهداف التعلم

في هذه الوحدة، سوف تتمكن من:

(أ) فهم الميزات الرئيسية لنماذج البرمجة المختلفة

(ب) تطوير حل برنامج واختباره

(ج) مراجعة حل البرنامج

كيف سيتم تقييمك

يتم تقييم هذه الوحدة داخليًا من خلال موجزات الواجبات الفردية أو المتعددة التي يقدمها لك معلمك. وسيكون عليك تقديم أدلة لإثبات أنك حققت أهداف التعلم. يسرد مخطط الدرجات في المواصفات والواجب المحدد من قِبل بيرسون لهذه الوحدة ما يجب عليك القيام به للحصول على درجات النجاح والتفوق والامتياز. وستوجهك أنشطة التقييم في هذه الوحدة خلال المهام التي ستساعدك على تحقيق النجاح في هذه الوحدة.

سيخبرك معلمك بالضبط بالشكل الذي ستتخذه تقييماتك، ولكن قد يُطلب منك:

- إظهار فهمك لمختلف النماذج
- تطوير التعليمات البرمجية المكتوبة بلغة برمجة معينة واختباره ومراجعته.

الشكل 5.1 التعليمات البرمجية. هل يمكنك تحديد لغة البرمجة المستخدمة في هذه الصورة؟

مقدمة لهذه الوحدة

يتكون جهاز الحاسوب من نوعين رئيسيين من المكونات. الأول هو الأجهزة، التي تضم المكونات المادية الملموسة، مثل الماوس ولوحات المفاتيح والشاشات اللمسية. والثاني هو البرمجيات، المعروف أيضًا باسم البرامج أو التطبيقات. وتتيح البرامج للمستخدمين إكمال المهام أو الأنشطة على أجهزة الحاسوب، مثل تشغيل برنامج جداول البيانات أو لعب لعبة على الحاسوب.

الجدول 5.1 تقنيات البرمجة المستخدمة في هذه الوحدة

التقنيات	يمكنني مساعدة الآخرين أو إطلاعهم على كيفية إجراء ذلك	أحتاج إلى بعض المساعدة للقيام بذلك	أود ممارسة ذلك	هذا جديد بالنسبة لي
إعداد متغير أو ثابت				
استخدام عوامل حسابية ومنطقية مختلفة				
استخدام جمل الاختيار مثل if... elif... else				
استخدم التكرار مثل حلقات for وwhile وdo-while				
تخزين البيانات وتحريرها في بنية بيانات المصفوفة				

ما هي التعليمات البرمجية؟

لكي تعمل البرامج بشكلٍ صحيح، يجب أن تحتوي البرامج على مجموعة من التعليمات التي يجب اتباعها. على سبيل المثال، إذا نقر المستخدم فوق زر، يحتاج البرنامج ذي الصلة إلى تعليمات لإكمال مجموعة الإجراءات الصحيحة عند الضغط على هذا الزر.

جميع البرامج مصنوعة باستخدام **التعليمات البرمجية**. فهذه هي التعليمات التي تجعل البرامج تؤتي ثمارها. سيكتب **المبرمج** التعليمات البرمجية **بلغة برمجة** معينة. تتضمن أمثلة لغات البرمجة Python وJava وJavaScript وC وC++ وC#. وتحتوي كل لغة برمجة على مجموعة من المفردات والقواعد التي يجب على المبرمِج اتباعها عند كتابة الكود. ويُعرف هذا باسم **صيغة الكود**.

هناك المئات من لغات البرمجة المختلفة للاختيار من بينها. وهذا لأنه لا توجد لغة واحدة يتفق عليها الجميع، أو تلبي احتياجات الجميع. تحتوي كل لغة برمجة على أدوات وميزات وتقنيات مختلفة يمكن استخدامها لحل أنواع مختلفة من المشكلات. ويوضح الشكل 5.2 لغات البرمجة الأكثر شيوعًا في وقت كتابة هذا الكتاب.

بدء النشاط

يوضح الجدول 5.1 مختلف التقنيات التي ستحتاج إلى استخدامها في هذه الوحدة. قبل بدء هذه الوحدة، أكمل هذا الجدول لمساعدتك على التفكير في التعلم السابق. سيساعدك هذا على تقييم مهاراتك، لتحديد نقاط القوة لديك وتسليط الضوء على مجالات التنمية.

المصطلحات الرئيسية

التعليمات البرمجية: مجموعة من التعليمات التي يتبعها برنامج الحاسوب عند إكمال المهام.

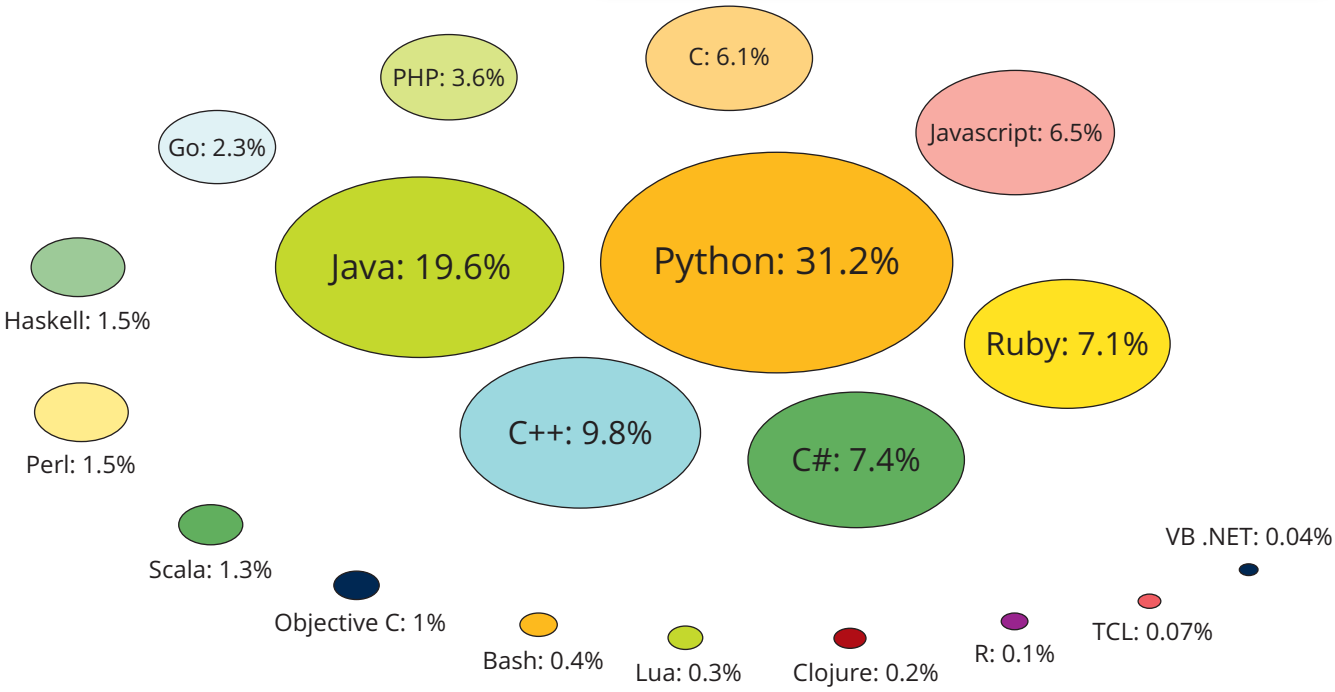
المبرمج: الشخص الذي يكتب التعليمات البرمجية بلغة برمجة معينة.

لغة البرمجة: تُستخدم لكتابة التعليمات البرمجية. وتحتوي كل لغة على قواعد بناء معينة يجب على المبرمِج اتباعها.

صيغة الكود: مجموعة من المفردات والقواعد في كل لغة برمجة معينة يجب على المبرمِجين اتباعها عند كتابة التعليمات البرمجية.

هل تعلم؟

يُعتقد أن أول مبرمج حاسوب كان امرأة تدعى آدا لوفلايس. في أربعينيات القرن التاسع عشر، كتبت آدا أول خوارزمية لمحرك تشارلز باباج التحليلي.



الشكل 5.2 لغات البرمجة الشائعة. هل استخدمتَ أيًا من لغات البرمجة هذه من قبل؟ ولماذا استخدمتها؟

هل تعلم؟

تعود لغات البرمجة بالفعل إلى أوائل القرن التاسع عشر. حتى الآن، تم إنشاء ما يقرب من 9000 لغة برمجة مختلفة، رغم أن غالبيتها العظمى لم تعد قيد الاستخدام اليوم.

نماذج البرمجة

نموذج البرمجة هو أسلوب أو طريقة لهيكلة التعليمات البرمجية. وتتضمن أمثلة نماذج البرمجة ما يلي:

- **النماذج القائمة على الأحداث**
- **البرمجة الكائنية**
- **النماذج الإجرائية**

تستخدم بعض لغات البرمجة نموذج برمجة معين، بينما قد تستخدم لغات أخرى أكثر من نموذج واحد. يعتمد اختيارك لنموذج البرمجة الذي تريد استخدامه على المهمة أو المشكلة التي تحاول حلها.

المصطلحات الرئيسية

نموذج البرمجة: أسلوب كتابة وهيكلة التعليمات البرمجية؛ تشمل الأمثلة النماذج القائمة على الأحداث البرمجة الكائنية والنماذج الإجرائية.

النماذج القائمة على الأحداث: إعطاء تعليمات للحاسوب بشأن كيفية الاستجابة لمختلف الأحداث.

البرمجة الكائنية (OOPs): إعطاء تعليمات لأجهزة الحاسوب استنادًا إلى عناصر العالم الحقيقي التي لها خصائص مشتركة. وتتفاعل هذه العناصر مع بعضها بعضًا لحل مشكلة.

النماذج الإجرائية: أخبر الحاسوب بما يجب فعله بناءً على تعليمات تدريجية.

هدف التعلم (أ): فهم الميزات الرئيسية لنماذج البرمجة المختلفة

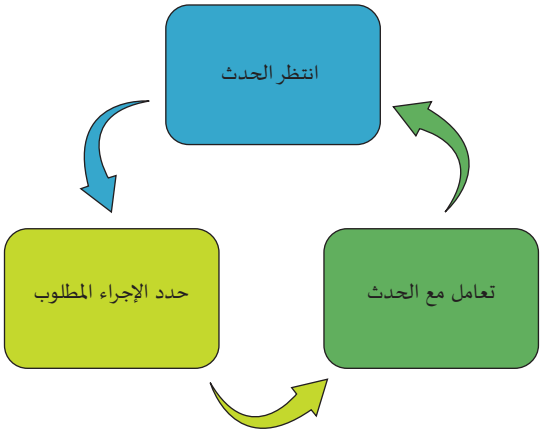
بدء النشاط

فكّر في برنامج أو تطبيق استخدمته مؤخرًا واكتب ثلاثة أحداث محتملة يمكن وقوعها أثناء استخدام البرنامج. فبالنسبة لكل حدث، اكتب كيفية استجابة البرنامج عند وقوع هذه الأحداث.

(أ1) ميزات البرمجة القائمة على الأحداث

توضح النماذج القائمة على الأحداث الحاسوب كيفية الاستجابة للأحداث. ويحتوي كل حدث محتمل على مجموعة من التعليمات الخاصة بهذا الحدث. ومن ثم ينتظر البرنامج وقوع الحدث ثم يُشغِّل التعليمات الخاصة بهذا الحدث المحدد. على سبيل المثال، عندما ينقر المستخدم فوق زر "حفظ"، سيشغّل البرنامج بعد ذلك التعليمات التي ستحفظ عمله.

يوضح الشكل 5.3 كيفية عمل النماذج القائمة على الأحداث. يكرر البرنامج هذه العملية باستمرار. وهذا يمكّنها من الاستجابة لجميع الأحداث المحتملة المختلفة التي يمكن أن تقع.



الشكل 5.3 كيفية عمل اللغات القائمة على الأحداث.

عادةً ما تُستخدم نماذج البرمجة القائمة على الأحداث عندما:

- تحتوي البرامج على الكثير من الأحداث
- يمكن أن تقع الأحداث بأي ترتيب
- تكون هناك واجهة مستخدم بعناصر مختلفة يمكن تحديدها
- يجب اتخاذ إجراءات أو تقديم ردود معينة استجابةً لإدخال المستخدم.

تتضمن بعض اللغات التي تدعم النماذج القائمة على الأحداث Visual Basic وVisual C++ وJavaScript. كما يدعم الكثير من البرمجة الكائنية النماذج القائمة على الأحداث.

المصطلحات الرئيسية

الحدث: إجراء يحتاج البرنامج إلى الاستجابة له.

واجهة المستخدم: تتيح للمستخدمين إدخال البيانات والأوامر في برنامج حاسوبي، غالبًا باستخدام نموذج إدخال المستخدم.

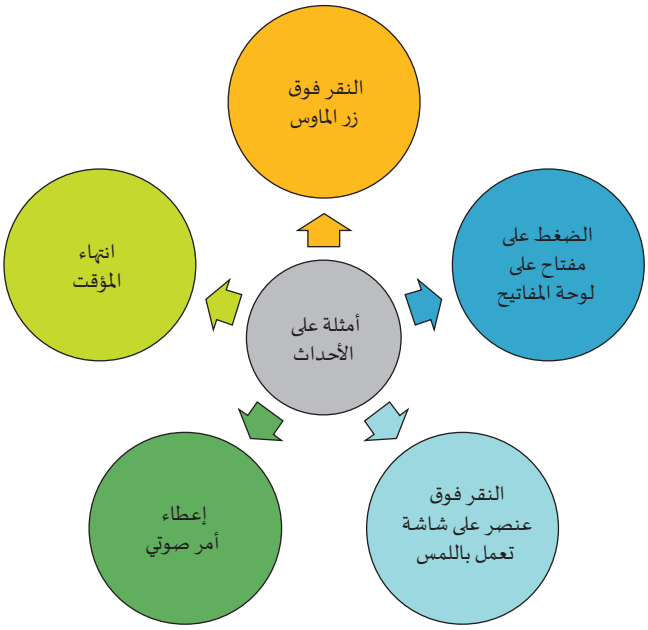
مشغّل الحدث: تحدد التعليمات البرمجية التي سيتم تشغيلها بمجرد اكتشاف الحدث.

الميزات الرئيسية

مشغّل الحدث

الحدث هو إجراء وقع بالفعل يتعين على البرنامج الاستجابة له. يلخص الشكل 5.4 مجموعة من مختلف أنواع الأحداث التي يمكن أن تقع في البرنامج. تتضمن العديد من الأحداث الموضحة في الشكل 5.4 **واجهة مستخدم**، والتي تتيح للمستخدم التفاعل مع البرنامج.

فبمجرد وقوع الحدث، سيحدد **مشغّل الحدث** بعد ذلك الكود الذي سيتم تشغيله. بعد ذلك، يؤدي هذا إلى حدوث شيء ما في البرنامج.



الشكل 5.4 أمثلة للأحداث. هل يمكنك التفكير في أي أمثلة أخرى للأحداث؟

معالجو الأحداث وموزعو رسائل الأحداث

يراقب **موزع رسائل الأحداث** الأحداث التي تقع في البرنامج. فعندما يكتشف موزع رسائل الأحداث وقوع حدث، فإنه سيستدعي **معالج الأحداث**. معالج الأحداث عبارة عن مجموعة من الأكواد، تُعرف باسم **الدوال الفرعية**، يتم تشغيلها للاستجابة لحدث ما. ويتولى الروتين الفرعي تنفيذ إجراء. وغالبًا ما يؤدي هذا إلى حدوث شيءٍ ما على الشاشة، مثل:

- ظهور رسالة تؤكد أن شيئًا ما كان ناجحًا أو غير ناجح
- نتيجة الحساب الظاهر
- تغير الألوان على واجهة المستخدم
- فتح نافذة مختلفة
- تشغيل الصوت

يعرض الشكل 5.5 جزءًا من شاشة التسجيل لصالة الألعاب الرياضية، قبل النقر فوق زر الإرسال وبعده. ففي العملية الموضحة، ينتظر موزع رسائل الأحداث حتى يتفاعل المستخدم مع نموذج إدخال المستخدم. وعندما ينقر المستخدم فوق مربع النص الخاص بالاسم الأول، يستجيب معالج الأحداث عن طريق وضع المؤشر في هذا المربع في انتظار المستخدم لكتابة اسمه. في وقتٍ لاحق من العملية، عند النقر فوق زر الإرسال، سيتحقق معالج الأحداث من إدخال البيانات.

قبل النقر فوق زر الإرسال

صفحة التسجيل

للتسجيل، يرجى إكمال النموذج أدناه.

الاسم الأول: براد

اسم العائلة: هدسون

عنوان البريد الإلكتروني:

إرسال

بعد النقر فوق زر الإرسال

صفحة التسجيل

للتسجيل، يرجى إكمال النموذج أدناه.

الاسم الأول: براد

اسم العائلة: هدسون

عنوان البريد الإلكتروني: *

إرسال

الشكل 5.5 مثال لصفحة التسجيل. هل يمكنك التفكير في أي أحداث أخرى محتملة يمكن أن تقع في صفحة التسجيل هذه؟

المصطلحات الرئيسية

موزع رسائل الأحداث: يراقب الأحداث التي تجري في برنامج حاسوبي وينتظرها.

معالج الأحداث: مجموعة من الأكواد التي سيتم تشغيلها استجابةً لحدث وقع في أحد البرامج.

الدوال الفرعية: مجموعة مستقلة من الأكواد يمكن للبرنامج استدعاؤها لإكمال مهمة.

على يمين الشكل يمكن أن ترى أنه عند إدخال البيانات الخاصة بالاسم الأول واللقب، فقد غيّر معالج الأحداث لون مربعات النص هذه إلى اللون الأخضر. ونظرًا لعدم إدخال عنوان البريد الإلكتروني، وضع معالج الأحداث علامة نجمة حمراء على الشاشة لتنبيه المستخدم بأن هذا مفقود.

مواضيع ذات صلة

لمعرفة المزيد عن مشغّلي الأحداث ومعالجة الأحداث، راجع **الوحدة 8: مقدمة لتطوير التطبيق**.

النشاط

1 تتضمن مجموعة واحدة من الأحداث استخدام الماوس. أعد إنشاء هذا الجدول وأكمله مع أحد الزملاء. بالنسبة لكل نوع من أحداث الماوس، ابحث عن تعريف ثم أعطِ مثالًا.

حدث الماوس	التعريف	مثال
أحداث النقر		
تحريك الماوس		
الإدخال بالماوس		
ترك الماوس		

2 تتضمن مجموعة أخرى من الأحداث الضغط على المفاتيح. أعد إنشاء هذا الجدول وأكمله مع أحد الزملاء. بالنسبة لكل نوع من أحداث الضغط على المفاتيح، ابحث عن تعريف ثم أعطِ مثالاً.

حدث الضغط على لوحة المفاتيح	التعريف	مثال
الضغط على مفتاح		
تحرير المفتاح		
المفتاح لأسفل		
المفتاح لأعلى		

المصطلحات الرئيسية

حلقة الأحداث: تستمع باستمرار للأحداث التي يتم تشغيلها ثم تستدعي معالج الأحداث المناسب عند اكتشاف حدث معين.

النهج من أعلى إلى أسفل: عند تقسيم البرامج إلى أقسام أصغر يمكن ترميزها وتحسينها بشكلٍ منفصل.

مخطط الهيكل: يقسم المشكلة بصريًا إلى مشكلات أصغر.

حلقات الأحداث

يراقب موزع رسائل الأحداث الأحداث التي تقع عن طريق التحقق مما إذا كان كل حدث محتمل يمكن أن يحدث أم لا. وفي حالة اكتشاف حدث ما، سيتم استدعاء معالج الأحداث المناسب.

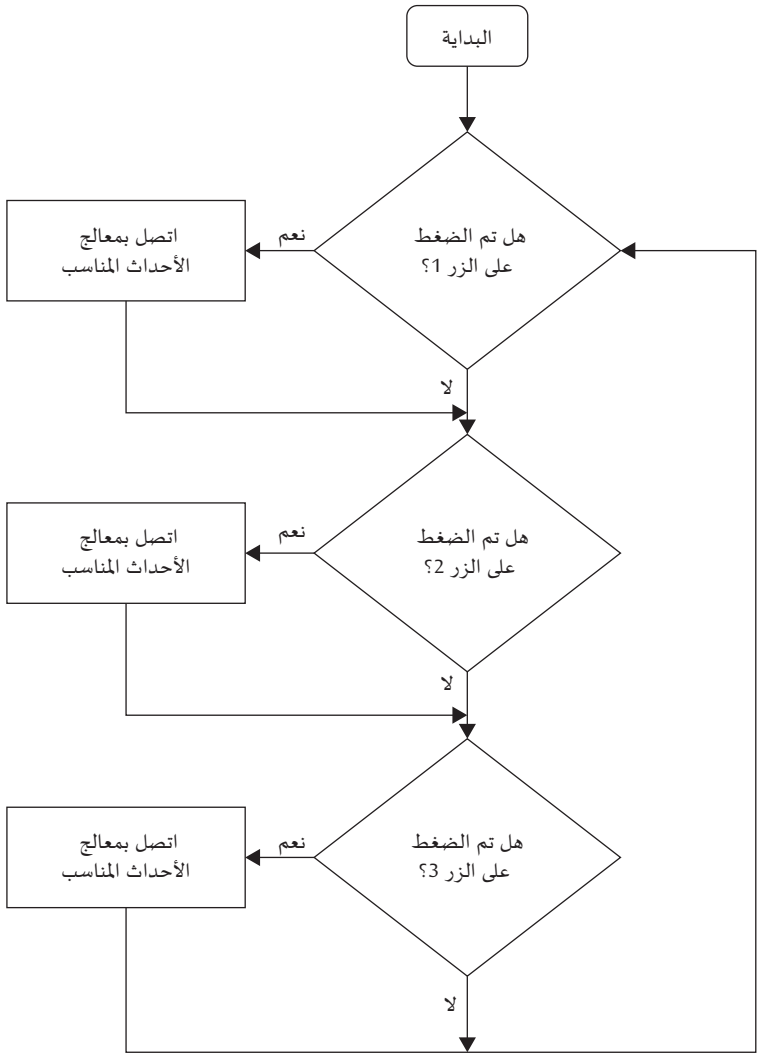
وعندما يتحقق موزع رسائل الأحداث من كل حدث محتمل، فإنه يحتاج بعد ذلك إلى التحقق مرة أخرى ومواصلة إجراء ذلك. يتم ذلك باستخدام **حلقة الحدث**. يوضح الشكل 5.6 كيفية عمل حلقة الحدث.

بدون حلقة الحدث، سيتحقق موزع رسائل الأحداث من كل حدث مرة واحدة فقط. ومع ذلك، باستخدام حلقة الحدث، سيستمر البرنامج في التحقق باستمرار من وقوع الأحداث. وهذا يتيح الرد على مدخلات المستخدم في أي وقت.

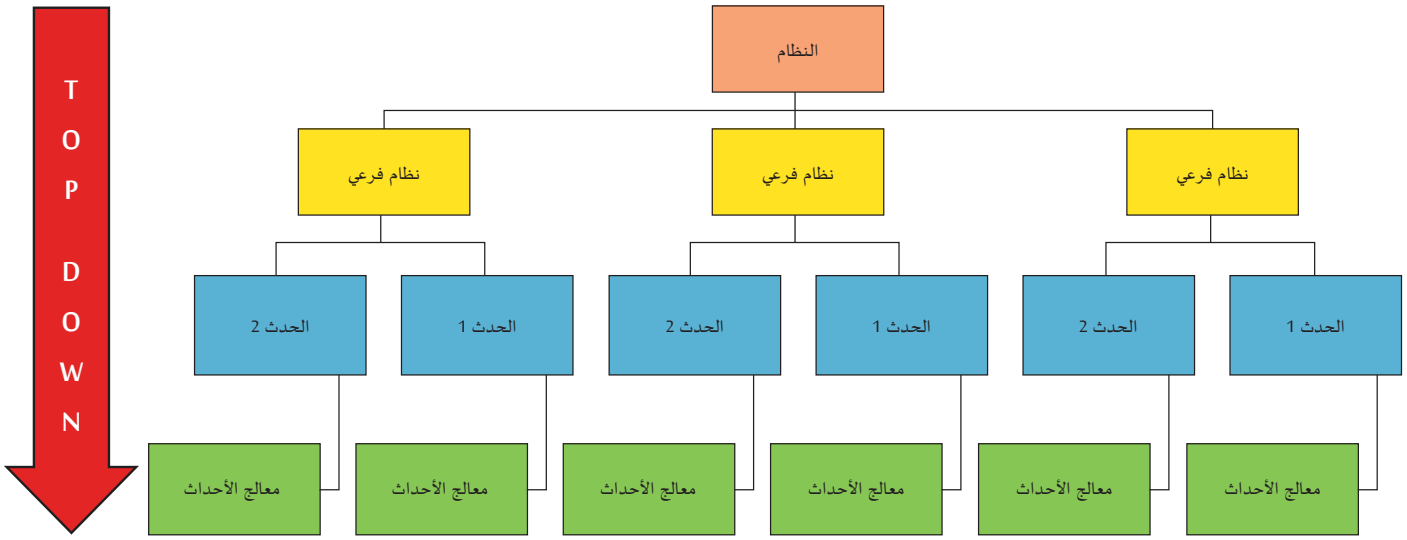
النهج من أعلى إلى أسفل

يتناسب نموذج البرمجة القائمة على الأحداث مع النهج **من أعلى إلى أسفل**. وهذا النهج هو الذي يتم فيه تقسيم البرنامج إلى أقسام فردية صغيرة. وقد يكون من الأسهل التفكير في النهج من أعلى إلى أسفل باستخدام **مخطط هيكلي** مثل الذي يظهر في الشكل 5.7.

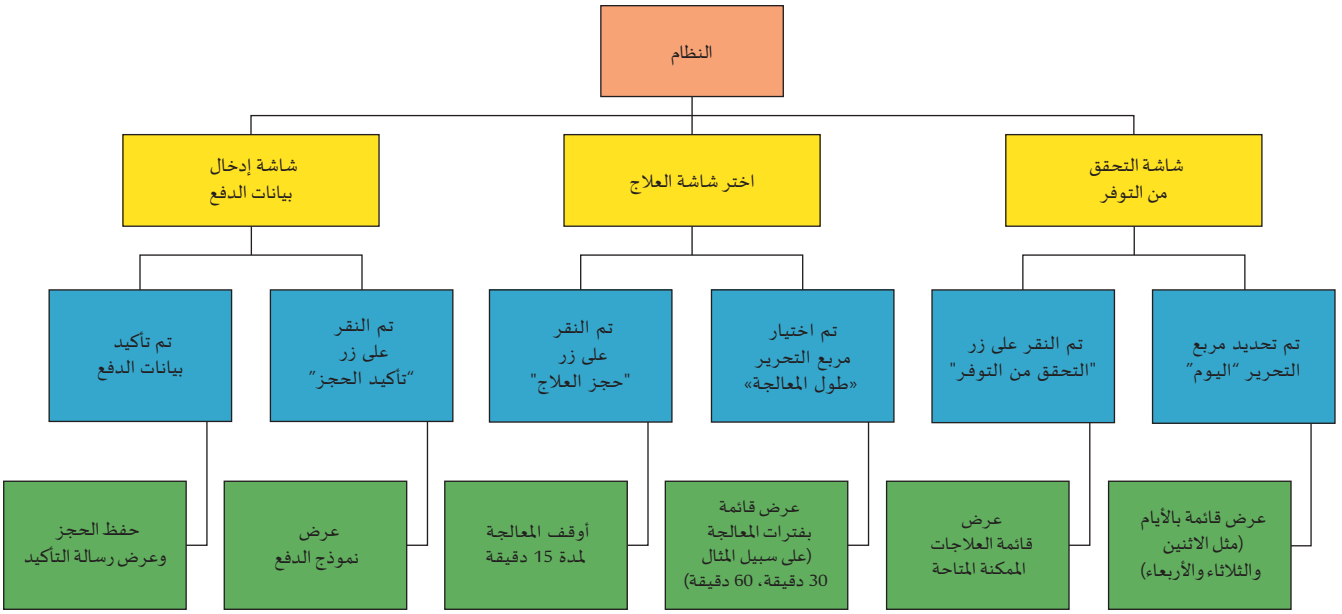
وفي المخطط الهيكلي الموضح في الشكل 5.7، تم تقسيم البرنامج إلى أنظمة فرعية مختلفة. كما ثم تم تقسيم كل نظام فرعي إلى أحداث مختلفة محتملة. فبمجرد إنشاء الأحداث، يمكن إنشاء معالجات الأحداث. وفي المستوى الأدنى، ستؤدي كل منطقة مهمة محددة للغاية في البرنامج. ويمكن ترميز هذه المهام واختبارها وتحسينها بشكلٍ منفصل.



يوضح الشكل 5.6 مخططًا انسيابيًا لكيفية عمل حلقة الحدث.



الشكل 5.7 النهج من أعلى إلى أسفل. هل يمكنك التفكير في أي مز ايا لاستخدام نهج من أعلى إلى أسفل عند كتابة البرامج؟



الشكل 5.8 نهج من أعلى إلى أسفل لنظام الحجز. هل يمكنك التفكير في أي برامج أخرى تستخدم هذا النهج؟

يوضح مخطط الهيكل في الشكل 5.8 هيكل نظام حجز علاج النادي الصحي. حيث يحتوي على واجهة مستخدم تتيح للمستخدم التحقق من العلاجات المتاحة واختيار العلاج ثم إدخال تفاصيل الدفع الخاصة به. في المستوى الأدنى، عند كتابة معالج أحداث واحد وتحسينه، بإمكان المبرمج الانتقال إلى القسم التالي وهكذا لحين اكتمال الكود.

المصطلحات الرئيسية

تصحيح الأخطاء: عندما يحاول المبرمج العثور على الأخطاء وإصلاحها في التعليمات البرمجية.

مزايا النماذج القائمة على الأحداث وعيوبها

عندما تقرر ما إذا كنت ستستخدم نموذجًا قائمًا على الأحداث، ينبغي لك أولاً التفكير في مزايا هذا النهج وعيوبه (راجع الجدول 5.2).

الجدول 5.2 النماذج القائمة على الأحداث: المزايا والعيوب

المزايا	العيوب
تُعد سهلة للتعلم: من السهل تعلم لغات البرمجة القائمة على الأحداث مقارنةً بنماذج البرمجة الأخرى مثل البرمجة الكائنية. وعادةً ما تكون التقنيات أقل تعقيدًا وتتيح لك الطريقة إنشاء عناصر جديدة بسهولة (مثل الأزرار) والتعليمات البرمجية لها.	**التأثير على موارد النظام:** غالبًا ما تستخدم النماذج القائمة على الأحداث الكثير من موارد النظام، بما في ذلك ذاكرة الحاسوب. وهذا لأنها تستخدم حلقة رئيسية تعمل باستمرار في الخلفية. ونظرًا لأن هذه النماذج مناسبة تمامًا لواجهات المستخدم، فإنها غالبًا ما تُستخدم مع عناصر رسومية كثيفة الاستهلاك للذاكرة.
سهولة التطوير: غالبًا ما تُوضع الأكواد لكل معالج أحداث في مجموعات مستقلة. ويمكن كتابة هذا الكود وتصحيحه بشكلٍ منفصل عن بقية الأكواد في البرنامج.	**قيود الترجمة:** غالبًا ما يصعب ترجمة لغات البرمجة القائمة على الأحداث إلى لغات برمجة أخرى. وهذا لأن الأحداث مثل نقرات الأزرار مبرمجة للعمل فقط مع نماذج البرمجة القائمة على الأحداث.
المرونة: بمجرد تطوير البرنامج، بإمكان المبرمج العودة بسهولة وتحرير الكود وتغيير شيء ما. وهذا لأنه يمكنه الانتقال مباشرةً إلى مجموعة واحدة من الأكواد وتحريرها، بدلاً من الاضطرار إلى المرور عبر عدة مجموعات من الأكواد.	**تدفق البرنامج:** ليس هناك تدفق برنامج محدد. ومن ثم تحتوي نماذج البرمجة الأخرى (مثل النموذج الإجرائي) على تدفق واضح للبرنامج نظرًا لأن التعليمات البرمجية يتم تشغيله بالترتيب. ومع ذلك، يتم تحديد تدفق البرنامج في نموذج قائم على الأحداث بترتيب الأحداث التي يتم تشغيلها من قِبل المستخدم.

(أ2) ميزات البرمجة الكائنية

تعتمد البرمجة الكائنية (OOP) على مفهوم عناصر العالم الحقيقي التي تتمتع بخصائص مشتركة.
ثم تتفاعل هذه العناصر مع بعضها بعضًا لتشكِّل حلاً لمشكلة المستخدم.

وعادةً ما تُستخدم نماذج البرمجة الكائنية في الحالات الآتية:

- تحتوي البرامج على عناصر من العالم الحقيقي مثل الأشخاص أو الأشياء أو الأماكن
- البرامج كبيرة أو معقدة
- وجود الكثير من الأكواد التي يجب إعادة استخدامها داخل البرنامج
- يُعد أمان البيانات في البرنامج أولوية عالية
- من المحتمل أن يتطلب الكود الكثير من التحديثات أو الصيانة في المستقبل.

تتضمن بعض اللغات التي تدعم نموذج البرمجة الكائنية Python وC# وC++ وJava وJavaScript.

بدء النشاط

أحد أمثلة النموذج الموجه للعناصر هو النموذج الذي يحتوي على عشرة أشياء، يخزن كل منها تفاصيل شخصية عن أعضاء مختلفين في صالة الألعاب الرياضية. وتحتوي جميعها على خصائص مشتركة لأن كل عنصر (عضو) سيخزن اسم الشخص وعنوانه وتاريخ ميلاده. هل يمكنك التفكير في مثال آخر لكيفية استخدام مثل هذا العنصر؟

الميزات الرئيسية

تجريد البيانات

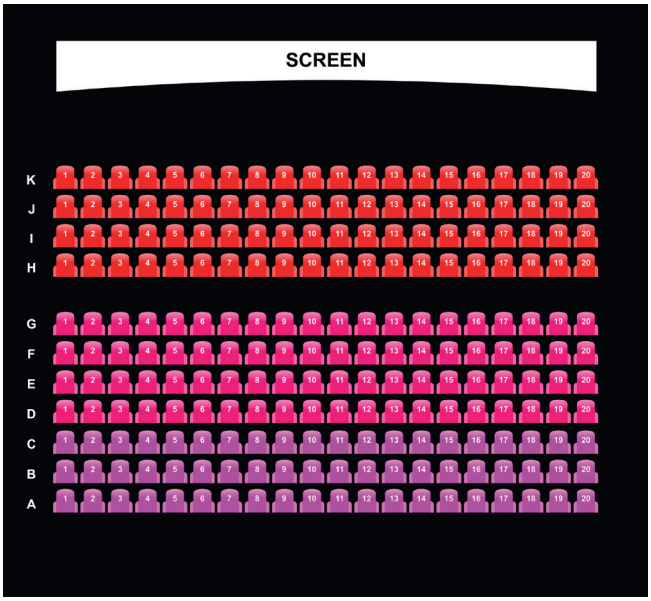
يتضمن تجريد البيانات إزالة التفاصيل غير الضرورية من المشكلة من أجل تمثيل شيء ما في أبسط أشكاله. وسيُخفي التجريد أي تعقيدات غير ضرورية للبرنامج.

على سبيل المثال، يعرض الشكل 5.9 برنامج حجز قاعة المؤتمرات، والذي يتميز بعرض تجريدي لخطة الجلوس.
فالميزات الأساسية هي موقع الشاشة والمقاعد، والتي تظهر في المخطط.
تم تجاهل التفاصيل الأخرى مثل:

- نوع السماعات وموقعها
- لون السجادة
- علامات الاحتكاك على المقاعد.

المصطلحات الرئيسية

تجريد البيانات: يُقصد به إزالة التفاصيل غير الضرورية من مشكلة للسماح للمبرمِجين بالتركيز على الجوانب الأساسية لها للبرنامج.



الشكل 5.9 عرض تجريدي للجلوس في قاعة المؤتمرات. هل يمكنك التفكير في أي تفاصيل أخرى غير مدرجة؟

عند انتهاء عملية التجريد، ينبغي معرفة أهم عناصر المشكلة التي تتعامل معها. ويمكنك بعد ذلك استخدام هذه المعرفة لإنشاء فئات.

المصطلحات الرئيسية

الفئات: عندما يتم وضع البيانات في كائنات بناءً على خصائصها أو خصائصها.

الكائن: يحدد المتغيرات والطرق في البرنامج وتعمل كقالب لإنشاء الدوال.

المتغيرات: جزء من البيانات أو القيمة المحفوظة في البرنامج.

الطرق (البرمجة): مجموعات التعليمات التي ستنفذ الإجراءات في البرنامج.

مخطط الكائنات: يُستخدم لتخطيط الكائنات المستخدمة في نموذج موجه للدوال - يعرض كل مخطط العلاقات مع الكائنات الأخرى في البرنامج نفسه.

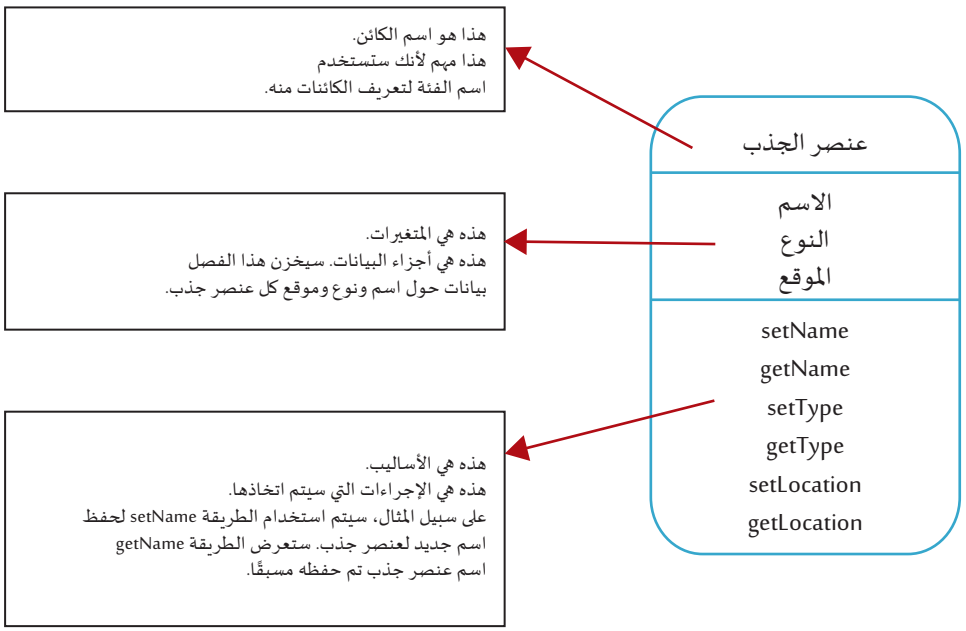
الدوال: مكونات مستقلة في برنامج يمكن استخدامه لتمثيل الأشخاص أو الأماكن أو الأشياء أو المفاهيم.

الفئات والكائنات

الفئات هي عندما تضع البيانات في فئات بناءً على بياناتها أو خصائصها. على سبيل المثال، قد يكون لديك كائن يسمي الشخص وسيكون لجميع الأشخاص في هذا الكائن الخصائص نفسها. على سبيل المثال، سيكون لكل شخص في الكائن الاسم الأول واللقب وعنوان البريد الإلكتروني. فبمجرد فصل البيانات إلى كائنات، يمكنك بعد ذلك البدء في تطوير كائنات لتخزين البيانات المتعلقة بها.

يحدد **الكائن** المتغيرات في البرنامج. و**المتغيرات** هي جزء من البيانات أو قيمة داخل الكائن. كما يحدد الكائن **الطرق**. وهذه هي مجموعات التعليمات التي ستنفذ الإجراءات في البرنامج. وسيكون الكائن بمنزلة قالب لبناء الدوال في البرنامج (راجع الشكل 5.11).

فعندما تُصمم فئةً، قد يكون من المفيد إنشاء مخطط **للفئة**. ومن ثم يسلط هذا الضوء على المتغيرات والطرق التي سيحتوي عليها الكائن. على سبيل المثال، مخطط الكائنات الموضح في الشكل 5.10 مخصص للعبة محاكاة مدينة الملاهي. سيُستخدم الكائن الموضح في هذا المخطط لأنواع مختلفة من مناطق الجذب مثل الألعاب والمطاعم ومتاجر الهدايا.



الشكل 5.10 دوال مخطط الكائنات.

الدوال

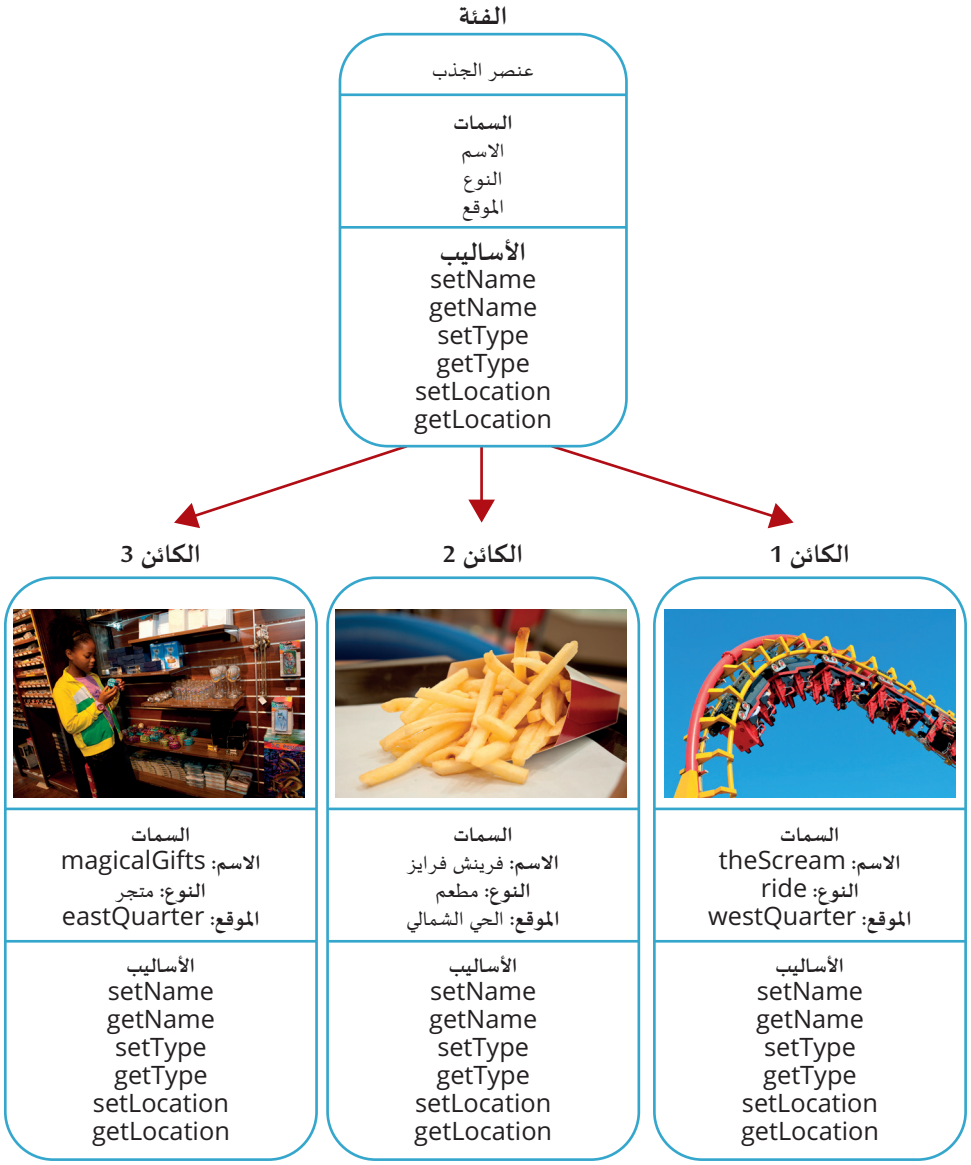
الدوال هي مكونات مستقلة في برنامج يمكن استخدامه لتمثيل الأشخاص أو الأماكن أو الأشياء أو المفاهيم. على سبيل المثال، قد يحتوي مرآب السيارات الذي يقدم خدمة إصلاح السيارات على دوال للعمال والسيارات. وفي مجال الجراحة البيطرية، قد يحتوي على دوال بيطرية وحيوانية.

يمكن الآن استخدام مخطط الكائنات الخاص بلعبة محاكاة مدينة الملاهي (على النحو الموضح في الشكل 5.10) لإنشاء دوال مختلفة سيتم وضعها في اللعبة. على سبيل المثال، يمكن أن تكون هذه الأشياء عبارة عن ألعاب الركوب والمطاعم ومحلات بيع الهدايا. ويوضح الشكل 5.11 كيف يمكن إنشاء كل هذه الدوال من الكائن الموضح في الشكل 5.10.

هل تعلم؟

يعمل النموذج الموجه للعناصر على فكرة أن العناصر الفعلية يمكن تمثيلها كتعليمات حاسوب.

يمكن تخزين كل شيء من الأشخاص والأماكن والأشياء والمفاهيم كسمات وطرق.



الشكل 5.11 يمكن إنشاء عناصر مختلفة من الفئة ذاتها. هل يمكنك اقتراح عنصر آخر يمكن إنشاؤه من هذه الفئة؟

الطرق

الطريقة هي مجموعة من أكواد البرمجة التي ستنفذ مهمة أو إجراء معين. ويتم تعريف الطريقة داخل فئة ويمكن استخدامها من قِبل أي عنصر من تلك الفئة. ويمكن أن تكون الطريقة:

- **طريقة المنشئ** - أسلوب الروتين الخاص يتم استدعاؤه عند تعريف كائن Constructor
- **طريقة Getter** - تُستخدم لقراءة البيانات في سمة
- **طريقة Setter** - تُستخدم لتعديل البيانات في سمة

الوراثة

حتى الآن، لقد تعرفتَ على إنشاء فئات فردية فقط. ومع ذلك، عند استخدام نموذج البرمجة الكائنية، ستحتاج غالبًا إلى إنشاء عدة فئات. فعند إنشاء أكثر من فئة واحدة، يمكن أن تكتسب من بعضها بعضًا. **الوراثة** يُقصد بها عندما تأخذ (تكتسب) إحدى الفئات السمات والطرق المحددة بالفعل في فئة أخرى، مثل فئة فائقة أو فئة رئيسية أو فئة فرعية (راجع الجدول 5.3).

المصطلحات الرئيسية

الوراثة: عندما تأخذ إحدى الفئات السمات والطرق المحددة بالفعل في فئة أخرى.

الجدول 5.3 مختلف مستويات الفئات

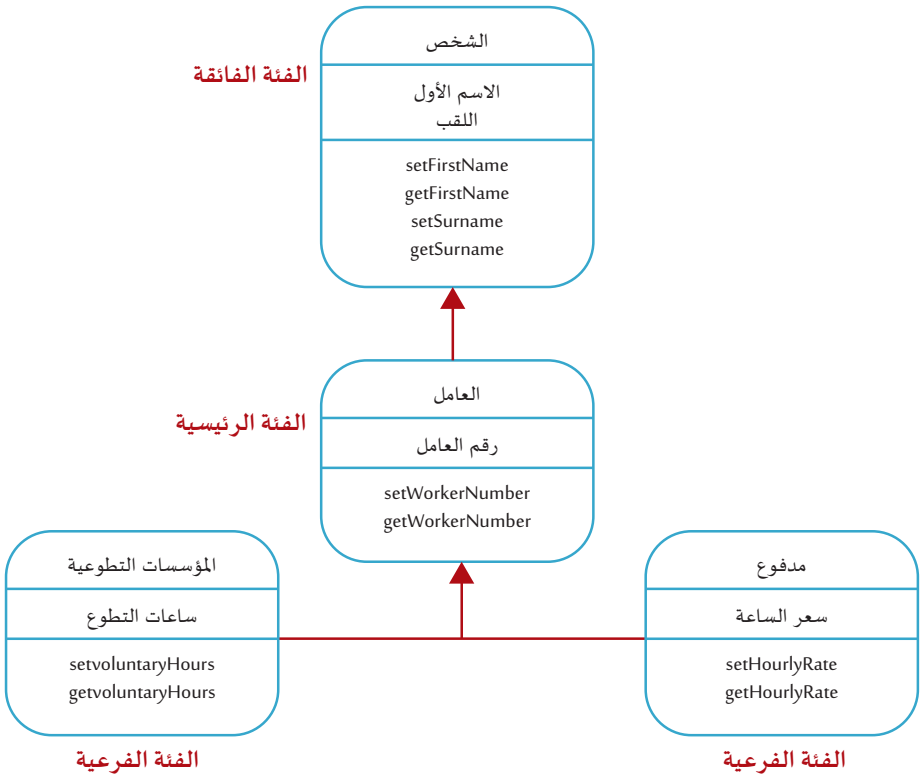
النوع	المعنى
الفئة الفائقة	هذه هي الفئة الرئيسية التي تكتسب منها جميع الفئات الأخرى السمات والطرق.
الفئة الرئيسية	ستكتسب هذه الفئة السمات والطرق من فئة أخرى، غالبًا ما تكون الفئة الفائقة.
الفئة الفرعية	ستكتسب هذه الفئة السمات والطرق من فئة رئيسية أو فئة فائقة.

يعرض مخطط الفئات الموضح في الشكل 5.12 العديد من الفئات التي تستخدم الوراثة. ويمثل المخطط مؤسسة لديها العديد من العمال، بعضهم يتقاضى أجرًا والبعض الآخر متطوع.

ستكتسب جميع الفئات في الشكل 5.12 السمات والطرق من فئة الشخص الفائقة. كما تكتسب الفئة المدفوعة من فئة العامل الرئيسية. وفي مخطط الفئات هذا، في حالة إنشاء عنصر من الفئة المدفوعة، سيتضمن جميع السمات والطرق من الفئة المدفوعة وفئة العامل وفئة الشخص. ولن يتمكن العنصر من الوصول إلى الطرق الموجودة في الفئة التطوعية.

هل تعلم؟

لعبت برمجة الحاسوب دورًا أساسيًا في المساعدة على إنهاء الحرب العالمية الثانية. استخدم رجل يدعى آلان تورينج مهاراته في التكويد وحل المشكلات لكسر آلة إنجما.



الشكل 5.12 يمكن للفئات المختلفة الوراثة من بعضها بعضًا.

تعددية الأشكال

تعددية الأشكال يُقصد به عندما يتصرف شيء ما بشكلٍ مختلف في المواقف المختلفة. على سبيل المثال، يعرض الجدول 5.4 اثنين من الأكواد التي تستخدم الرمز + (الإضافة) بشكلٍ مختلف.

الجدول 5.4 مثال على تعددية الأشكال

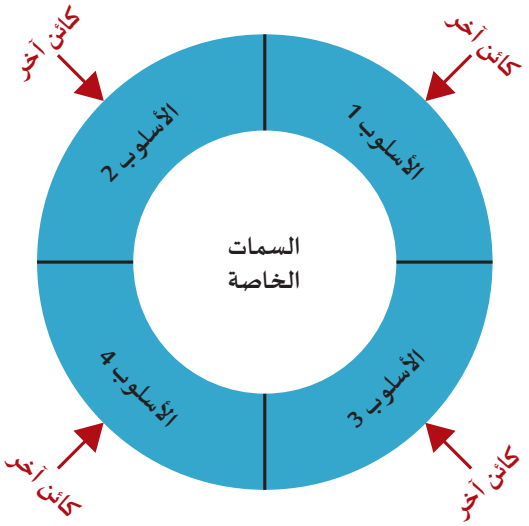
	المثال 1	المثال 2
التعليمات	num1 = 2 num2 = 3 answer = num1+num2	"word1 = "Good "word2 = "Morning answer = word1+word2
النتيجة	5	صباح الخير

في المثال 1، تُستخدم علامة + لجمع الرقمين 2 و3 معًا. ومع ذلك، في المثال 2، تُستخدم علامة + لربط كلمتين معًا. ومن ثم تضمن تقنية تعدد الأشكال مراعاة الاستخدام الصحيح لعلامة + في كل سياق محدد.

وفي البرمجة الكائنية، يمكن أن يكون تعدد الأشكال مفيدًا جدًا عندما تكون إحدى الفئات مكتسبة من فئة أخرى. على سبيل المثال، باستخدام مخطط الفصل الموضح في الشكل 5.12، تكتسب الفئة التطوعية من فئة العامل. ويعني هذا أنها تكتسب الطريقة setWorkerNumber. ومع ذلك، تخيل أن الفئة التطوعية أرادت تكييف هذه الطريقة وإضافة حرف V قبل رقم العامل للإشارة إلى أن العامل متطوع. ويعني هذا وجود نسختين من الطريقة setWorkerNumber. لذا، سيكون هناك إصدار واحد في فئة العامل وستوجد نسخة واحدة في الفئة التطوعية. فباستخدام تعددية الأشكال، عند استدعاء الطريقة setWorkerNumber، سيعرف التعليمات البرمجية الإصدار الذي يجب استخدامه اعتمادًا على العنصر الذي يستدعي الطريقة.

التغليف

يتكون نموذج البرمجة الكائنية من عناصر، حيث يكون لكل عنصر مجموعة السمات والطرق الخاصة به. ويحمي **التغليف** سمات العنصر عن طريق جعلها خاصة. ويعني هذا عدم إمكانية الوصول إلى السمات مباشرةً من قِبل عناصر أخرى. وهذا موضح في الشكل 5.13.



الشكل 5.13 تتفاعل العناصر والطرق من أجل تغيير السمات الخاصة.

المصطلحات الرئيسية

تعددية الأشكال: عندما يمكن أن تعني التقنية أشياءً مختلفة في فئات أو عناصر مختلفة. و"ستعرف" هذه التقنية تلقائيًا المعنى الصحيح لشيءٍ ما بناءً على سياق التعليمات.

التغليف: يُقصد به عندما تصبح السمات خاصة بحيث لا يمكن تغييرها إلا بالطرق الموجودة في الفئة ذاتها.

وعند جعلها خاصة، لا يمكن تغيير السمات إلا بالطرق الموضحة في الفئة ذاتها. وبإمكان هذه الطرق التحقق من البيانات قبل تغيير السمات. على سبيل المثال: لا يمكن تغيير سمة تسمى TelephoneNumber إلا بطريقة تسمى setTelephoneNumber. فقبل تغيير السمة، يمكن للطريقة التحقق من أن رقم الهاتف هو الطول الصحيح. ويعني هذا أنه يتم تغيير السمات بطريقة مقصودة.

المصطلحات الرئيسية

النهج التصاعدي (Incremental Approach): النهج التصاعدي هو استراتيجية تطوير تهدف إلى تقسيم البرنامج إلى وحدات صغيرة قابلة للفهم والتطوير، ثم تكوين هذه الوحدات بشكل تدريجي لتشكل البرنامج الكامل. هذا يُعرف أيضًا بمفهوم التطوير التدريجي

النمطية والنهج التصاعدي

نموذج البرمجة الكائنية مناسب **للنهج التصاعدي**. ففي هذا النهج، يحدد المبرمِجون أصغر المشكلات ويحلونها أولاً. حيث تُعرف هذه المشكلات الصغيرة بالوحدات. وستؤدي كل وحدة مهمة محددة وصغيرة جدًا. ثم يتم دمج كل هذه الوحدات النمطية الصغيرة معًا لتشكيل برنامج أكبر يحل أشياء أكثر تعقيدًا ويتيح هذا الأسلوب إعادة استخدام الأكواد. وهذا لأنه بمجرد كتابة الكود في وحدة واحدة، يمكن إعادة استخدامه في وحدات أخرى من البرنامج. لذا، تتمثل الفائدة الرئيسية للنهج التصاعدي في تقليل مقدار التعليمات البرمجية الذي يجب تكراره.

هذا هو السبب في أن النهج التصاعدي مناسب تمامًا للبرمجة الكائنية. على سبيل المثال، في نموذج البرمجة الكائنية، ستطوّر فئة أولاً. ويُعرف هذا باسم المستوى الأدنى. فبمجرد كتابة الفئة واختبارها، يمكنك بعد ذلك إنشاء فئة ثانية. ومع ذلك، بإمكان الفئة الثانية اكتساب السمات والطرق من الفئة الأولى. بعد ذلك، يمكن تكرار ذلك مع الفئة الثالثة فصاعدًا لحين حل المشكلة.

مزايا البرمجة الكائنية وعيوبها

عندما تقرر ما إذا كنت تريد استخدام البرمجة الكائنية أم لا، ينبغي لك أولاً التفكير في مزايا هذا النهج وعيوبه، على النحو الموضح في الجدول 5.5.

الجدول 5.5 البرمجة الكائنية: المزايا والعيوب

المزايا	العيوب
البنية المعيارية: العناصر التي تم إنشاؤها في البرمجة الكائنية كلها مستقلة. فمع التغليف، لا يمكن أن يتداخل عنصر واحد مع عنصر آخر. لذا، يمكن إخفاء السمات من الأجزاء الأخرى من التعليمات البرمجية لإبقائها أكثر أمانًا.	**حجم البرنامج:** البرمجة الكائنية مناسبة للبرامج الكبيرة التي تحتاج إلى التحديث والصيانة. فهي غير مناسبة حقًا للبرامج الصغيرة. وهذا لأن التعليمات البرمجية يجب تخطيطه بشكلٍ صحيح، ويمكن أن يستغرق ذلك وقتًا طويلاً للكتابة ويتطلب تقنيات متقدمة لا تناسب دائمًا البرامج الأصغر.
استكشاف الأخطاء وإصلاحها بشكلٍ أسهل: نظرًا لتقسيم الكود إلى فئات مختلفة، يسهُل تحديد مكان المشكلة. على سبيل المثال، إذا كان العنصر الحيواني لا يعمل، فيجب أن تكون المشكلة ضمن فئة الحيوانات.	**التكويد الأكثر تعقيدًا:** تُستخدم البرمجة الكائنية العديد من التقنيات المتقدمة الخاصة بالبرمجة الكائنية. ويشمل ذلك تقنيات مثل الوراثة والتغليف وتعددية الأشكال، مما يجعل التعليمات البرمجية أكثر تعقيدًا.
إعادة استخدام الكود: بمجرد إعداد فئة مثل الفئة الفائقة، يمكن لفئة أخرى، مثل الفئة الفرعية، اكتساب السمات والطرق منها. وبالتالي يقلل هذا من مقدار التعليمات البرمجية المطلوب عند تطوير برامج أكبر.	**مستوى مهارة أعلى:** غالبًا ما يحتاج المبرمِجون إلى مستوى مهارة أعلى للكتابة في البرمجة الكائنية، عند مقارنتها بالنماذج الأخرى. كما يُعد نموذج البرمجة الكائنية أكثر صعوبة في التعلم مقارنةً بالنماذج الأخرى.

(أ3) ميزات البرمجة الإجرائية

تخبر نماذج البرمجة الإجرائية الحاسوب بما يجب القيام به بناءً على تعليمات تدريجية. حيث تعمل على تقسيم البرنامج إلى وحدات مختلفة. ويتم دمج هذه الوحدات لتشكِّل حلاً لمشكلة المستخدم.

وعادةً ما تستخدم نماذج البرمجة الإجرائية عندما:

- تعمل البرامج بطريقة مرتبة تدريجية
- لا تستخدم المشكلة العناصر الواقعية التي تحتاج إلى التفاعل مع بعضها بعضًا
- لا يوجد الكثير من الأكواد التي يجب إعادة استخدامها داخل البرنامج
- لا يمثل أمان البيانات داخل البرنامج أولوية عالية
- يتطلب الكود القليل من التحديثات أو الصيانة أو لا يتطلب أي تحديثات في المستقبل.

تشمل بعض اللغات التي تدعم النموذج الإجرائي Python و C.

بدء النشاط

مثال على برنامج تدريجي يمكن إعداده باستخدام نموذج إجرائي هو الآلة الحاسبة. سيؤدي ذلك إلى أخذ رقمين من المستخدم وإجراء الحساب المطلوب ثم عرض النتيجة. هل يمكنك التفكير في مثال آخر لبرنامج يعمل بترتيب تدريجي؟

الميزات الرئيسية

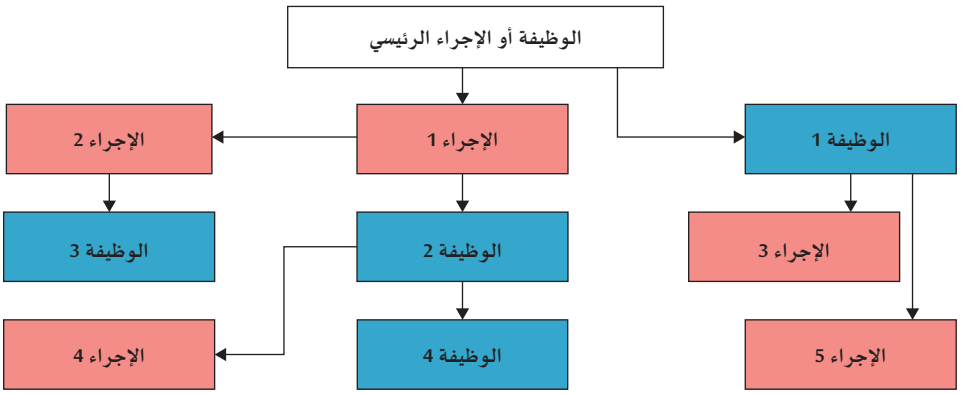
البنية

تتكون النماذج الإجرائية من الكثير من الإجراءات الفرعية المختلفة على النحو الموضح في الشكل 5.14. هذه الإجراءات الفرعية هي إما إجراءات أو دوال. ومن ثم يُطبَّق إجراء أو دالة رئيسية لبدء تشغيل البرنامج. بعد ذلك، سيؤدي هذا إلى استدعاء إجراءات أو دوال أخرى وسيتم تمرير أجزاء من البيانات فيما بينها من أجل توفير استجابة لمدخلات المستخدم.

المصطلحات الرئيسية

الإجراءات: الإجراءات الفرعية التي يمكن استدعاؤها لإكمال المهمة - لا تُرجِع القيم مرة أخرى إلى أجزاء أخرى من البرنامج.

الدوال: الإجراءات الفرعية التي يمكن استدعاؤها لإكمال المهمة - ستُرجِع القيمة مرة أخرى إلى أجزاء أخرى من البرنامج.

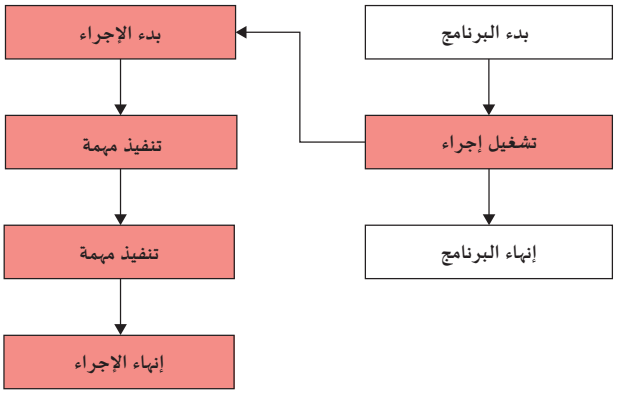


الشكل 5.14 هيكل البرامج الإجرائية.

الإجراءات

الإجراء عبارة عن مجموعة واحدة من الأكواد وهو نوع من الروتين الفرعي (راجع الشكل 5.15). عند الاتصال من نقطة أخرى في البرنامج، يتم إكمال المهمة ثم التحكم اليدوي مرة أخرى إلى سطر الكود الذي استدعاها.

ولن يعمل الإجراء على إرجاع قيمة. على سبيل المثال، بإمكان المستخدم إدخال خمسة أرقام يتم إرسالها بعد ذلك إلى الإجراء. بعد ذلك، يتولى الإجراء فرز الأرقام وطباعتها بالترتيب من الأصغر إلى الأكبر. وتُعرض الأرقام التي تم فرزها على الشاشة، ولكن لن يتم إرجاعها مرة أخرى إلى سطر الكود الذي يسمى الإجراء.



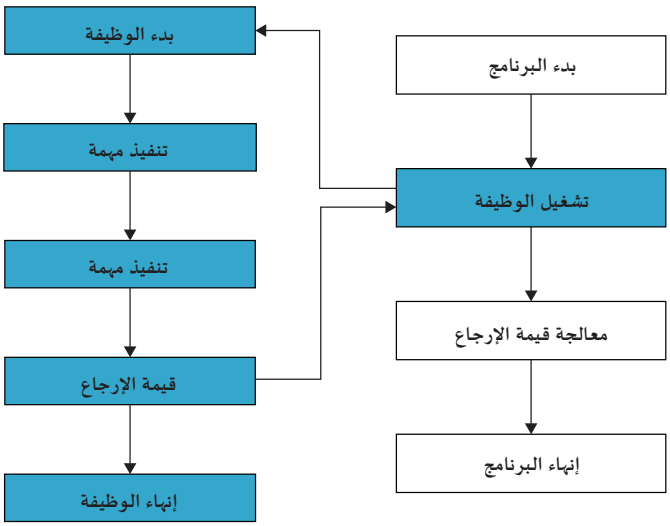
الشكل 5.15 كيف يعمل الإجراء.

الدوال

الدالة هي مجموعة أخرى من الأكواد ونوع الروتين الفرعي (راجع الشكل 5.16). عند الاتصال من نقطة أخرى في البرنامج، يتم إكمال المهمة ثم التحكم اليدوي مرة أخرى إلى سطر الكود الذي استدعاها. وستقوم الدالة بإرجاع قيمة. على سبيل المثال، إذا أكملت الدالة عملية حسابية، فسيتم إرجاع القيمة التي تم حسابها إلى السطر الذي يسمى الدالة.

هل تعلم؟

لا يمكن للحاسوب فهم التعليمات البرمجية الذي نكتبه بالفعل. هذا لأنه يجب تحويل جميع أكواد البرمجة إلى سلسلة من 1s و0s أولاً حتى يتمكن الحاسوب من معالجتها.



الشكل 5.16 كيف تعمل الدالة.

الدوال المحددة مسبقًا

تأتي نماذج البرمجة الإجرائية مع الكثير من **الدوال المحددة مسبقًا**. وهذه هي الدوال التي تمت كتابتها بالفعل ويتم توفيرها من قِبل لغة البرمجة. وهي توفر حلولًا لمهام البرمجة الشائعة، على سبيل المثال:

- قراءة المدخلات من المستخدم
- عرض الإخراج على الشاشة
- استرداد التاريخ الحالي
- التحويل من نوع بيانات إلى آخر
- فرز قوائم البيانات بالترتيب
- البحث عن قيمة في قائمة البيانات

المصطلحات الرئيسية

الدوال المحددة مسبقًا: مجموعات الأكواد المكتوبة مسبقًا والمضمنة في لغة البرمجة - فهي تكمل مهام محددة.

على سبيل المثال، تحتوي لغة Python على العديد من الدوال المحددة مسبقًا مثل:

- **round()** - ستؤدي هذه إلى تقريب الرقم إلى عدد محدد من المنازل العشرية
- **open()** - ستؤدي هذه إلى فتح ملف خارجي
- **read()** - ستؤدي هذه إلى قراءة سطر من ملف
- **write()** - ستؤدي هذه إلى كتابة سطر جديد إلى ملف
- **min()** - ستجد هذه أصغر قيمة في القائمة
- **max()** - ستجد هذه أكبر قيمة في القائمة

ينطوي استخدام الدوال المحددة مسبقًا على العديد من المزايا، بما في ذلك:

- تقليل وقت البرمجة نظرًا لوجود كود أقل للكتابة
- تقليل وقت الاختبار لأن الكود سيكون قد تم اختباره بالفعل
- يلزم معرفة تقنية أقل.

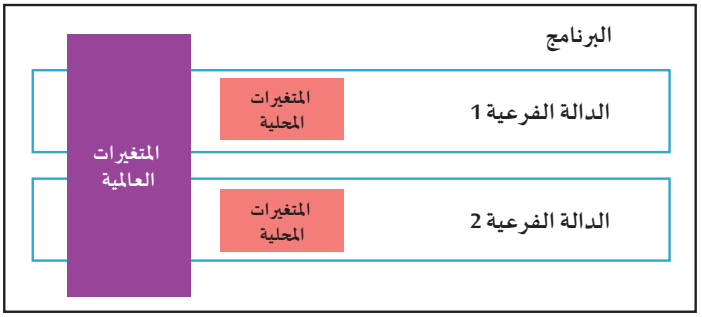
المتغيرات المحلية والعالمية

تتكون النماذج الإجرائية من الكثير من الإجراءات الفرعية المختلفة، والتي يمكن أن تكون إجراءات أو دوال. ويستدعي الإجراء أو الدالة الرئيسية التي يتم تحميلها في بداية البرنامج إجراءات أو دوال أخرى. ويمكن تعريف المتغيرات داخل هذه الإجراءات الفرعية المختلفة. ففي بعض الأحيان تحتاج المتغيرات فقط إلى رؤيتها من خلال دالة فرعية معينة، ولكن في بعض الأحيان يجب رؤية المتغيرات من خلال إجراءات فرعية متعددة. لذا سيحتاج المبرمِجون إلى استخدام **المتغيرات المحلية** و**المتغيرات العالمية** (راجع الشكل 5.17).

المصطلحات الرئيسية

المتغيرات المحلية: المتغيرات التي تم إنشاؤها داخل الدوال الفرعية والتي تكون مرئية فقط في هذا الدوال الفرعية.

المتغيرات العالمية: المتغيرات التي تم إنشاؤها خارج الدوال الفرعية والتي يمكن الوصول إليها من خلال جميع أجزاء البرنامج.



الشكل 5.17 رؤية المتغيرات المحلية والعالمية في البرنامج.

يتم إنشاء متغير محلي داخل الدوال الفرعية ويكون مرئيًا في هذا الروتين الفرعي فقط. ويعني هذا أنه لا يمكنك الوصول إلى القيمة داخل متغير محلي إلا إذا كنت تحاول الوصول إليها من داخل الروتين الفرعي. يوضح الجدول 5.6 مزايا المتغيرات المحلية وعيوبها.

الجدول 5.6 مزايا المتغيرات المحلية وعيوبها

المزايا	العيوب
تُخزَّن المتغيرات المحلية فقط في الذاكرة أثناء تشغيل الروتين الفرعي. وتتم إزالتها عند انتهاء الروتين الفرعي.	لا يمكن الوصول إلى البيانات إلا ضمن الروتين الفرعي الذي تم إنشاؤها فيه. فإذا احتاجت وحدة أخرى إلى رؤية القيمة، فلا يمكنها إجراء ذلك.
تعمل المتغيرات المحلية على تسهيل إصلاح الأخطاء لأن القيمة في الدوال الفرعية واحد لن تؤثر على الدوال الفرعية آخر.	يمكن أن تكون البرمجة أكثر تعقيدًا حيث ستحتاج إلى مواصلة تمرير القيم بين مختلف الإجراءات الفرعية.

عادةً ما يتم إنشاء متغير عمومي خارج الدوال الفرعية ويكون مرئيًا في جميع أنحاء البرنامج. ويعني هذا أن جميع أجزاء الكود بما في ذلك جميع الإجراءات الفرعية يمكنها الوصول إلى القيمة داخل متغير عالمي. يوضح الجدول 5.7 مزايا المتغيرات العالمية وعيوبها.

أفضل ممارسة

إذا كنت تستخدم المتغيرات العالمية، ينبغي أن تأخذ في الاعتبار تأثير هذا القرار. وينبغي لك التحقق من أنها لن تؤثر على أجزاء أخرى من البرنامج أو على المبرمجين الآخرين الذين يعملون في المشروع نفسه.

الجدول 5.7 مزايا المتغيرات العالمية وعيوبها

المزايا	العيوب
يجب الإعلان عن القيم مرة واحدة فقط ويمكن الوصول إليها عدة مرات من إجراءات فرعية متعددة.	يصعب اختبار مجموعة واحدة من الأكواد، حيث سيحتاج المبرمج إلى تشغيل الكود بأكمله لإنشاء المتغيرات العالمية.
يقلل وقت البرمجة لأنك لست بحاجة إلى تمرير المتغيرات بين مختلف الإجراءات الفرعية.	إذا غيّر أحد الإجراءات الفرعية المتغير العام، فيمكن أن يؤثر ذلك على دالة فرعية أخرى يصل أيضًا إلى المتغير العام نفسه.

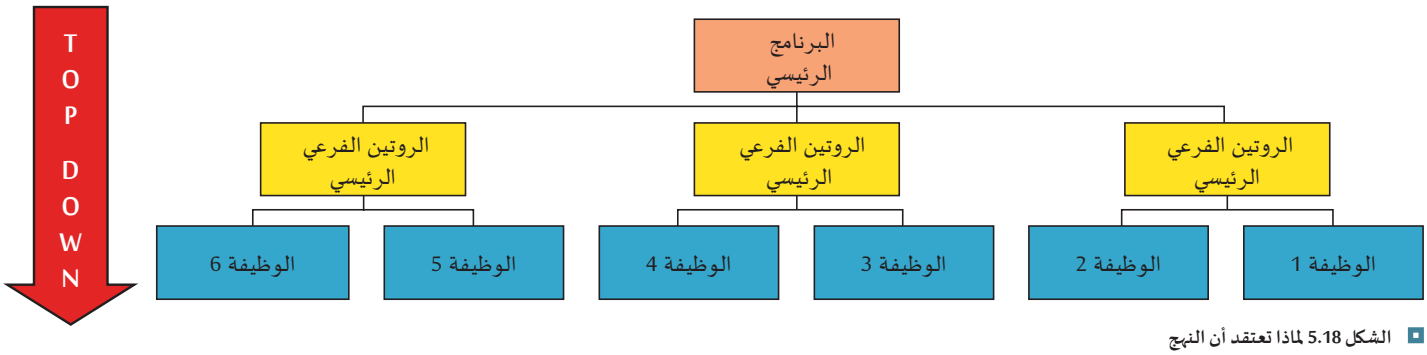
النشاط

فريق مكون من 30 مبرمِجًا على وشك بدء العمل في مشروع جديد. وأنت المبرمِج الرئيسي وتريد التأكد من أن الجميع يسمي المتغيرات ويستخدمها بطريقة متسقة.

اكتب **خمس** قواعد ستعطيها لجميع المبرمِجين الذين يعملون في هذا المشروع. فبالنسبة لكل قاعدة، اذكر سبب اختيارك لها.

النهج من أعلى إلى أسفل

لقد تعلمتَ أن النهج من أعلى إلى أسفل هو النهج الذي يتم فيه تقسيم البرنامج إلى أقسام صغيرة فردية. وتعد لغة البرمجة الإجرائية مناسبة تمامًا لهذا النوع من النهج. وقد يكون من المفيد التفكير في النهج من أعلى إلى أسفل باستخدام مخطط هيكلي مثل الذي يظهر في الشكل 5.18.



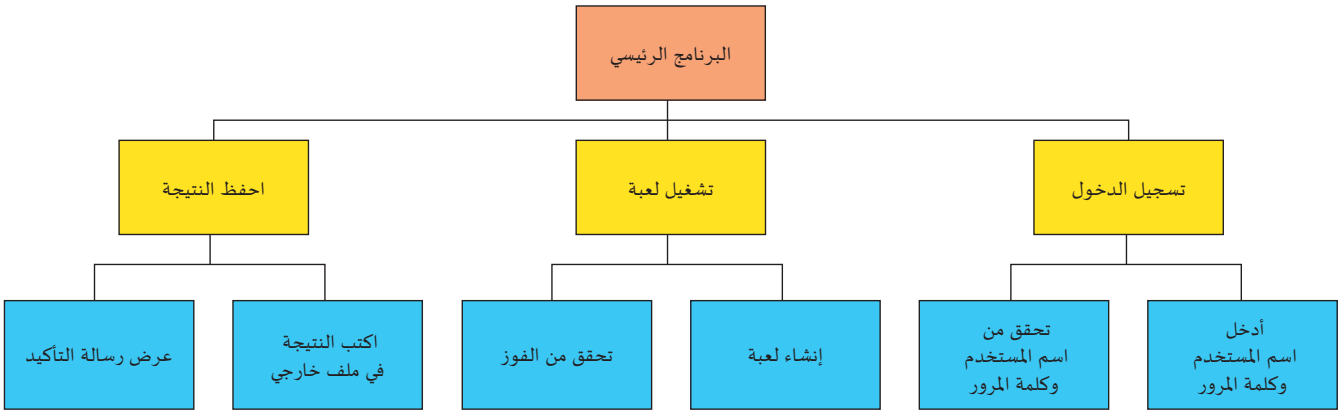
الشكل 5.18 لماذا تعتقد أن النهج من أعلى إلى أسفل لن يكون مناسبًا للنموذج الموجه للعناصر؟

في مخطط الهيكل في الشكل 5.18، تم تقسيم البرنامج إلى إجراءات فرعية مختلفة. فعند تحميل البرنامج للمرة الأولى، سيتم تحميل البرنامج الرئيسي. بعد ذلك، سيتم تقسيم البرنامج الرئيسي إلى إجراءات فرعية أصغر، وهي دالات أو إجراءات. وفي المستوى الأدنى، تضطلع كل دالة أو إجراء بأداء مهمة محددة جدًا كجزء من البرنامج. ومن ثم يمكن ترميز هذه الدوال والإجراءات واختبارها وصقلها بشكلٍ منفصل عن بعضها بعضًا.

على سبيل المثال، يعرض مخطط الهيكل الموضح في الشكل 5.19 بنية لعبة X - O. حيث يتضمن ثلاثة أقسام رئيسية تتيح للمستخدم تسجيل الدخول ولعب اللعبة ثم حفظ نتيجته. وهذه الأقسام كالآتي:

- يتكون قسم تسجيل الدخول من دالة تتيح للمستخدم إدخال تفاصيل تسجيل الدخول الخاصة به، ودالة أخرى تتحقق من هذه التفاصيل.
- يحتوي قسم تشغيل اللعبة على دالة من شأنها إنشاء لعبة جديدة ثم دالة أخرى ستتحقق مما إذا كان المستخدم قد فاز بعد كل خطوة.
- يتكون قسم حفظ النقاط من دالة ستكتب نقاط كل مستخدم إلى ملف خارجي، ودالة أخرى تعرض رسالة تأكيد.

في المستوى الأدنى، عند كتابة مقطع واحد من الكود وتحسينه، بإمكان المبرمج الانتقال إلى القسم التالي وهكذا لحين اكتمال الكود.



الشكل 5.19 هل يمكنك التفكير في أي دوال أو إجراءات أخرى يمكن إضافتها إلى مخطط الهيكل هذا؟

دراسة الحالة

اقرأ المعلومات أدناه بشأن شركة خيالية تقع في الأردن تسمى شركة برمجة عالمية. ثم ناقش الأسئلة في مجموعات أو مع أحد الزملاء.

شركة برمجة عالمية

شركة البرمجة العالمية هي شركة تطوير برمجيات. لديها عشرة مبرمجين يعملون جميعًا معًا في مشاريع البرمجة. طُلب من الفريق مؤخرًا العمل على مشروع يتضمن عمل لعبة حاسوب.

لعبة الحاسوب هي لعبة كلاسيكية للبحث عن الكنز حيث يتحرك اللاعب على خريطة افتراضية ويجمع الأشياء. ستحتوي الخريطة على عشر غرف مختلفة ويجب على اللاعب جمع الأشياء من جميع الغرف العشر من أجل الفوز باللعبة. لدى اللاعب 75 دقيقة فقط لإكمال اللعبة.

في كل مرة يتم فيها تحميل اللعبة، سيتم إخفاء الكنز في مكان مختلف.

يتبع دراسة الحالة

سيتم ترميز كل غرفة داخل دالة وسيكتب كل مبرمج التعليمات البرمجية لدالة واحدة. بعد ذلك، يدمج المبرمجون الأكواد الخاصة بهم معًا لإنشاء اللعبة الشاملة. فإذا نجحت اللعبة، فإنهم يخططون لإنشاء 100 غرفة إضافية.

وقبل أن يبدأ أفراد فريق البرمجة بكتابة التعليمات البرمجية، فقد اتفقوا على القواعد التالية.

- سيخططون للعبة باستخدام نهج من أعلى إلى أسفل.
- سينشؤون لعبة الحاسوب باستخدام لغة برمجة Python.
- يكتب كل منهم التعليمات البرمجية باستخدام نموذج إجرائي.
- سيستخدمون المتغيرات المحلية والعالمية في اللعبة.

الأسئلة

1. راجع القواعد التي وافق عليها فريق البرمجة.
   هل تعتقد أن هذه القواعد مناسبة للعبة البحث عن الكنز؟
2. أي من هذه القواعد ستغيّرها ولماذا؟
3. ما القواعد الأخرى التي تعتقد أن فريق البرمجة ينبغي أن يراعيها؟

المهارات

مهارات التواصل الشخصي: العمل الجماعي والتعاون:

- التواصل
- التعاون
- العمل الجماعي
- التعاطف/تحديد المنظور

مزايا النماذج الإجرائية وعيوبها

قبل اتخاذ قرار بشأن استخدام نموذج إجرائي، ينبغي مراعاة مزايا هذا النهج وعيوبه (راجع الجدول 5.8).

الجدول 5.8 النماذج الإجرائية: المزايا والعيوب

المزايا	العيوب
أسهل في البرمجة غالبًا ما تكون النماذج الإجرائية أسهل في الترميز مقارنةً بالنماذج الأخرى (مثل البرمجة الكائنية). وهذا لأن الكود يميل إلى اتباع نمط تدريجي وغالبًا ما يكون بناء جملة اللغات الإجرائية أسهل في الاستخدام والتعلم.	**يصعب توسيع البرنامج** يمكن أن تزيد النماذج الإجرائية من صعوبة توسيع البرنامج. وهذا لأنها تفتقر إلى التقنيات التي تستخدمها النماذج الأخرى لإعادة استخدام الكود. ويعني هذا أنه غالبًا ما يكون هناك الكثير من الأكواد المكررة. وبالتالي يمكن أن يجعل هذا من الصعب الحفاظ على الكود منظمًا بطريقة جيدة.
تستلزم ذاكرة أقل يمكن أن تستخدم النماذج الإجرائية ذاكرة أقل من النماذج الأخرى (مثل النماذج القائمة على الأحداث). ويرجع هذا إلى أن اللغات الإجرائية لا تستخدم حلقة رئيسية تعمل في الخلفية. فنظرًا لأن النماذج الإجرائية أقل ملاءمةً للاستخدام مع واجهة المستخدم، فإنها لا تحتاج دائمًا إلى تشغيل واجهات مستخدم رسومية مكثفة.	**يصعب صيانة الكود** لا تستخدم اللغات الإجرائية التغليف. ويعني هذا أنه عند إجراء تغييرات في جزء واحد من البرنامج، يمكن أن يكون لها عواقب على جزء آخر من البرنامج. ويزيد هذا من مخاطر الأخطاء في الأكواد. بالإضافة إلى ذلك، نظرًا لأن اللغات الإجرائية لا تستخدم الفئات والعناصر، يمكن أن يبدو الكود في كثيرٍ من الأحيان أقل تنظيمًا.

مراجعة ما تعلمته ✔

1. ما المقصود بمصطلح نموذج البرمجة؟
2. صفِ المقصود بمصطلحات حلقة الحدث والمشغّل والمعالج وموزع رسائل الأحداث.
3. صفِ المقصود بمصطلحات الفئة والعنصر والاكتساب وتعدد الأشكال والتغليف.
4. صفِ المقصود بمصطلحات الإجراء والدالة والمتغير المحلي والمتغير العام.
5. وضّح متى ستستخدم نموذجًا إجرائيًا قائمًا على الأحداث وموجهًا للعناصر.

إجراء التقييم هدف التعلم (أ)

السيناريو

يرغب المبرمج في تطوير لعبة حيوانات أليفة تتيح للاعب ما يلي:

- اختيار نوع من الحيوانات الأليفة (على سبيل المثال كلب، قط، أرنب، هامستر).
- تسمية الحيوان الأليف
- اللعب مع الحيوان الأليف
- إطعام الحيوان الأليف

ينبغي أن تحتوي اللعبة على دوال متعددة اللاعبين. وينبغي أن يتيح هذا للاعبين الآخرين اختيار حيواناتهم الأليفة.

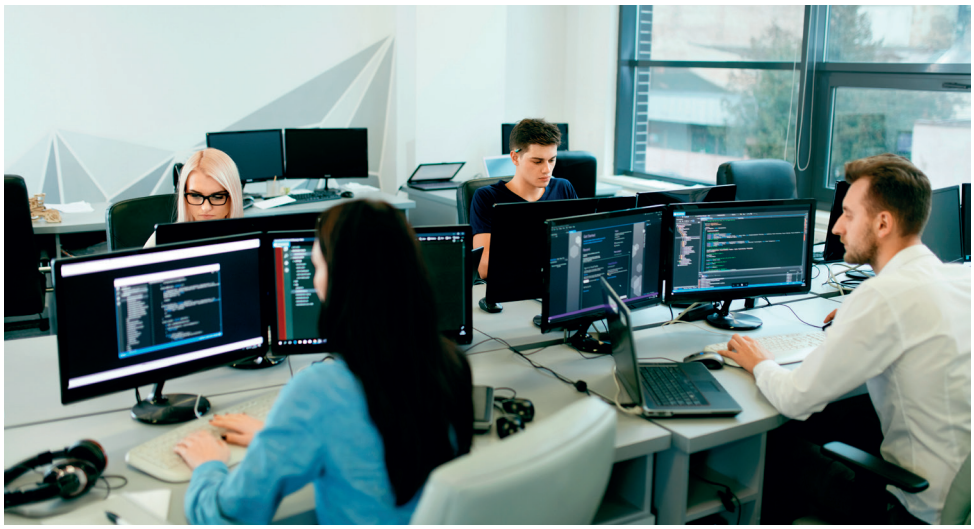
كما ينبغي أن تتمكن الحيوانات المختلفة أيضًا من اللعب مع بعضها بعضًا في عالم افتراضي.

المهمة

قيّم مدى ملاءمة نماذج البرمجة القائمة على الأحداث والبرمجة الكائنية والإجرائية لهذه اللعبة. بالنسبة لكل نموذج، ينبغي لك:

- وصف الميزات التي تحتوي عليها والتي لها صلة بلعبة الحيوانات الأليفة
- وصف مزايا كل منها وعيوبه في سياق لعبة الحيوانات الأليفة
- تبرير سبب ملاءمة كل نموذج برمجة أو عدم ملاءمته

ينبغي أن تعطي استنتاجًا يوصي بأفضل نموذج للعبة



الشكل 5.20 فريق من المبرمِجين يعملون على مشروع برمجة مشترك.

هدف التعلم (ب): تطوير حل برنامج واختباره

تستخدم التعليمات البرمجية للبرامج تقنيات مختلفة لتنفيذ الإجراءات.

وفي هذا القسم، ستتعرف على التقنيات المختلفة التي يمكن استخدامها لتطوير البرامج وتحسينها.

وفي هدف التعلم هذا، تتم كتابة التعليمات البرمجية بلغة Python وC# وJava. فمن المهم ملاحظة أن طريقة تصميم هذا الكود وتقديمه لا تتجاوز كونها مثال. فهناك العديد من الطرق المختلفة التي تتيح تصميم التعليمات البرمجية ويتم تحديد ذلك بشكلٍ أساسي من خلال الإعدادات داخل بيئة التطوير المتكامل.

بدء النشاط

راجع الجدول 5.9 للحصول على بعض الأمثلة على أسطر التعليمات البرمجية Python. اكتب الإجراء الذي تعتقد أن كل جزء من التعليمات البرمجية يقوم به.

(ب1) تطوير البرمجيات

ملاحظة: عند إكمال مهمتك لهدف التعلم هذا، سيُطلب منك استخدام لغة برمجة **واحدة** فقط. يمكنك اختيار اللغة التي تريد استخدامها لكتابة التعليمات البرمجية. ويتضمن هذا القسم أمثلة مختلفة التعليمات البرمجية المكتوبة بلغة C# (C Sharp) و Python وJava. ولا يُتوقع منك معرفة كل هذه اللغات. كما يمكنك اختيار استخدام إحدى لغات البرمجة هذه أو قد تفضل استخدام لغة مختلفة لإنجاز مهمتك.

الجدول 5.9 مثال كود Python

	سطر الكود
1	الاسم = الإدخال ("أدخل اسمك: ")
2	الإجمالي = num1 + num2
3	إذا كان الرقم > 0: الطباعة ("الرقم إيجابي.")
4	إذا كان الرقم% 2 == 0: الطباعة ("الرقم زوجي ")

بيئات التطوير المتكاملة

عند كتابة التعليمات البرمجية، يحتاج المبرمِجون إلى استخدام **بيئة التطوير المتكامل (IDE)**.

وهذا هو البرنامج الذي يفهم لغة البرمجة المستخدمة. ومن ثم يتيح للمبرمِجين كتابة الكود وتحريره واختباره.

تتضمن بيئة التطوير المتكامل الكثير من الأدوات المختلفة التي من شأنها مساعدة المبرمِجين عند كتابة الأكواد. وتشمل ما يلي:

- **الإكمال التلقائي** - ستحاول هذه الميزة إكمال البيانات نيابةً عنك. على سبيل المثال، عند فتح أقواس على سطر الكود، سيتم إغلاقها تلقائيًا نيابةً عنك.
- **المسافة البادئة التلقائية** - تضع هذه الميزة مسافة بادئة تلقائيًا للتعليمات البرمجية في الأماكن الصحيحة. على سبيل المثال، إذا كنتَ تستخدم عبارة IF، فستضع تلقائيًا مسافة بادئة للكود المرتبط بشرط IF.
- **الخطوة** - تتيح هذه الميزة للمبرمِجين تشغيل سطر كود واحد في كل مرة للتحقق من أن البرنامج يعمل بشكلٍ صحيح.
- **نافذة مراقبة المتغيرات** - تعرض قائمة بجميع **المتغيرات** و**الثوابت** في نافذة منفصلة للسماح للمبرمِجين بتتبع قيمهم أثناء تشغيل البرنامج.
- **نقاط التوقف** - تتيح هذه الميزة إيقاف الكود عند نقطة معينة أثناء تشغيله كي يتمكن المبرمِج من التحقق من صحة أي متغيرات.

المصطلحات الرئيسية

بيئة التطوير المتكامل (IDE): برنامج يستخدمه المبرمِجون لكتابة الأكواد وتصحيحها.

المتغيرات: أجزاء مفردة من البيانات التي تم حفظها في ذاكرة الحاسوب والتي يمكن تغييرها في التعليمات البرمجية.

الثوابت: أجزاء مفردة من البيانات التي تم حفظها في ذاكرة الحاسوب والتي لا يمكن تغييرها في التعليمات البرمجية.

التركيبات والتقنيات

المتغيرات والثوابت

المتغير هو جزء واحد من البيانات التي تم حفظها في ذاكرة الحاسوب. ويمكن الوصول إلى قيمتها من خلال أجزاء أخرى من التعليمات البرمجية وتغييرها.

الثابت هو أيضًا جزء واحد من البيانات التي تم حفظها في ذاكرة الحاسوب. ويمكن الوصول إلى القيمة التي يتم الاحتفاظ بها بشكلٍ ثابت من خلال أجزاء أخرى من التعليمات البرمجية، ومع ذلك، لا يمكن تغييرها أثناء تشغيل البرنامج.

يجب إعطاء المتغيرات والثوابت على حدٍ سواء أسماء فريدة تُعرف أحيانًا باسم المعرّفات. فهناك قواعد عامة ينبغي اتباعها عند تسمية المتغيرات والثوابت.

يتضمن ذلك أن يكون الاسم:

- معقول ومرتبط بالبيانات التي يتم تخزينها
- يبدأ بحرف وليس برقم
- لا يحتوي على مسافات
- ليس كلمة محفوظة
- يتبع اصطلاح تسمية مناسب.

يعرض الجدول 5.10 بعض أمثلة اصطلاحات التسمية.

الجدول 5.10 اصطلاحات التسمية الشائعة

اسم الاصطلاح	الوصف	أمثلة
Camel case	تتم كتابة الحرف الأول من كل كلمة بأحرف كبيرة باستثناء الكلمة الأولى. تتم إزالة المسافات بين كل كلمة	firstName dateOfBirth
Snake case	تتم كتابة كل كلمة بأحرف صغيرة وتُستبدل المسافات بشرطة سفلية	first_name date_of_birth
Kebab case	تتم كتابة كل كلمة بأحرف صغيرة وتُستبدل المسافات بشرطة	first-name date-of-birth

المصطلحات الرئيسية

الكلمة المحجوزة: كلمة لا يمكن للمبرمج استخدامها لتسمية عناصر مثل المتغير. فالكلمة المحجوزة لها معنى خاص في لغة البرمجة المستخدمة.

اصطلاح التسمية: طريقة متفق عليها لتسمية الأشياء، مثل المتغيرات أو الدوال، في البرنامج.

نوع البيانات: يحدد هذا كيفية حفظ البيانات في برنامج (على سبيل المثال، كرقم أو نص).

عند إنشاء متغير أو ثابت، ستحتاج دائمًا إلى ذكر اسمه ثم استخدام عامل التعيين (=) لتعيين قيمة له. ففي بعض لغات البرمجة، تحتاج أيضًا إلى تحديد نوع البيانات.

ومن ثم تختلف طريقة إنشاء متغير أو ثابت اعتمادًا على لغة البرمجة التي تستخدمها. فالأمثلة (على اليمين) هي من ثلاث لغات برمجة مختلفة وتوضح كيفية إعداد متغير يسمى الرقم وتعيين القيمة 2 له.

C#	`int number = 2;`
Python	`number = 2`
Java	`int number = 2;`

أنواع البيانات

يحدد نوع البيانات كيفية تخزين البيانات في التعليمات البرمجية. وسيكون لكل جزء من البيانات المخزنة في متغير أو ثابت نوع بيانات محدد.

وغالبًا ما يُطلب منك تحديد نوع البيانات عند إعداد متغير أو ثابت. في بعض اللغات، يكون نوع البيانات الافتراضي عبارة عن سلسلة. يسرد الجدول 5.11 على اليمين أنواع البيانات الشائعة التي يمكن العثور عليها في لغات البرمجة.

الجدول 5.11 أنواع البيانات الشائعة

نوع البيانات	المعنى	مثال
حرف	يُستخدم هذا لتخزين حرف واحد أو رقم أو علامات ترقيم أو رمز	(أ) &
سلسلة	يُستخدم هذا لتخزين أي مجموعة من الأحرف أو الأرقام أو علامات الترقيم أو الرموز	جيسون 10 شارع دي لا بومب
عدد صحيح	يُستخدم هذا لتخزين الأرقام الصحيحة فقط	18 1–
حقيقي	يُستخدم هذا لتخزين الأرقام ذات المنزلة العشرية	12.9 99.123
منطقي	يُستخدم هذا لتخزين القيم التي يمكن أن تكون صحيحة أو خاطئة فقط.	صواب خطأ

المصطلحات الرئيسية

العوامل الحسابية: تُستخدم لإجراء الحسابات الأساسية مثل عمليات الجمع والطرح والضرب والتقسيمات.

العوامل المنطقية: تُستخدم لإجراء عمليات حسابية أكثر تعقيدًا مثل مقارنة جزء من البيانات بجزء آخر منها.

العوامل الحسابية

تُستخدم عوامل التشغيل لتنفيذ الإجراءات. فهناك نوعان من العوامل هما **العوامل الحسابية** و**العوامل المنطقية**. وتُجري العوامل الحسابية العمليات الحسابية الأساسية. يوضح الجدول 5.12 بعض العوامل الحسابية الشائعة.

الجدول 5.12 العوامل الحسابية الشائعة

العامل الحسابي	الرمز	مثال
جمع	+	3 = 2 + 1
طرح	–	2 = 7 – 9
ضرب	*	42 = 7 * 6
قسمة	/	5 = 2 / 10
المعامل	%	1 = 3 % 10

يعمل المعامل على حساب العدد الصحيح المتبقي بعد القسمة. على سبيل المثال، 10 المعامل 3 هو 1 لأن 3 سينتقل إلى 10 ثلاث مرات (3 + 3 + 3 = 9) وسيتبقى 1.

العوامل المنطقية

تُستخدم العوامل المنطقية لاختبار شروط مختلفة. فبمجرد اختبار الشرط، سيتم إعطاؤه نتيجة صحيحة أو خاطئة. يوضح الجدول 5.13 بعض العوامل المنطقية.

الجدول 5.13 العوامل المنطقية

العامل المنطقي	ماذا يعني	مثال: نتيجة صحيحة	مثال: نتيجة خاطئة
<	أقل من	4 < 10	3 < 2
<=	أقل من أو يساوي	7 <= 8	11 <= 10
>	أكبر من	15 > 9	10 > 12
>=	أكبر من أو يساوي	3 >= 3	9 >= 12
==	يساوي	5 == 5	5 == 8
=!	لا يساوي	6 =! 7	6 =! 6
و	سيكون صحيحًا إذا كان كلا العنصرين موجودين	a = 50 b = 25 a > 10 و b > 10	a = 50 b = 25 a > 40 و b > 40
أو	سيكون صحيحًا في حالة وجود عنصر واحد على الأقل	a = 50 b = 25 a > 40 أو b > 40	a = 50 b = 25 a == 25 أو b == 25

هل تعلم؟

هل تعلم أنه يمكنك استخدام العديد من لغات البرمجة كآلة حاسبة؟ افتح لغة برمجة وأدخل بعض الحسابات مثل 4 + 7، 4 > 9، 6 >= 10 وشاهد ما سيحدث.

كلمات محجوزة

ستأتي كل لغة برمجة بمجموعة من الكلمات المحجوزة. ولا يمكن استخدام هذه الكلمات من قبل مبرمج لتسمية عناصر مختلفة في الكود مثل الثوابت أو المتغيرات. وهذا لأن هذه الكلمات لها معنى خاص في تلك اللغة المعينة.

فالكثير من الكلمات المحجوزة هي نفسها في العديد من اللغات المختلفة. على سبيل المثال، كلمة print هي كلمة محجوزة في العديد من لغات البرمجة لأنها تستخدم لعرض النص على الشاشة. يعرض الجدول 5.14 الكلمات المحجوزة الشائعة التي لا يمكن استخدامها لتسمية عناصر مختلفة في التعليمات البرمجية.

الجدول 5.14 الكلمات المحجوزة الشائعة

case	class	do	elif	else
for	function	global	if	in
input	null	private	print	procedure
public	return	switch	var	while

أوامر الإدخال

يشير مصطلح "الإدخال" إلى المستخدم الذي يدخل البيانات في البرنامج. ويمكن إجراء ذلك عن طريق:

- كتابة نص بلوحة المفاتيح
- أحداث النقر
- النقر فوق أحد الخيارات على الشاشة اللمسية.

عند استخدام **أمر الإدخال**، ستحتاج إلى تعيين القيمة التي يتم إدخالها إلى متغير. وفي بعض لغات البرمجة، ستحتاج أيضًا إلى تحديد نوع بيانات الإدخال.

حيث تختلف طريقة استخدامك لأمر الإدخال اعتمادًا على لغة البرمجة التي تستخدمها. الأمثلة الواردة في الجدول 5.15 هي من ثلاث لغات مختلفة. وهي توضح كيفية استخدام أمر الإدخال وكيفية تعيين القيمة التي تم إدخالها إلى متغير يسمى num.

المصطلحات الرئيسية

أمر الإدخال: يأخذ هذا البيانات التي أدخلها المستخدم في البرنامج.

الجدول 5.15 أمثلة على التعليمات البرمجية التي تستخدم أمر الإدخال

لغة البرمجة	سطر من التعليمات البرمجية	التوضيح
C#	int num = Console.ReadLine();	Console - يُستخدم هذا للحصول على إدخال أو عرض إخراج. ReadLine - يُستخدم هذا لقراءة سطر من النص أدخله المستخدم. int num = - سيؤدي هذا إلى حفظ المدخلات كقيمة عددية تحت اسم num.
Python	num = input ("Enter Number")	Input - يُستخدم هذا لقراءة المدخلات من المستخدم. "Enter Number" - هذه مطالبة تخبر المستخدم بما يجب إدخاله. num = - يُستخدم هذا لتخزين المدخلات تحت اسم num.
Java	int number = scanner.nextInt();	.scanner - يُستخدم هذا لقراءة المدخلات من المستخدم. NextInt - يعمل هذا مع الماسح الضوئي لقراءة العنصر التالي. int number = - سيؤدي هذا إلى حفظ الإدخال كقيمة عددية تحت اسم num.

المصطلحات الرئيسية

أمر الإخراج: يعرض هذا شيئًا من التعليمات البرمجية للمستخدم.

أوامر الإخراج

يُستخدم مصطلح الإخراج لعرض شيء ما للمستخدم. على سبيل المثال، يمكن استخدامه من أجل:

- عرض النص على الشاشة
- تغيير الخصائص مثل لون مربع النص
- تشغيل مؤثر صوتي

عند استخدام **أمر الإخراج**، ستحتاج إلى تحديد ما تريد إخراجه. فيمكن أن تكون هذه القيمة الموجودة في متغير أو يمكن أن تكون رسالة عامة للمستخدم.

حيث تختلف طريقة استخدامك لأمر الإخراج اعتمادًا على لغة البرمجة التي تستخدمها. الأمثلة الواردة في الجدول 5.16 هي من ثلاث لغات مختلفة. وهي تعرض كيفية عرض رسالة على الشاشة.

الجدول 5.16 أمثلة على التعليمات البرمجية التي تستخدم أمر الإخراج

لغة البرمجة	سطر من التعليمات البرمجية	التوضيح
C#	Console.WriteLine("Enter Width");	Console - يُستخدم هذا للحصول على إدخال أو عرض إخراج. WriteLine - يُستخدم هذا لعرض النص للمستخدم. "Enter Width" - هذا هو النص الذي سيُعرَض.
Python	print ("Enter Width")	print - يُستخدم هذا لعرض النص على الشاشة. "Enter Width" - هذا هو النص الذي سيُعرَض.
Java	System.out.println("Enter Width");	System. - يتيح لك هذا الوصول إلى موارد النظام. out. - يتيح لك هذا إخراج شيء ما. println - سيؤدي ذلك إلى طباعة النص المحدد على الشاشة. "Enter Width" - هذا هو النص الذي سيُعرَض.

دوال المستخدم

الدوال الفرعية عبارة عن مجموعة من الأكواد التي ستكمل مهمة معينة. فعادةً ما يكون جزءًا من برنامج أكبر بكثير. ويجب إعطاء اسم لكل دالة فرعية. فيمكن استخدام هذا الاسم
في نقطة أخرى في البرنامج لاستدعاء الروتين الفرعي لإكمال مهمته.

تتمتع الإجراءات الفرعية بالعديد من المزايا، بما في ذلك ما يلي.

- يمكن تقسيم التعليمات البرمجية إلى أقسام تُسهّل إدارته.
- يمكن اختبار كل دالة فرعية على حدة.
- يجب إنشاء كل دالة فرعية مرة واحدة فقط ويمكن بعد ذلك استدعاؤه عدة مرات. فهذا يجعل التعليمات العامة أكثر كفاءة.

يمكن أن تكون الدوال الفرعية إجراءً أو دالة. فكلاهما متشابهان جدًا، ومع ذلك، ستعيد الدالة قيمة إلى سطر الكود الذي يسمى الدالة، بينما لن يُرجِع الإجراء قيمة.

وعند إنشاء إجراء، ستحتاج إلى ذكر اسم الإجراء ثم **المعاملات والوسائط** التي يتطلبها. فالمعاملات والوسائط هي أجزاء من البيانات التي يحتاجها الإجراء لإكمال مهمته.

على سبيل المثال، في كود Python هذا، سيأخذ الإجراء معاملين لكلمة المرور ويتحقق مما إذا كانت متماثلة أم لا. وإذا كانت كلمتا المرور متماثلتين، فستطبع عبارة "كلمات المرور متطابقة"، وإلا فستطبع "كلمات المرور غير متطابقة".

```
def checkPasswords(password1, password2):
    if password1 == password2:
        print("Passwords match")
    else:
        print("Passwords do not match.")
```

المصطلحات الرئيسية

المعاملات والوسائط: هي أجزاء من البيانات توضع في إجراء أو دالة للسماح لها بإكمال مهمتها.

عند إنشاء دالة، ستحتاج إلى ذكر اسم الدالة ثم المعاملات والوسائط التي ستحتاجها الدالة لإكمال مهمتها. كما ستحتاج إلى تحديد قيمة الإرجاع. على سبيل المثال، إذا أكملت الدالة حساب الإضافة، فقد تُرجِع الدالة الإجمالي. فبمجرد إرجاع ذلك، سيستخدم البرنامج الرئيسي هذه القيمة لتحديد ما سيحدث بعد ذلك في البرنامج.

على سبيل المثال، في كود Python هذا، ستأخذ الدالة معاملين لكلمة المرور وتتحقق مما إذا كانت متماثلة. وإذا كانت كلمتا المرور متماثلتين، فستعيد كلمة "True"، وإلا فإنها ستعيد "False" إلى سطر الكود الذي يسمى الدالة.

```
def checkPasswords(password1, password2):
    if password1 == password2:
        return True
    else:
        return False
```

هل تعلم؟

يعتقد العديد من الباحثين أن Microsoft Windows® 10 يحتوي على نحو 50 مليون سطر من التعليمات البرمجية . كما يقترح الباحثون أن كل إصدار جديد من Microsoft Windows يزيد الكود الخاص به بنحو 5 إلى 7 ملايين سطر مقارنةً بالإصدار السابق.

المتغيرات المحلية والعالمية

يتم إنشاء متغير محلي داخل الدوال الفرعية ويكون مرئيًا في هذا الروتين الفرعي فقط.

في مثال التعليمات البرمجية Python الموضح أدناه، تم تعريف كلمة المرور المتغيرة في الدالة checkLength(). ويعني هذا أن أي دالة أخرى داخل البرنامج نفسه لا يمكنها الوصول إلى هذا المتغير. وبالتالي فإن أي تغييرات يتم إجراؤها على هذا المتغير ستؤثر على هذه الدالة فحسب.

```
def checkLength():
    password = input ("Enter New Password")
    if len(password) >= 8:
        return True

    else:
        return False

checkLength()
```

عادةً ما يتم إنشاء متغير عمومي خارج الدوال الفرعية ويكون مرئيًا في جميع أنحاء البرنامج. في مثال التعليمات البرمجية Python أدناه، تم تعريف كلمة مرور المتغير خارج الدالة checkLength(). ويعني هذا أن أي دالة فرعية أخرى داخل البرنامج نفسه يمكنه الوصول إلى المتغير. لذا، قد تؤثر أي تغييرات يتم إجراؤها على هذا المتغير على دوال أخرى في البرنامج.

```
password = input ("Enter New Password")

def checkLength():
    if len(password) >= 8:
        return True

    else:
        return False

checkLength()
```

من المهم أن نتذكر أنه ينبغي تجنب المتغيرات العالمية إن أمكن. وهذا لأنها يمكن أن تجعل من الصعب صيانة الكود. فعند استخدام المتغيرات العالمية، لا يكون كل إجراء أو دالة مستقلة تمامًا لأنها تعتمد على متغيرات من مكان آخر في الكود. وبالتالي يؤدي استخدام المتغيرات العالمية أيضًا إلى صعوبة اختبار البرنامج لأنها تجعل من الصعب اختبار مجموعة واحدة من الأكواد بمعزل عن غيرها.

الفئات

تحدد الفئة السمات في البرنامج. فالسمات هي المتغيرات المرتبطة بالفئة. وعند تحديد سماتك، تحتاج إلى تحديد نوع البيانات لكل منها. ومن ثم يتم جعل السمات خاصة بشكلٍ عام بحيث لا يمكن الوصول إليها من قِبل عناصر أخرى بمجرد إنشائها.

في هذا المثال من كود Java، يتم الإعلان عن فئة تسمى Car ستخزن تفاصيل بشأن مختلف السيارات. وهي تحتوي على خمس سمات مختلفة تم جعلها خاصة بحيث لا يمكن الوصول إليها من قِبل عنصر آخر.

```
public class Car {
    private String registrationNumber;
    private String make;
    private String model;
    private String colour;
    private int numberOfDoors;
}
```

الطرق

يتم تعريف الطرق أيضًا داخل الفئة. فهي تؤدي مهمةً أو إجراءً. وتعتمد الطرق المطلوبة على المهام التي يكملها البرنامج. ومع ذلك، فإن إحدى الطرق التي يجب أن تمتلكها جميع العناصر هي طريقة المُنشئ. فعند استدعاء هذه الطريقة، يتم تمرير البيانات إليها وستقوم بإنشاء عنصر. بعد ذلك، ستقوم بتعيين البيانات للسمات.

في هذا المثال لكود Java، تأخذ طريقة المُنشئ البيانات التي يتم تمريرها إليها كمعاملات ثم تعيّنها للسمات.

```
public Constructor(String registrationNumber, String make,
String model, String colour, int numberOfDoors) {
 this.registrationNumber = registrationNumber;
    this.make = make;
    this.model = model;
    this.colour = colour;
    this.numberOfDoors = numberOfDoors;
}
```

الطرق الشائعة الأخرى هي طرق Setter وGetter.

ستقرأ طرق Getter قيمة السمة. ففي مثال كود Java هذا، تقرأ طريقة Getter القيمة الموجودة في سمة RegistrationNumber.

```
public String getRegistrationNumber() {
    return registrationNumber;
}
```

المعلمة ثم تعيّنها في سمة Setter هذا، ستقبل طريقة Java لتعديل البيانات في إحدى السمات. ففي كود Setter تُستخدم طرق RegistrationNumber.

```
public void setRegistrationNumber(String registrationNumber) {
    this.registrationNumber = registrationNumber;
}
```

الكائنات

عندما تُنشئ مثيلاً لفئة فإنه يطلق عليه اسم كائن. حيث يأخذ كل عنصر تم إعداده من الفئة ذاتها السمات والطرق من تلك الفئة. وإليك مثال من Java. يعرض إعداد ثلاثة كائنات باستخدام طريقة Constructor في فئة السيارة. حيث يكون لكل سيارة معرف مختلف مثل car1 وcar2 وcar3.

```
Car car1 = new Constructor("7HAC710", "Honda", "Pilot", "Red", 5);
Car car2 = new Constructor("MN7719", "Nissan", "Rogue ", "Blue", 5);
Car car3 = new Constructor("718ZEW", "Fiat", "500", "Black", 3);
```

هياكل التحكم

يتحكم هيكل التحكم في البرمجة، والمعروف أيضًا باسم **البنية**، في تدفق البرنامج. ومن ثم تحدد البنية أسطر التعليمات البرمجية التي يتم تشغيلها وترتيب تشغيل كل سطر من التعليمات البرمجية. فهناك ثلاثة أنواع من البناء هي التسلسل والتحديد والتكرار.

المصطلحات الرئيسية

الإنشاء: يشير إلى تدفق البرنامج - فهو يشير إلى أسطر الكود التي يتم تشغيلها وترتيب تشغيلها.

المصطلحات الرئيسية

التسلسل: عند تشغيل كل سطر من التعليمات البرمجية واحدًا تلو الآخر وبترتيب كتابته.

التحديد: عند مقاطعة تدفق البرنامج واختبار الشرط. بعد ذلك، سيتم تمرير التحكم إلى نقطة أخرى في البرنامج بناءً على نتيجة هذا الشرط.

التسلسل

التسلسل هو عندما يتم تشغيل كل سطر من التعليمات البرمجية سطرًا تلو الآخر، بدءًا من السطر الأول من الكود وصولاً إلى السطر الأخير من الكود. على سبيل المثال، في كود C# هذا، سيتم تشغيل الكود كل سطر بالتسلسل حيث سيتم تشغيل كل سطر بالترتيب الذي تمت كتابته به.

```
int a = 5;
int b = 10;
int sum = a + b;
Console.WriteLine("The sum of {0} and {1} is {2}.", a, b, sum);
```

التحديد

التحديد، الذي يُعرف أحيانًا باسم التفريع، يحدث عند مقاطعة تدفق البرنامج واختبار أحد الشروط. ثم يتم تمرير عنصر التحكم إلى سطر آخر أو مجموعة من الأكواد في البرنامج اعتمادًا على نتيجة الشرط.

أحد أنواع جمل الاختيار هو **if... elif... else**. فأول "if" هو المكان الذي يتم فيه اختبار الشرط الأول. فإذا كان هذا الشرط صحيحًا، فسيتم تشغيل مجموعة الأكواد المرفقة به. وإذا كان الشرط خاطئًا، فسينتقل بعد ذلك إلى "elif" التالي الذي سيختبر الشرط التالي. وسيتم تكرار هذه العملية لحين اختبار جميع الشروط. فإذا كانت جميع الشروط خاطئة، فسيتم تشغيل الكود الموجود في قسم else كخيار احتياطي.

على سبيل المثال، في كود Python هذا، تم استخدام تحديد **if... elif... else**. ومن ثم يطلب البرنامج من المستخدم إدخال رقمين مختلفين ثم اختبار ما إذا كان الرقم الأول أكبر من الرقم الثاني. وإذا كان هذا صحيحًا، فسيتم تشغيل سطر الكود ذي المسافة البادئة الموجود أسفل الشرط. وإذا كان الشرط خاطئًا، فسيتم اختبار ما إذا كان الرقم الثاني أكبر من الرقم الأول أم لا. وإذا كان هذا الشرط صحيحًا، فسيتم تشغيل سطر الكود ذي المسافة البادئة الموجود أسفل هذا الشرط. وفي حالة خطأ كلا الشرطين، يجب أن يعني ذلك أن الأرقام متساوية مع بعضها بعضًا. لذا، سيتم تشغيل الكود الاحتياطي في قسم else.

```
num1 = input("Enter the first number: ")
num2 = input("Enter the second number: ")

if num1 > num2:
    print("The first number is bigger.")
elif num2 > num1:
    print("The second number is bigger.")
else:
    print("Both numbers are equal.")
```

هناك نوع آخر من التحديد هو **Switch/case**. ويُستخدم هذا لاختبار متغير مقابل قيم متعددة ممكنة.

على سبيل المثال، في كود C# هذا، تم **استخدام تحديد Switch/case**. وسيطلب من المستخدم إدخال النتيجة. بعد ذلك، تُستخدَم عبارة switch لمقارنة القيمة التي تم إدخالها للنتيجة بعدد من الحالات المختلفة. وفي الحالة الأولى، سيتم اختبار ما إذا كانت النتيجة أكبر من 30 أم لا. وإذا كانت هذه الحالة صحيحة، فسيتم تشغيل سطر الكود ذي المسافة البادئة أسفلها. وإذا كانت الحالة خاطئة، فسيتم اختبار الحالة التالية. وإذا كانت جميع الحالات خاطئة، فسيتم تشغيل الأكواد ذات المسافة البادئة في القسم الافتراضي.

```
Console.WriteLine("Enter your score:");
int score = int.Parse(Console.ReadLine());

switch (score)
{
    case var _ when score > 30:
        Console.WriteLine("Great work!");
        break;
    case var _ when score > 20:
        Console.WriteLine("Well done!");
        break;
    case var _ when score > 10:
        Console.WriteLine("Good attempt.");
        break;
    default:
        Console.WriteLine("Keep practising!");
        break;
}
```

هل تعلم؟

يمكن أن يؤدي تحديد Switch/case إلى جعل التعليمات البرمجية أكثر كفاءة.

فإذا كان لديك الكثير من الشروط، فيمكن أن يكون تحديد Switch/case أكثر كفاءة من استخدام عبارات if... elif... else المتعددة.

النشاط

اكتب التعليمات البرمجية للبرامج الثلاثة أدناه بأي لغة برمجة من اختيارك. ستحتاج إلى استخدام جمل الاختيار مثل if... elif... else أو switch/case.

البرنامج 1:

اكتب التعليمات البرمجية لتلبية القواعد التالية:

- اطلب من المستخدم إدخال ثلاثة أرقام.
- أخرج الرقم الأكبر من الأرقام الثلاثة.

البرنامج 2:

اكتب التعليمات البرمجية لتلبية القواعد التالية:

- اطلب من المستخدم تسمية إحدى القيم الأولمبية (الاحترام أو التميز أو الصداقة).
- إذا قام بتسمية أي من القيم الأولمبية بشكلٍ صحيح، فسيتم إخراج "صحيح".
- وإلا فإنه سيتم إخراج "غير صحيح".

البرنامج 3:

يقدم مركز السباحة المحلي الخصومات التالية:

1. يحصل الأعضاء الذين تتراوح أعمارهم بين 12 و15 عامًا على خصم بنسبة 30%.
2. يحصل الأعضاء الذين تتراوح أعمارهم بين 16 و18 عامًا على خصم بنسبة 20%.
3. يحصل الأعضاء الذين تتراوح أعمارهم بين 19 أو 20 عامًا على خصم بنسبة 10%.
4. يحصل الأعضاء الذين تبلغ أعمارهم 50 عامًا أو أكثر على خصم بنسبة 40%.
5. لا يحصل جميع الأعضاء الآخرين على أي خصم.

اكتب التعليمات البرمجية لتلبية هذه القواعد.

التكرار

التكرار، الذي يُعرف أحيانًا باسم التكرار، يحدث عندما يتم تشغيل سطر واحد أو أكثر من الكود أكثر من مرة. وتستخدم هذه البنية حلقة واحدة مما يلي.

- **حلقة يتم التحكم فيها بالعد** - سيؤدي ذلك إلى تكرار مقطع من الكود لعدد مرات ثابت. المثال الشائع لهذا النوع من الحلقات هو حلقة for.
- **حلقة يتم التحكم فيها بالشرط** - سيؤدي ذلك إلى تكرار مقطع من الكود لحين استيفاء الشرط. وتتضمن الأمثلة الشائعة لهذا النوع من الحلقات حلقات while وحلقات do-while.

حلقات for

تُعد حلقة for مثالاً على حلقة يتم التحكم فيها بالعد. فهي تكرر مقطعًا من الكود لعدد مرات ثابت.

على سبيل المثال، في كود C# هذا، تُخرج حلقة for كلمة "hello" 10 مرات. ومن ثم تُعيّن متغيرًا يسمى "i" إلى 0 ثم تختبر ما إذا كان هذا أقل من 10 أم لا.

وإذا كان الأمر كذلك، فسيتم إخراج كلمة "hello" ثم إضافة 1 إلى المتغير "i". وتستمر في إخراج كلمة "hello" حتى يصبح المتغير "i" يساوي 10.

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine("Hello");
}
```

حلقات while

حلقة while هي مثال على حلقة يتم التحكم فيها بالشرط. وهي تختبر الشرط في بداية الحلقة. وفي حالة استيفاء الشرط، فلن يتم تشغيل الحلقة. وفي حالة عدم استيفاء الشرط، فسيتم تشغيل الكود الموجود داخل الحلقة.

على سبيل المثال، في كود C# هذا، ستتحقق حلقة while مما إذا كان الرقم لا يساوي سبعة. فإذا كان الرقم لا يساوي سبعة، فسيطلب باستمرار من المستخدم إدخال رقم. وعندما يدخل المستخدم الرقم سبعة، ستتوقف حلقة while.

```
Console.Write("Enter a number: ");
number = int.Parse(Console.ReadLine());

while (number != 7)
{
    Console.Write("Enter a number: ");
    number = int.Parse(Console.ReadLine());
}
```

حلقات Do-while

حلقة do-while هي مثال آخر للحلقة التي يتم التحكم فيها بالشرط. فهي تختبر شرطًا في نهاية الحلقة. ويعني هذا أنه سيتم تشغيل الكود داخل الحلقة مرة واحدة على الأقل.

على سبيل المثال، في كود C# هذا، ستطلب حلقة do-while من المستخدم إدخال رقم ثم التحقق مما إذا كان هذا لا يساوي سبعة. وستطلب باستمرار من المستخدم إدخال رقم حتى يدخل المستخدم الرقم 7.

المصطلحات الرئيسية

التكرار: عند تكرار سطر واحد أو أكثر من الكود أكثر من مرة.

حلقة for: حلقة يتم التحكم فيها بالعد والتي ستكرر مقطعًا من الكود لعدد مرات ثابت.

حلقة while: حلقة يتم التحكم فيها بالشرط والتي ستكرر مقطعًا من الكود لحين استيفاء الشرط. وستختبر الشرط في بداية الحلقة.

حلقة Do-while: حلقة يتم التحكم فيها بالشرط والتي ستكرر مقطعًا من الكود لحين استيفاء الشرط. وستختبر الشرط في نهاية الحلقة.

أفضل ممارسة

من الممارسات الجيدة الحفاظ على الكود بأكبر قدر ممكن من الكفاءة. وسيُسهّل ذلك صيانة الكود في المستقبل. ويُعد استخدام الحلقة إحدى الطرق الجيدة لجعل الأكواد فعالة.

```
do
{
    Console.Write("Enter a number: ");
    number = int.Parse(Console.ReadLine());

} while (number != 7);
```

النشاط

اكتب التعليمات البرمجية للبرامج الثلاثة أدناه بأي لغة برمجة من اختيارك. تأكد من استخدام النوع الأنسب من حلقة البرمجة.

البرنامج 1:

اكتب التعليمات البرمجية لإخراج الكلمات "Hello Learner" عشر مرات.

البرنامج 2:

اكتب التعليمات البرمجية لإخراج جميع الأرقام الفردية بين 1 و20.

البرنامج 3:

اكتب التعليمات البرمجية الذي سيتيح للمستخدم إدخال عدد الأرقام التي يريد إضافتها معًا. ينبغي أن يتيح البرنامج بعد ذلك للمستخدم إدخال العديد من الأرقام ثم إخراج الإجمالي.

مواضيع ذات صلة

لمعرفة المزيد عن هياكل التحكم في البرمجة، راجع **الوحدة 8: مقدمة لتطوير التطبيق**.

بُنى البيانات

ستعمل المتغيرات والثوابت على تخزين جزء واحد من البيانات في برنامج حاسوبي. ومع ذلك، إذا كنتَ ترغب في تخزين الكثير من البيانات، فستحتاج إلى الكثير من المتغيرات والثوابت. تتمثل الطريقة البديلة في استخدام **بنية بيانات** مثل **سجل** أو **مصفوفة**.

بُنى البيانات هي طريقة لتنظيم وتخزين البيانات في برنامج حاسوبي. فهي تتيح للمبرمجين تشغيل عمليات مختلفة على البيانات مثل فرزها بالترتيب أو البحث عن قيمة أو إدراج قيم جديدة.

المصطلحات الرئيسية

بنية البيانات: تستخدم لتخزين البيانات وتنظيمها في برنامج حاسوبي.

السجل: صف في جدول - يحتوي كل صف على عدة حقول بشأن شخص أو مكان أو شيء أو مفهوم.

المصفوفة: تُستخدم لتخزين قوائم البيانات التي تحتوي جميعها على نوع البيانات نفسه - يتم إعطاء كل عنصر في المصفوفة رقم فهرس مختلف.

بنية السجل: تُستخدم لتخزين البيانات بشأن شخص أو مكان أو شيء أو مفهوم - يمكن إعطاء كل حقل في البنية نوع بيانات مختلف.

الحقل: عمود في جدول - يحتوي كل عمود على معلومات عن جانب واحد من الشخص أو المكان أو الشيء أو المفهوم.

أنواع البيانات المحددة وهياكل السجلات من قِبل المستخدم

تتيح لك **بنية السجل** تخزين عناصر بيانات متعددة ذات صلة بأنواع بيانات مختلفة معًا. فقد يكون من الأسهل التفكير في بنية السجل كجدول، مثل المثال الوارد في الجدول 5.17.

الجدول 17-5 مثال لهيكل السجل: الشخص

معرّف المستخدم	الاسم الأول	اللقب	تاريخ الميلاد
1	لورنا	زونيغا	18/06/2008
2	أبي	بيترسون	14/04/2007
3	آرون	روبرتس	29/07/2009
4	سينياد	بول	05/01/2006

يُطلق على بنية السجل الموضحة في الجدول 5.17 اسم الشخص. ويحتوي كل صف، المعروف أيضًا باسم السجل، على تفاصيل شخص واحد. ويتم إعطاء كل عمود، المعروف أيضًا باسم **الحقل**، نوع بيانات محدد من قبل المستخدم (مثل عدد صحيح) يتعلق بالبيانات المخزنة في هذا العمود. كما تتمثل إحدى طرق إعداد بنية بيانات السجل في استخدام فئة.

يُعِد كود Python هذا بنية سجل لتخزين البيانات في الجدول، على النحو الموضح في الجدول 5.17 أعلاه.

```
from dataclasses import dataclass

class Person:
    userID: int
    firstName: str
    surname: str
    dateOfBirth: str
```

عند إعداد بنية السجل، يمكنك بعد ذلك إنشاء سجل لكل شخص.

فيما يلي مثال من Python يوضح كيف يمكن إضافة السجلات إلى بنية السجل.

```
person1 = Person(1, "Lorna", "Zuniga", "18/06/2008")
person2 = Person(2, "Abi", "Peterson", "14/04/2007")
person3 = Person(3, "Aron", "Roberts", "29/07/2009")
person4 = Person(4, "Sinead", "Paul", "05/01/2006")
```

المصفوفات

تعمل المتغيرات والثوابت على تخزين جزء واحد من البيانات في برنامج حاسوبي. على سبيل المثال، في التعليمات البرمجية C# هذا، هناك خمسة متغيرات مختلفة يخزن كل منها قيمة عددية.

```
int num1 = 2;
int num2 = 5;
int num3 = 8;
int num4 = 4;
int num5 = 5;
```

عندما تخزّن المتغيرات بهذه الطريقة، يجب عليك الإعلان عن الكثير من المتغيرات المختلفة، مما يعني أنه يجب عليك كتابة المزيد من أسطر التعليمات البرمجية. وهناك أسلوب بديل وهو استخدام مصفوفة.

فالمصفوفة عبارة عن قائمة بالمتغيرات التي تنتمي جميعها إلى نوع البيانات نفسه. ومن ثم يتم إعطاؤهم جميعًا الاسم نفسه. ويعني هذا أنه يمكنك تخزين الأرقام الخمسة جميعها في المثال أعلاه في مصفوفة واحدة، بدلاً من استخدام خمسة متغيرات مختلفة. فعندما تعرّف مصفوفة جديدة، تحتاج إلى ذكر اسم المصفوفة وعدد العناصر التي سيتم تخزينها ونوع البيانات.

على سبيل المثال، في التعليمات البرمجية C# هذه، تم الإعلان عن **مصفوفة** جديدة **أحادية البعد** تسمى الأرقام التي ستخزن خمس قيم صحيحة.

وتحتوي المصفوفة أحادية البعد على صف بيانات واحد.

```
int[] numbers = new int[5];
```

بدلاً من ذلك، يمكنك تعريف المصفوفة وإدخال البيانات في المصفوفة في الوقت نفسه، على النحو الموضح في التعليمات البرمجية C# هذه.

```
int[] numbers = new int[] { 2, 5, 8, 4, 5 };
```

المصطلحات الرئيسية

المصفوفة أحادية البعد: تُخزّن البيانات بطريقة خطية في صف واحد، حيث يتم تخزين البيانات في مواقع متتالية.

كل عنصر في المصفوفة له اسم المعرف نفسه (مثل الأرقام)، ومع ذلك، سيكون لكل عنصر **رقم فهرس** مختلف. وتعتمد أرقام الفهرس على الصفر مما يعني أن العنصر الأول في المصفوفة سيحتوي على فهرس 0.

على سبيل المثال، في التعليمات البرمجية C# هذه، ستطبع **الأرقام [0]** الرقم 2.

```
Console.WriteLine(numbers[0]);
```

في التعليمات البرمجية C# هذه، ستطبع **الأرقام [2]** الرقم 8.

```
Console.WriteLine(numbers[2]);
```

يمكن أيضًا أن تكون المصفوفة **ثنائية الأبعاد**. سيخزّن هذا النوع من المصفوفات البيانات في صفوف وأعمدة متعددة.

على سبيل المثال، في التعليمات البرمجية C# هذه، تم الإعلان عن مصفوفة ثنائية الأبعاد لتخزين تسعة أرقام منظمة كشبكة 3 × 3.

```
int[,] array = new int[3,3]
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};
```

النشاط

اكتب التعليمات البرمجية للبرنامجين أدناه بأي لغة برمجة من اختيارك. تتطلب البرامج تخزين البيانات باستخدام بنية بيانات المصفوفة.

البرنامج 1:

اكتب التعليمات البرمجية لإنشاء مصفوفة أحادية البُعد تسمى الأرقام لتخزين القيم الصحيحة 1، 2، 3، 4، 5.

البرنامج 2:

اكتب التعليمات البرمجية لإنشاء مصفوفة ثنائية الأبعاد تسمى NumberCard لتخزين القيم الصحيحة:

1، 6، 9

3، 5، 7

2، 4، 8

في حين أن المصفوفات يمكنها المساعدة في جعل التعليمات البرمجية أكثر كفاءةً، إلا أنها تتضمن بعض العيوب. وتشمل هذه العيوب:

- **المصفوفات ثابتة:** عند الإعلان عن المصفوفات، يجب تعيين حجم المصفوفة. وهذا يحفظ مساحة في الذاكرة للعديد من عناصر البيانات. ومع ذلك، لا يمكن زيادة هذا أو تقليله أثناء تشغيل البرنامج.
- **يجب أن تكون جميع القيم من نوع البيانات نفسه:** عندما يتم الإعلان عن المصفوفات، يجب الإعلان عن نوع البيانات. فبمجرد تعيين ذلك، يجب أن تكون جميع البيانات التي المضافة إلى المصفوفة ضمن نوع البيانات هذا. على سبيل المثال، لا يمكن تخزين قيم الأعداد الصحيحة وقيم السلسلة على حدٍ سواء.

المصطلحات الرئيسية

أرقام الفهرس: تُستخدم لتحديد موضع كل عنصر في المصفوفة.

المصفوفة ثنائية الأبعاد: تُخزن البيانات في بنية تشبه الشبكة تتكون من صفوف وأعمدة.

المصطلحات الرئيسية

النموذج: يتيح للمستخدم إدخال البيانات والأوامر إلى الحاسوب باستخدام العناصر الرسومية - غالبًا ما تحتوي النماذج على أزرار ومربعات نصية لمساعدة المستخدم.

عناصر تحكم النموذج: هذه أجزاء مفردة من نموذج إدخال المستخدم تتيح للمستخدمين التفاعل مع البرنامج - وهي تتضمن عناصر مثل مربعات النص والأزرار.

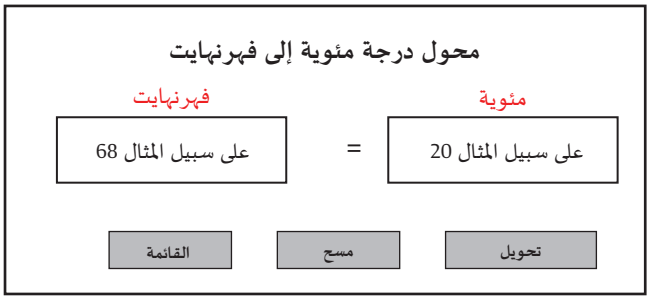
معالجة الأحداث

النماذج

النموذج، المعروف أيضًا باسم نموذج إدخال المستخدم، يتيح للمستخدم التفاعل مع البرنامج، غالبًا باستخدام الرسومات. ويحتوي على عناصر مختلفة مثل مربعات النص والقوائم المنسدلة والأزرار للسماح للمستخدم بإدخال البيانات والأوامر.

كما يتم إرسال البيانات والأوامر من النموذج إلى التعليمات البرمجية الذي يعالجها بعد ذلك. فبمجرد أن تجري التعليمات البرمجية ذلك، ستُعرض النتيجة مرة أخرى في واجهة المستخدم.

في المثال الموضح في الشكل 5.21، يُطلب من المستخدم إدخال درجة الحرارة بالدرجة المئوية. ومن ثم يتم أخذ هذه القيمة من النموذج ونقلها إلى التعليمات البرمجية. بعد ذلك، يحوّل هذا الكود درجة الحرارة إلى فهرنهايت ثم يرسل القيمة مرة أخرى إلى واجهة المستخدم.



الشكل 5.21 مثال لنموذج إدخال المستخدم.

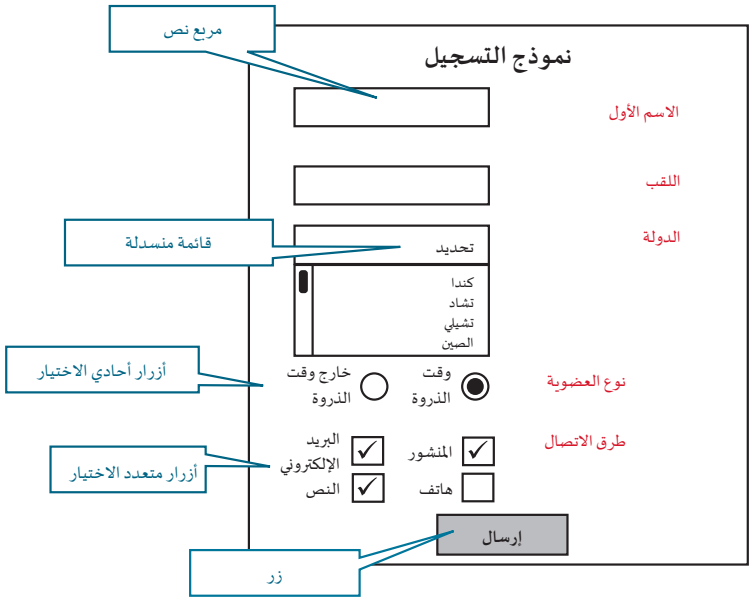
هل يمكنك تحديد جميع الأحداث المحتملة التي يمكن أن تقع على هذه الشاشة؟

خصائص الشاشة

يحتوي النموذج على العديد من المكونات المختلفة، المعروفة باسم **عناصر تحكم النموذج**، والتي تتيح للمستخدم التفاعل مع البرنامج.

وتتضمن عناصر تحكم النموذج ما يلي:

- **الأزرار** - هذه هي العناصر التي يمكن للمستخدمين النقر فوقها أو الضغط عليها. حيث يُشغِّل الزر معالج الأحداث، والذي يُشغِّل بعد ذلك الدوال الفرعية ينفّذ الإجراء.
- **مربعات النص** - هذه مناطق صغيرة من الشاشة يُدخِل المستخدمون النص فيها.
- **القوائم المنسدلة** - تعرض هذه العناصر قائمة بالخيارات المحددة مسبقًا ليختارها المستخدم. وتُعرف أحيانًا باسم مربعات التحرير والسرد.
- **أزرار أحادي الاختيار** - تتيح هذه العناصر للمستخدم قائمة بالخيارات المحددة مسبقًا لتحديدها. ومع ذلك، بإمكان المستخدم اختيار خيار واحد فقط في كل مرة.
- **أزرار متعدد الاختيار** - توفر هذه العناصر للمستخدم قائمة بالخيارات المحددة مسبقًا لتحديدها. بإمكان المستخدم اختيار العديد من الخيارات كما يريد.



الشكل 5.22 مثال لنموذج إدخال البيانات.
هل يمكنك التفكير في أي معلومات أخرى بإمكان المستخدم إدخالها؟ ما عناصر تحكم النموذج التي سيستخدمها؟

يوضح الشكل 5.22 نموذجًا لإدخال البيانات تم إنشاؤه لجمع تفاصيل المستخدم لعضوية صالة الألعاب الرياضية.

التحقق من صحة البيانات

التحقق من صحة البيانات يأخذ البيانات التي أدخلها المستخدم ويتحقق من أنها صحيحة ومعقولة. وتتضمن الأنواع المختلفة للتحقق من صحة البيانات ما يلي:

- **عمليات التحقق من النطاق** - تتحقق من وقوع البيانات ضمن حدين مختلفين. على سبيل المثال، التحقق من استخدام رقم بين 1 و31 لتمثيل اليوم في التاريخ.
- **عمليات التحقق من النوع** - تتحقق هذه الاختبارات من استخدام البيانات لنوع البيانات الصحيح. على سبيل المثال، التحقق من احتواء الاسم فقط على أحرف ولا يتضمن أرقامًا.
- **عمليات التحقق من الطول** - تتحقق من أن البيانات التي تم إدخالها تحتوي على الحد الأدنى لعدد الأحرف. على سبيل المثال، التحقق من أن كلمة المرور الجديدة تحتوي على ثمانية أرقام على الأقل.
- **عمليات التحقق من التوافر** - تتحقق هذه الاختبارات من إدخال البيانات. على سبيل المثال، التحقق من أن المستخدم قد أدخل عنوانه.
- **عمليات التحقق من التنسيق** - تتحقق من أن البيانات منظمة بشكلٍ صحيح. على سبيل المثال، سيتحقق من أن التاريخ بالتنسيق DD/MM/YYYY أو MM/DD/YYYY.

ويمكن أن يحتوي جزء واحد من البيانات على العديد من عمليات التحقق من الصحة التي يتم إجراؤها عليها. على سبيل المثال، في التعليمات البرمجية Java هذا، تم إجراء التحقق من التوافر والتحقق من النوع على اسم المستخدم الأول.

```
public static boolean isValidFirstName(String firstName) {
    if (firstName.isEmpty()) {
        return false;
    }
    if (!firstName.matches("[a-zA-Z]+")) {
        return false;
    }
        return true;
    }
```

المصطلحات الرئيسية

توضيح التحقق : يتم بواجهة المستخدم وبنفس الصفحة الحالية.

لا تضمن عملية التحقق من صحة البيانات دقة البيانات. ففي المثال أعلاه، ستضمن عمليات التحقق إدخال الاسم الأول واحتوائه على أحرف فقط. ومع ذلك، لن تتحقق من أن الاسم الأول الذي تم إدخاله هو الاسم الصحيح.

الإجراءات

الإجراءات هي معالجات الأحداث التي يتم تشغيلها عند وقوع الأحداث. يقدم الجدول 5.18 أمثلة مختلفة لبعض الإجراءات المحتملة التي يمكن أن تحدث.

الجدول 18-5 أمثلة على الإجراءات الممكنة

نوع الحدث	مثال للحدث	الإجراء
حدث نقر	ينقر المستخدم فوق قائمة منسدلة.	تُعرض قائمة بالخيارات.
حدث تمرير الماوس	يمرر المستخدم مؤشر الماوس فوق الزر.	تظهر رسالة مختصرة تشرح ما يفعله الزر.
حدث ضغط على مفتاح	يدخل المستخدم اسمه.	يتم إجراء التحقق للتأكد من أن ما تمت كتابته عبارة عن حرف.

ممارسة كتابة التعليمات البرمجية الجيدة

هناك العديد من الأسباب التي قد تؤدي إلى تغيير التعليمات البرمجية، بما في ذلك:

- قد يكون لدى مستخدم البرنامج متطلبات جديدة يحتاج البرنامج إلى تلبيتها.
- قد يكون هناك تهديد أمني جديد يحتاج البرنامج للدفاع ضده.
- قد لا يتوافق البرنامج مع نظام تشغيل جديد.

عند كتابة تعليمات برمجية، من المهم دائمًا التأكد من إمكانية تغيير الكود بسهولة. فاتباع الممارسات الجيدة يُسهّل على المبرمِجين الآخرين النظر إلى الكود في المستقبل وفهم ما يفعله الكود وإجراء أي تغييرات ضرورية عليه.

التعليقات

تشرح **تعليقات الأكواد** ما يفعله سطر أو مجموعة من الأكواد بالفعل. وعند تشغيل تعليمات برمجية، يتم تجاهل جميع تعليقات الأكواد. ولا يكون وضع التعليقات على كل سطر مفيدًا في بعض الأحيان لأنه قد يجعل قراءة الكود أكثر صعوبة. لذا، من الممارسات الجيدة للمبرمِجين تقديم تعليقات لكل مجموعة من الأكواد. ومع ذلك، إذا كان من الصعب فهم سطر معين من الكود، فينبغي أن يكون له تعليق خاص به.

لذا تختلف كيفية إضافة تعليقات الأكواد اعتمادًا على لغة البرمجة التي تستخدمها (راجع الجدول 5.19).

الجدول 5.19 تعليقات الكود

اللغة	التعليق
C#	// تستخدم خطوط مائلة مزدوجة للأمام
Python	# تستخدم رمز التجزئة
Java	// تستخدم خطوط مائلة مزدوجة للأمام

المهارات

المهارات المعرفية: الانفتاح الفكري:

- المسؤولية الشخصية والاجتماعية

المصطلحات الرئيسية

تعليقات الكود: التعليقات الإضافية التي تتم إضافتها إلى تعليمات برمجية لشرح معنى سطر أو مجموعة أسطر بالكود.

على سبيل المثال، في كود Python هذا، تمت إضافة تعليقات الأكواد لشرح ما تفعله كل مجموعة أكواد.

```
def checkTelephoneNumber(telephone):
    # Check the length of the telephone number to ensure it has
    11 digits
    if len(telephone) != 11:
        return False

    # Check if the telephone number starts with a '+' symbol
    if not telephone.startswith('+'):
        return False

    # All criteria are met
    return True
```

أفضل ممارسة

من الممارسات الجيدة استخدام تعليقات الكود لتذكيرك بما يفعله الكود في وقتٍ لاحق. سيساعدك هذا إذا كنت بحاجة إلى صيانة الكود في المستقبل. كما سيساعد المبرمِجين الآخرين على فهم ما يفعله الكود.

ترتيب التعليمات البرمجية لسهولة القراءة للبداية والنهاية

المصطلحات الرئيسية

المسافة البادئة للكود: عند وضع علامة تبويب في بداية سطر الكود للإشارة إلى متى يبدأ شيء ما وينتهي.

ترتيب التعليمات البرمجية لسهولة القراءة للبداية والنهاية يُقصد بها عند وضع علامة تبويب في بداية سطر من التعليمات البرمجية تساعد التعليمات البرمجية لسهولة القراءة للبداية والنهاية على وضع الأكواد في مجموعات وتشير إلى متى يبدأ جزء من الكود وينتهي. حيث يتمثل الاستخدام الشائع للمسافة البادئة في تسليط الضوء على جمل الاختيار مثل if...elif...else أو عناصر التكرار مثل حلقات for وwhile وdo-while.

فعادةً ما يتم وضع التعليمات البرمجية لسهولة القراءة للبداية والنهاية تلقائيًا من قِبل IDE. كما تصر بعض لغات البرمجة مثل Python على أن الكود مزود بمسافة بادئة ولن يعمل إذا كان ترتيب التعليمات البرمجية لسهولة القراءة للبداية والنهاية غير صحيح. وفي لغات البرمجة الأخرى مثل C أو JavaScript، لا يلزم وجود ترتيب تعليمات برمجية لسهولة القراءة للبداية والنهاية للكود لتشغيل التعليمات البرمجية.

فبغض النظر عما إذا كانت لغة البرمجة تصر على المسافة البادئة للكود أم لا، يوصى بشدة بأن يستخدمها المبرمِجون.

وهذا لأن ترتيب التعليمات البرمجية لسهولة القراءة للبداية والنهاية الفعال سيجعل الفعالة للكود ستجعل من السهل رؤية:

- كيف يتم تجميع كود البرنامج في مجموعات
- عندما تبدأ مجموعة من الأكواد وتنتهي
- ما هي الشروط التي يجب الوفاء بها لتشغيل الكود

أسماء المتغيرات المناسبة

إن استخدام الأسماء التي تصف محتويات المتغيرات بدقة يساعد المبرمِجين على فهم البيانات التي تمثلها المتغيرات. ويساعد هذا المبرمِجين على قراءة الكود وفهم ما يفعله. كما أنه يُسهّل صيانة الكود. يقدم الجدول 5.20 أمثلة لأسماء المتغيرات السيئة والجيدة.

الجدول 5.20 أمثلة لأسماء المتغيرات السيئة والجيدة

أسماء المتغيرات السيئة	أسماء المتغيرات الجيدة
n	number
pn	playerName
hs	highScore

الشكل 5.23
متغيرات التسمية.
هل يمكنك التفكير في أي قواعد أخرى ينبغي اتباعها عند تسمية المتغيرات؟

يوضح الشكل 5.23 الممارسات الجيدة التي ينبغي اتباعها عند تسمية المتغيرات.

(ب2) اختبار حل برمجي وتحسينه

يضمن الاختبار عمل جميع أجزاء التعليمات البرمجية على النحو المنشود.

ومن المهم اختبار خلو التعليمات البرمجية من الأخطاء وأدائه لما يريده المستخدم النهائي وموثوقيته وتقديمه نتائج دقيقة.

ما هو الاختبار؟

الاختبار يمكن تقسيمه إلى فئتين رئيسيتين.

الاختبار التكراري هو عندما يتم اختبار التعليمات البرمجية أثناء تطويرها. سيكتب المبرمج تعليمات برمجية الدوال الفرعية ويختبره ثم ينتقل إلى المرحلة التالية.

يتم إجراء الاختبار النهائي عند انتهاء البرنامج تمامًا واختبار جميع أجزاء البرنامج بشكلٍ منفصل. فغالبًا ما يختبر المستخدم النهائي أن جميع الدوال المطلوبة موجودة في هذه المرحلة.

كما أن هناك أنواع مختلفة من الأخطاء التي يجب أن يكون المبرمِجون على دراية بها.

يحدث خطأ في بناء الجملة **عندما لا يتبع المبرمج قواعد لغة البرمجة وسيؤدي ذلك إلى تعطل البرنامج.**

على سبيل المثال، في كود Python هذا، هناك مجموعة من علامات الكلام مفقودة.

```
print("Hello Learner!)
```

ينبغي قراءة الكود على النحو التالي:

```
print("Hello Learner!")
```

هناك نوع آخر من الأخطاء وهو **الخطأ المنطقي**. ويحدث هذا عندما لا يقوم المبرمج ببرمجة القواعد الصحيحة. فلن يتسبب ذلك في تعطل البرنامج، ولكنه سيؤدي إلى إجراء شيء غير متوقع مثل إخراج قيمة غير صحيحة.

بدء النشاط

ماذا تعتقد أنه سيحدث إذا أصدرت شركة برنامجًا لم يتم اختباره بالكامل؟ ما التأثيرات المحتملة على الشركة ومستخدمي البرنامج؟

المصطلحات الرئيسية

الاختبار: التحقق من خلو التعليمات البرمجية من الأخطاء وعمله على النحو المنشود وتلبيته لمتطلبات المستخدم.

الاختبار التكراري: اختبار التعليمات البرمجية أثناء كتابة الكود.

الاختبار النهائي: اختبار التعليمات البرمجية عند الانتهاء من الكود.

خطأ في بناء الجملة: عندما لا يتم اتباع القواعد للغة البرمجة، مما يتسبب في تعطل البرنامج.

خطأ منطقي: عندما تكون قواعد البرنامج أو المنطق غير صحيحة - سيؤدي ذلك إلى قيام البرنامج بشيء غير متوقع.

على سبيل المثال، في كود Python هذا، إذا كانت القيمة الموجودة في المتغير num1 أكبر من القيمة الموجودة في المتغير num2، فسيخرج البرنامج "الرقم 2 أكبر".

```
num1 = input ("Enter Number 1")
num2 = input ("Enter Number 2")
if num1 > num2:
    print("Number 2 is bigger")
```

بيانات الاختبار

بيانات الاختبار هي البيانات التي تُستخدم للتحقق من عمل البرنامج بشكلٍ صحيح. فعند اختبار التعليمات البرمجية، من المهم اختباره بأنواع مختلفة من بيانات الاختبار. وسيضمن ذلك أن الاختبارات تغطي أكبر عدد ممكن من الاحتمالات المختلفة. حيث تُعرض الأنواع الرئيسية لبيانات الاختبار في الجدول 5.21.

الجدول 5.21 الأنواع الرئيسية لبيانات الاختبار

نوع البيانات	الوصف
البيانات العادية	البيانات العادية هي البيانات الصحيحة والتي ينبغي أن يقبلها البرنامج دون إنتاج أي أخطاء.
البيانات غير العادية	البيانات غير العادية هي البيانات غير الصحيحة والتي ينبغي ألا يقبلها البرنامج. ويتضمن ذلك البيانات من نوع البيانات غير الصحيحة أو البيانات التي تقع خارج نطاق التحقق المقبول.
البيانات المتطرفة	البيانات المتطرفة هي بيانات من النوع الصحيح، ومع ذلك، فهي على وشك أن تكون صالحة. وينبغي قبولها دون إنتاج أي أخطاء.

المصطلحات الرئيسية

بيانات الاختبار: البيانات المحددة للتحقق من عمل التعليمات البرمجية بشكلٍ صحيح.

على سبيل المثال، خذ برنامجًا بسيطًا يتيح للمعلم إدخال علامات الاختبار في الحاسوب. يجب أن تكون العلامة التي تم إدخالها بين 0 و70 ويجب أن تكون في شكل عدد صحيح. يظهر نموذج إدخال المستخدم لهذا البرنامج في الشكل 5.24.

يوضح الجدول 5.22 أنواعًا مختلفة من البيانات العادية وغير العادية والمتطرفة التي يمكن استخدامها لاختبار البرنامج.

الشكل 5.24 نموذج إدخال مستخدم بسيط.

الجدول 5.22 أمثلة على البيانات العادية وغير العادية والمتطرفة

نوع البيانات	بيانات المثال 1	بيانات المثال 2	بيانات المثال 3
البيانات العادية	9	36	55
البيانات غير العادية	5–	78	"خمسة وخمسون"
البيانات المتطرفة	0	70	

المصطلحات الرئيسية

خطة الاختبار: مستند يسرد جميع الاختبارات المختلفة التي ينبغي إجراؤها على البرنامج لاختبار أن البرنامج يعمل بشكلٍ صحيح.

خطط الاختبار

خطة الاختبار هي وثيقة تسرد كيفية اختبار التعليمات البرمجية. وينبغي أن يكملها المبرمج أو المختبِر. حيث يكملان الاختبارات على النحو الموضح في الخطة ثم يسجلان نتيجة كل اختبار عليها.

عادةً ما تتضمن خطة الاختبار ما يلي:

- رقم الاختبار لكل اختبار
- وصف لكل اختبار
- نوع كل اختبار
- البيانات التي سيتم استخدامها لكل اختبار
- النتيجة المتوقعة لكل اختبار

على سبيل المثال، في التعليمات البرمجية Python هذه، يُطلب من المستخدم إدخال رقم بين 1 و3.

```
print("Welcome to Rock, Paper, Scissors!")
print("Enter your choice: ")
print("1. Rock")
print("2. Paper")
print("3. Scissors")
playerChoice = int(input("Enter a number between 1 and 3"))

if playerChoice < 0 or playerChoice >= 3:
    print("Invalid choice")
```

يظهر جزء من خطة الاختبار التي يمكن استخدامها لاختبار التعليمات البرمجية هذا في الجدول 5.23.

الجدول 5.23 جزء من خطة الاختبار

رقم الاختبار	وصف الاختبار	نوع الاختبار	البيانات المستخدمة	النتيجة المتوقعة
1	يقبل البرنامج البيانات ضمن النطاق المقبول	عادي	2	يقبل البرنامج البيانات
2	يرفض البرنامج البيانات غير الصحيحة	غير عادي	"اثنان"	غير مقبول - يُعرض "اختيار غير صالح"
3	يقبل البرنامج البيانات الموجودة على الحدود	متطرف	3	يقبل البرنامج البيانات

أفضل ممارسة

من الممارسات الجيدة اختبار التعليمات البرمجية بأكبر قدر ممكن من الدقة. وينبغي لك اختبار كل إدخال ممكن للمستخدم (أو مجموعة من المدخلات) للتأكد من أن التعليمات البرمجية آمن وفعال وقوي.

النشاط

1 اذكر جزءًا **واحدًا** من البيانات العادية وغير العادية والمتطرفة التي يمكن استخدامها لاختبار هذا البرنامج. سيقوم البرنامج بما يلي:

- مطالبة المستخدم بإدخال رقمين **موجبين**.
- قسمة الرقم الأول على الرقم الثاني.
- إخراج النتيجة.

2 يطلب نموذج إدخال المستخدم من المستخدم إدخال:

- الاسم الأول
- اللقب
- تاريخ الميلاد
- رقم الهاتف

حدد العديد من قواعد التحقق من صحة البيانات التي تعتقد أنه يمكن استخدامها على هذه الأجزاء من البيانات.

3 ترغب صالة الألعاب الرياضية في الحصول على برنامج يمكن للأعضاء الجدد استخدامه لتسجيل تفاصيلهم. سيحتاج البرنامج إلى نموذج إدخال مستخدم يمكن للمستخدمين استخدامه لإدخال بياناتهم مثل:

- الاسم
- تاريخ الميلاد
- الارتفاع بالأقدام والبوصة
- الوزن بالحجارة والرطل
- تفضيل الدفع - بإمكان العضو الاختيار شهريًا أو سنويًا فقط
- أهداف اللياقة البدنية - بإمكان العضو اختيار العديد من الأهداف كما يحلو له: المزيد من الطاقة أو النوم بشكلٍ أفضل أو تحسين اللياقة العامة أو تحسين الصحة العقلية.

ارسم نموذج إدخال المستخدم المناسب الذي يمكن لصالة الألعاب الرياضية استخدامه. تأكد من استخدام عناصر تحكم النموذج المناسبة.

المهارات

المهارات المعرفية: الإبداع:

- الإبداع
- الابتكار

توثيق التحسينات

بمجرد اختبار التعليمات البرمجية وإكمال خطة الاختبار، عليك التفكير في أي من الاختبارات التي لم تسفر عن النتائج التي توقعتها. تُعرف هذه الاختبارات باسم الاختبارات الفاشلة وينبغي أن تتيح لك معرفة أين ينبغي تحسين التعليمات البرمجية.

ينبغي توثيق أي تحسينات تُجريها. وهذا بسبب:

- سيتيح لك التعلم من التجارب السابقة
- يمكنك استخدام عملية التوثيق لحل المشكلات المماثلة التي تحدث في المستقبل
- سيتمكن المبرمجون الآخرون من رؤية كيف تغير الكود بمرور الوقت.

عند توثيق التحسينات التي تم إجراؤها على التعليمات البرمجية، من المهم تضمين:

- رقم الاختبار فيما يخص الاختبار الفاشل
- وصف واضح لماهية الخطأ
- كيف سيعالج التحسين الخطأ
- التغييرات المحددة التي تم إجراؤها بالفعل على التعليمات البرمجية
- تاريخ إجراء أي تغيير
- اسم المبرمج الذي أجرى التغيير

من المهم ملاحظة أنه بمجرد تغيير التعليمات البرمجية، يجب أيضًا اختبار أي تحسينات للتأكد من أنها تعمل بشكلٍ صحيح. كما أنه من المهم التأكد من أن الأجزاء الأخرى من التعليمات البرمجية لا تزال تعمل على النحو المنشود. وهذا لأنه عند تحسين الكود، يمكن أن يكون له تأثير على أجزاء أخرى من التعليمات البرمجية.

المهارات

المهارات الشخصية: أخلاقيات العمل/الضمير:

- التوجيه الذاتي
- المسؤولية

التقييم الذاتي الأساسي الإيجابي:

- المراقبة الذاتية
- التقييم الذاتي
- التعزيز الذاتي

مواضيع ذات صلة

لمعرفة المزيد عن اختبار البرامج وتحسينها، راجع **الوحدة 8: مقدمة لتطوير التطبيق**.

دراسة الحالة

اقرأ المعلومات بشأن شركة خيالية تقع في الأردن تسمى CodingSolutions123 أدناه ثم ناقش الأسئلة في مجموعات أو مع أحد الزملاء.

CodingSolutions123

CodingSolutions123 هي شركة تطوير برمجيات. فقد عملت على التعليمات البرمجية لبرنامج جديد يسمى SafeMate من شأنه تعزيز السلامة في المنزل لكبار السن. ويتولى البرنامج الآتي:

- تقديم نصائح السلامة المنزلية لمساعدة الناس على منع الحوادث في المنزل
- تخزين أرقام الاتصال في حالات الطوارئ للأشخاص الذين يمكن الاتصال بهم بسرعة في حالات الطوارئ
- تقديم تذكيرات السلامة مثل رسائل التحقق من عمل أجهزة كشف الدخان
- تقديم تعليمات الإسعافات الأولية والاتصال بخدمات الإسعاف في حالات الطوارئ.

تُصدِر شركة منافسة أيضًا برنامجًا مماثل.

لم تتمكن شركة CodingSolutions123 من إكمال برنامجها بحلول الموعد النهائي المحدد. فقد قررت أنها ستخفض كمية الاختبارات التي يتم إجراؤها من أجل إطلاق البرنامج بسرعة أكبر. فبمجرد إصدار البرنامج، ستقوم بعد ذلك باختبار البرنامج بشكلٍ صحيح. بعد ذلك، ستُصلِح أي أخطاء تجدها في التعليمات البرمجية باستخدام تحديثات البرامج التي سيحتاج المستخدم إلى تنزيلها على أجهزته.

الأسئلة

1. لماذا تعتقد أن شركة CodingSolutions123 أرادت إصدار برنامجها في وقتٍ سابق؟
2. هل تعتقد أن الشركة قد اتخذت القرار الصحيح لتقليل كمية الاختبارات التي يتم إجراؤها قبل إصدار برنامجها؟
3. ما هي بعض الآثار المترتبة على هذا القرار؟

جودة الكود

الكود عالي الجودة هو كود يستخدم تقنيات البرمجة بشكلٍ فعال. ويضمن هذا أن الكود قابل للقراءة وفعال وقوي. يلخص الشكل 5.25 المزايا الرئيسية لكتابة كود عالي الجودة.

قابلية الصيانة
كود الجودة الجيد يتميز بقابلية الصيانة. هذا يعني أنه منظم جيدًا ويستخدم التركيبات واصطلاحات التسمية والروتينات الفرعية على نحو جيد. يساعد ذلك في الحد من تعقيد الكود، والذي بدوره يجعل من السهل قراءته وإصلاحه وتحسينه بميزات جديدة..

قابلية النقل
تشير قابلية نقل الكود إلى التعليمات البرمجية التي تمت كتابتها باستخدام لغة برمجة لا تعتمد على منصة معينة. تتضمن الأمثلة Java أو Python أو .NET. تسمح قابلية النقل بإمكانية نقل التعليمات البرمجيةإلى منصات أخرى مع القليل من التعديل. على سبيل المثال، ستعمل التعليمات البرمجية المحمولة عبر أنظمة التشغيل المختلفة وIDE وأنواع الأجهزة المختلفة.

سهولة الاستخدام
تشير سهولة استخدام الكود إلى مدى سهولة فهم جزء من التعليمات البرمجية واستخدامه. جيد تساعد سهولة الاستخدام مبرمجًا آخر على فهم كود البرنامج. يمكن أن يقلل ذلك ذمن وقت تصحيح الأخطاء والصيانة.

الشكل 5.25 مزايا كتابة كود جيد. هل يمكنك التفكير في أي ميزات أخرى تُسهِم في كود عالي الجودة؟

مراجعة ما تعلمته ✔

1 صِف الفرق بين المتغير المحلي والمتغير العام.

2 صِف المقصود بتسلسل المصطلحات والتحديد والتكرار.

3 اشرح متى ستستخدم حلقة for وحلقة while وحلقة do-while.

4 ما الفرق بين الدالة والإجراء؟

5 صِف المقصود بمصطلح بنية بيانات المصفوفة.

6 صِف أي أربع قواعد للتحقق من الصحة يمكن استخدامها في البرنامج.

7 اذكر العناصر الأربعة التي ستجدها في خطة الاختبار.

8 صِف المقصود بمصطلحات البيانات العادية والبيانات غير العادية والبيانات المتطرفة.

إجراء التقييم هدف التعلم (ب)

السيناريو

يرغب المبرمج في تطوير برنامج للسماح للأشخاص بمراقبة عدد الدقائق التي يمارسونها كل أسبوع.

قواعد البرنامج:

- ينبغي أن يتمكن المستخدم من تسجيل تفاصيل بشأن كل تمرين. ينبغي أن يشمل ذلك:
- نوع التمرين (مثل المشي والجري والسباحة).
- يوم التمرين
- وقت بدء التمرين
- وقت نهاية التمرين
- ينبغي حساب عدد دقائق التمرين لكل تمرين.
- ينبغي أن ينتج البرنامج بعد ذلك:
- إجمالي عدد التدريبات التي تم الانتهاء منها
- إجمالي الوقت المنقضي في ممارسة الرياضة
- متوسط كمية التمارين التي يتم إكمالها كل يوم
- في أي يوم تم الانتهاء من أكبر قدر من التمارين
- في أي يوم تم الانتهاء من أقل قدر من التمرين.

المهمة

اختر لغة برمجة واحدة وطوِّر هذا البرنامج واختبره وحسّنه. ينبغي لك:

- تنفيذ الكود لضمان اتباع ممارسات التكويد الجيدة
- التأكد من أن البرنامج سهل الاستخدام
- وضع خطة اختبار وإكمالها لاختبار دوال البرنامج باستخدام مجموعة من بيانات الاختبار
- استخدام نتيجة خطة الاختبار لتحسين حل البرنامج
- تبرير أي تغييرات تم إجراؤها

هدف التعلم (ج): مراجعة حل البرنامج

عند إنشاء برنامج، من المهم إكمال المراجعة قبل تسليم البرنامج إلى المستخدمين النهائيين. وتُعد المراجعة مهمة للتأكد من عمل البرنامج بشكلٍ صحيح وللتأكد من أنه يؤدي ما يريده المستخدمون النهائيون.

بدء النشاط

فكّر في مشروع عملت عليه في مدرستك أو كليتك أو مكان عملك. كيف راجعتَ عملك؟ كيف أثّرت هذه المراجعة على عملك المستقبلي؟

(ج1) مراجعة الحل البرمجي

معايير مراجعة فعالية الحل البرمجي

متطلبات المستخدم

عند إنشاء البرنامج، من المهم تذكر المستخدمين الذين سيستخدمون البرنامج والتأكد من أن البرنامج مناسب لاحتياجاتهم.

يمكنك استخدام قائمة التحقق التالية لتحديد ما إذا كان برنامجك مناسبًا لمتطلبات المستخدم أم لا.

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
1	**هل البرنامج مناسب لجميع المستخدمين بما في ذلك أولئك الذين لديهم احتياجات إمكانية الوصول؟** أشياء يجب مراعاتها: • هل البرنامج مناسب للمستخدمين الذين لديهم احتياجات بصرية وسمعية وكلامية وحركية ومعرفية؟ • هل طرق إدخال المستخدم مناسبة لاحتياجات إمكانية الوصول للمستخدم؟ يتضمن ذلك مدخلات الماوس ومدخلات لوحة المفاتيح والمدخلات الصوتية والمدخلات اللمسية. • هل طرق الإخراج مناسبة لاحتياجات إمكانية الوصول للمستخدم؟ يتضمن ذلك المخرجات المرئية والمخرجات الصوتية والمخرجات اللمسية.	
2	**هل البرنامج مناسب لمستوى مهارة المستخدم؟** أشياء يجب مراعاتها: • هل سيتمكن المستخدمون ذوو المهارات الأساسية من استخدام البرنامج دون أن يتعثروا؟ • هل سيتمكن المستخدمون ذوو المهارات المتقدمة من استخدام البرنامج دون الشعور بالإحباط بسبب الكثير من التعليمات؟	
3	**هل البرنامج مناسب لعمر المستخدم؟** أشياء يجب مراعاتها: • هل البرنامج مناسب لعمر المستخدم الحالي؟ • هل البرنامج مناسب بالنظر إلى تجربة المستخدم السابقة؟ • هل يحتوي البرنامج على تخطيط مماثل للبرامج التي استخدمها المستخدمون سابقًا؟	

أفضل ممارسة

من الممارسات الجيدة إجراء المراجعة دائمًا، حتى إذا كنت تعتقد أنها ليست ضرورية. فأثناء المراجعة، قد تتمكن من تعلم أشياء يمكن أن تُغيّر طريقة إنجاز المهام في مشاريع البرمجة المستقبلية

المهارات

المهارات الشخصية: أخلاقيات العمل/الضمير:

- الأخلاقيات
- النزاهة

المصطلحات الرئيسية

الإخراج اللمسي: يتعلق بحاسة اللمس لدى المستخدم - النوع الأكثر شيوعًا من الإخراج اللمسي هو الاهتزاز.

الملاءمة للغرض

من المهم التفكير في الغرض من برنامجك والتأكد من أنه يفي به. فإذا لم يلبِ البرنامج الغرض منه، فقد لا يتيح للمستخدمين إكمال المهام التي يريدون إنجازها. يمكنك استخدام قائمة التحقق التالية لتحديد ما إذا كان برنامجك مناسبًا للغرض منه أم لا.

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
1	**هل يتيح للمستخدم إكمال المهام التي يحتاج إليها؟** أشياء يجب مراعاتها: • هل يتيح البرنامج للمستخدمين تنفيذ المهام التي يحتاجونها؟ • هل طريقة إدخال البيانات مناسبة للغرض منها؟ • هل طريقة إخراج البيانات مناسبة للغرض منها؟ • هل تُخزّن البيانات بطريقة مناسبة للغرض منها؟	
2	**هل يقدم البرنامج نتائج دقيقة؟** أشياء يجب مراعاتها: • هل تم اختبار البرنامج للتأكد من تقديمه مخرجات دقيقة؟ • هل تم اختبار البرنامج بأنواع مختلفة من بيانات الاختبار للتأكد من أنه قوي ويصعب كسره؟ • هل تم تحسين البرنامج لمعالجة أي اختبارات فاشلة؟	

تجربة المستخدم

يتوقع المستخدمون عمومًا أن البرنامج سيتيح لهم إكمال المهام بسرعة. فيمكن أن يصاب المستخدمون بالإحباط إذا تعثروا أو إذا لم يتمكنوا من استخدام البرنامج بسهولة. ويمكنك استخدام قائمة التحقق التالية عند التفكير فيما إذا كان برنامجك سهل الاستخدام أم لا.

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
1	**هل من السهل التنقل في البرنامج؟** أشياء يجب مراعاتها: • هل من الواضح كيف يتنقل المستخدم في واجهة المستخدم؟ • هل تحققتَ من عمل جميع مدخلات المستخدم (مثل الأزرار) بشكلٍ صحيح؟ • هل من الواضح ما تعنيه الخيارات المختلفة للمستخدم؟ • هل هناك تعليمات واضحة يمكن للمستخدم قراءتها إذا واجه مشكلة أو احتاج إلى مساعدة؟	
2	**هل مخرجات المستخدم مثل رسائل الخطأ واضحة؟** أشياء يجب مراعاتها: • هل رسائل الخطأ سهلة الفهم؟ • هل استخدمتَ لغة مناسبة للعمر ومناسبة لمستوى مهارة كل مستخدم؟ • هل ستتيح رسائل الخطأ للمستخدم تصحيح أخطائه بسهولة؟	

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
3	**هل يستجيب البرنامج بسرعة؟** أشياء يجب مراعاتها: • هل يتم تحميل البرنامج بسرعة عند بدء تشغيله؟ • هل يستجيب البرنامج بسرعة بمجرد إدخال مدخلات المستخدم؟	
4	**هل واجهة المستخدم سهلة الاستخدام؟** أشياء يجب مراعاتها: • هل استخدمتَ ألوانًا مناسبة لا تتعارض مع بعضها بعضًا؟ • هل نمط الخط سهل القراءة؟ • هل حجم الخط كبير بما يكفي للقراءة؟ • هل استخدمتَ تخطيطًا ثابتًا على كل شاشة؟	

مواضيع ذات صلة

لمعرفة المزيد عن كيفية تحسين قابلية استخدام البرنامج، راجع **الوحدة 8: مقدمة لتطوير التطبيق**.

النشاط

من خلال العمل مع أحد الزملاء، اختر برنامجًا أو تطبيقًا واحدًا لم تستخدمه من قبل ثم أجب عن الأسئلة التالية.

الأسئلة

1. هل يسهُل التنقل في البرنامج؟ ما سبب ذلك؟
2. هل المخرجات، مثل رسائل الخطأ، واضحة؟ ما سبب ذلك؟
3. هل يستجيب البرنامج أو التطبيق بسرعة؟
4. هل واجهة المستخدم واضحة؟
5. هل البرنامج سهل التعلم؟ كيف تعتقد أن مطوري البرامج قد جعلوا هذا هو الحال؟
6. كيف تعتقد أنه يمكن تحسين قابلية استخدام البرنامج؟

القيود

عندما كنتَ تكتب التعليمات البرمجية، ربما واجهتَ مشكلات لا يمكنك التحكم فيها. من المهم النظر في هذه الأمور لمعرفة ما إذا كان هناك حل ممكن أم لا. كما يمكن أن يساعد ذلك في اتخاذ القرار مستقبلاً.

النشاط

اختر لغة برمجة **واحدة** من اختيارك. ابحث عن القيود المحتملة المرتبطة بهذه اللغة المعينة. حيث يمكنك التفكير في البحث عن أحد الخيارات التالية.

1. نوع الأجهزة التي تناسبها لغة البرمجة.
2. أداء لغة البرمجة.
3. ما مدى سهولة تعلم مبرمج جديد لغة البرمجة.
4. مقدار المساعدة والدعم المتاحين عبر الإنترنت.

يمكنك استخدام قائمة التحقق التالية عند النظر في بعض القيود المحتملة التي أثرت على مشروعك.

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
1	**هل كانت هناك أي قيود على لغة البرمجة؟** أشياء يجب مراعاتها: • هل تحتوي لغة البرمجة على الأدوات التي تحتاجها لتتمكن من إنشاء واجهة مستخدم فعالة؟ • هل ستعمل لغة البرمجة على أنواع مختلفة من الأجهزة مثل الأجهزة اللوحية؟ • هل تعمل لغة البرمجة بسرعة كافية للسماح باستجابات فورية لإدخال المستخدم؟	
2	**هل كان لديك الوقت الكافي لإكمال المشروع؟** أشياء يجب مراعاتها: • هل تم إكمال الكود بحلول الموعد النهائي المحدد؟ • هل كان لديك الوقت الكافي لإكمال جميع متطلبات البرنامج؟ • هل استفدتَ من وقتك بشكلٍ فعال؟	
3	**هل هناك قيود تتعلق بقدرات الجهاز؟** أشياء يجب مراعاتها: • هل ستعمل التعليمات البرمجية عبر أنظمة تشغيل مختلفة؟ • هل يحتوي الجهاز الذي يُشغِّل البرنامج على أجهزة إدخال مناسبة للسماح للمستخدم بإدخال البيانات (مثل لوحة المفاتيح والماوس والشاشة اللمسية والميكروفون)؟ • هل يحتوي الجهاز الذي يُشغِّل البرنامج على أجهزة إخراج مناسبة لنقل الردود من التعليمات البرمجية (مثل الشاشة والسماعات)؟	
4	**هل كانت هناك أية قيود على موارد الأجهزة المتاحة؟** أشياء يجب مراعاتها: • هل يحتوي الجهاز على ذاكرة كافية لتشغيل التعليمات البرمجية؟ • هل يحتوي الجهاز على مساحة تخزين كافية لتخزين برنامجك؟ • هل حجم الشاشة كبير بما يكفي لعرض واجهة المستخدم بشكلٍ صحيح؟	
5	**هل ظهرت أية مشكلات في الاتصال؟** أشياء يجب مراعاتها: • هل تتطلب التعليمات البرمجية الاتصال ببرنامج آخر؟ وهل نجح ذلك؟ • هل تتطلب التعليمات البرمجية الاتصال بشبكة أخرى؟ وهل نجح ذلك؟ • هل تتطلب التعليمات البرمجية الاتصال بملف خارجي؟ وهل نجح ذلك؟	

جودة البرنامج

بمجرد كتابة التعليمات البرمجية، غالبًا ما يتطلب إجراء صيانة له في المستقبل. ويمكن إجراء ذلك من قِبل المبرمج نفسه أو من قِبل مبرمج مختلف. من المهم مراجعة جودة التعليمات البرمجية للتأكد من أن المبرمِجين الآخرين بإمكانهم قراءة التعليمات البرمجية وفهمها وصيانتها بسهولة في المستقبل.

كما يمكنك استخدام قائمة التحقق التالية لتقييم جودة التعليمات البرمجية.

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
1	**هل البرنامج موثوق؟** أشياء يجب مراعاتها: • هل اختبرت برنامجك بمجموعة كبيرة من بيانات الاختبار؟ • هل ينتج برنامجك مخرجات دقيقة؟ • هل سيستجيب برنامجك للمستخدمين بسرعة؟	
2	**هل البرنامج سهل الاستخدام؟** أشياء يجب مراعاتها: • هل من السهل التنقل في البرنامج؟ • هل التعليمات واضحة ويسهُل فهمها؟ • هل تم تصنيف نماذج إدخال المستخدم بوضوح؟ • هل هناك قائمة مساعدة للوصول إليها في حالة تعطل المستخدم؟	
3	**هل يؤدي البرنامج المهام بكفاءة؟** أشياء يجب مراعاتها: • هل استخدمتَ نموذج البرمجة الصحيح للمهمة؟ • هل جعلتَ التعليمات البرمجية فعالة من خلال إعادة استخدام مكونات البرنامج؟	
4	**هل سيكون من السهل صيانة البرنامج في المستقبل؟** أشياء يجب مراعاتها: • هل استخدمتَ تعليقات الأكواد للسماح للمبرمِجين الآخرين بفهم معنى الكود؟ • هل استخدمتَ المسافة البادئة للسماح للمبرمِجين الآخرين برؤية مختلف المجموعات في الكود ومعرفة الكود الذي ينتمي إلى أي شروط؟ • هل استخدمتَ أسماء متغيرات مناسبة حتى يعرف المبرمِجون الآخرون ما تعنيه؟ • هل استخدمتَ اصطلاحات تسمية متسقة في جميع أنحاء التعليمات البرمجية؟	
5	**هل البرنامج محمول؟** أشياء يجب مراعاتها: • هل استخدمتَ لغة برمجة غير خاصة بالجهاز؟ • هل فكرتَ في المكونات المتوفرة على أجهزة مختلفة؟ • هل فكرتَ في أحجام الشاشات المختلفة على الأجهزة المختلفة؟	

نقاط القوة والتحسينات.

من المهم مراعاة نقاط القوة في برنامجك. بصفتك مبرمِجًا، ستعمل على الكثير من مشاريع البرمجة المختلفة. وستساعدك معرفة نقاط القوة في عملك على تسليط الضوء على الممارسات الجيدة التي يمكن نقلها إلى المشاريع المستقبلية.

يمكنك استخدام قائمة التحقق التالية عند النظر في نقاط قوة برنامجك.

العنصر	العنصر الذي يلزم التحقق منه	نعم/لا؟
1	**هل نموذج البرمجة المختار مناسب للبرنامج؟** هل نموذج البرمجة الذي استخدمته مناسب لـ: • حجم البرنامج • تعقيد البرنامج • أمان البيانات • الصيانة المستقبلية للبرنامج؟	
2	**هل التعليمات البرمجية فعالة؟** أشياء يجب مراعاتها: • هل استخدمتَ التكرار (الحلقات) لتكرار الكود عند الضرورة؟ • هل أنشأتَ إجراءات فرعية، مثل الدوال أو الإجراءات، وأعدتَ استخدامها في أجزاء أخرى من البرنامج؟	
3	**هل اختبرتَ برنامجك للتأكد من فعاليته وقوته؟** أشياء يجب مراعاتها: • هل كتبتَ خطة اختبار شاملة وأكملتها لاختبار جميع المجموعات المختلفة من مدخلات المستخدم؟ • هل استخدمتَ مزيجًا من البيانات العادية وغير العادية والمتطرفة؟ • هل حسّنتَ برنامجك لإصلاح أي اختبارات فاشلة؟	
4	**هل توقعتَ سوء استخدام محتمل ووضعتَ خطوات للتخفيف من هذه المشكلة؟** أشياء يجب مراعاتها: • هل أضفتَ الأمان إلى برنامجك، مثل أسماء المستخدمين وكلمات المرور؟ • هل وضعتَ خطوات للتأكد من أن المستخدمين الحقيقيين فقط يمكنهم استخدام برنامجك؟	
5	**هل استخدمتَ التحقق الفعال من صحة البيانات لتحسين إدخال البيانات؟** أشياء يجب مراعاتها: • هل استخدمتَ اختبارات تحقق مختلفة للتأكد من أن إدخال البيانات منطقي ومعقول؟ • هل استخدمتَ رسائل خطأ واضحة لإعلام المستخدمين بأخطائهم؟	

فكر مليًا

ما المجالات التي يمكن تطويرها بشكلٍ أكبر؟

بمجرد مراجعة برنامجك، من المهم تحسينه، خاصةً إذا كنتَ تعتقد أن هناك طريقة أفضل لتلبية احتياجات المستخدمين أو لتحسين تجربتهم في استخدام البرنامج. لذا، استخدم قوائم المراجعة من هذا القسم لتحديد مجالات التطوير المستقبلي. حيث ستشير الأماكن التي لم تتمكن فيها من تحديد عمود "نعم/لا" إلى المناطق التي يمكن تطويرها بشكلٍ أكبر.

كما تحتاج إلى إلقاء نظرة على أي قيود واجهتها. وقد ترغب في التفكير في تحسين هذه المجالات من البرنامج، رغم أن طبيعة القيود قد تجعل أي تحسينات مستحيلة. ومع ذلك، سيساعدك هذا التقييم على تحسين طريقة عملك في مشاريع البرمجة المستقبلية.

مراجعة ما تعلمته

1. صِف **أربعة** عوامل مختلفة ينبغي مراعاتها أثناء مراجعة البرنامج.
2. ضع قائمة **بأربعة** قيود محتملة قد يواجهها المبرمج في مشروع برمجة.
3. ضع قائمة **بأربعة** عوامل ينبغي تقييمها عند النظر في جودة التعليمات البرمجية.

إجراء التقييم هدف التعلم (ج)

عند إجراء التقييم في نهاية هدف التعلم (ب)، طورتَ برنامجًا للسماح للأشخاص بمراقبة عدد الدقائق التي يمارسونها كل أسبوع. ينبغي لك استخدام البرنامج الذي طورته من أجل:

- مراجعة حل البرنامج للتأكد من أنه:
  - مناسب لمتطلبات المستخدم
  - مناسب للغرض
  - سهل الاستخدام
  - يستفيد بشكلٍ جيد من ميزات البرمجة وتقنياتها
- وصف نقاط القوة في برنامجك - أعطِ أسبابًا وجيهة.
- تقديم **ثلاث** توصيات على الأقل لإجراء المزيد من التحسينات التي ترغب في إجرائها على برنامجك.

أنشطة التقييم: أهداف التعلم (أ) و(ب) و(ج)

مقدمة

الآن بعد أن تناولتَ جميع المواضيع في هذه الوحدة، تحتاج إلى إظهار فهمك لنماذج البرمجة الثلاثة المختلفة التي درستها. بعد ذلك، ستحتاج إلى تطوير برنامجك الخاص للوفاء بالموجز واختبار البرنامج بحثًا عن الدوال وتحديد أي أخطاء وإصلاحها. أخيرًا، تحتاج إلى مراجعة برنامجك النهائي وتقييم الطرق التي يمكن تحسينها.

نقطة المراجعة

التعزيز

- صِف **ثلاثة** نماذج برمجة مختلفة يمكن استخدامها ونوع البرامج المناسبة لها أيضًا.
- اشرح الميزات الرئيسية لكل نموذج برمجة.
- صِف **ثلاثة** أنواع مختلفة من البيانات وما يتم استخدامها من أجله.
- اذكر **أربعة** أمثلة للممارسات الجيدة التي ينبغي اتباعها عند تسمية المتغيرات والثوابت.
- صِف **ثلاثة** أنواع مختلفة من بيانات الاختبار التي يمكن استخدامها لضمان عمل البرنامج وقوته.

التحدي

- حلل مزايا كل نموذج برمجة وعيوبه.
- أعط أمثلة لمختلف البرامج التي يمكن كتابتها في كل نموذج برمجة.
- قيّم مدى ملاءمة نماذج البرمجة المختلفة للبرامج المختلفة.
- صِف مختلف بُنى البيانات التي يمكن استخدامها لتخزين البيانات بشكلٍ أكثر كفاءة.
- وضّح **ثلاث** طرق للتأكد من أن الكود قابل للصيانة ويسهل على المبرمِجين الآخرين قراءته.

نشاط التقييم **هدف التعلم (أ)**

السيناريو

أنت مطور برامج مبتدئ في شركة برمجة محلية.

وطلب منك المدير إنشاء عرض تقديمي سيُستخدم لتدريب مطوري البرامج المبتدئين الجدد عند انضمامهم إلى الشركة.

كما يرغب المدير في إنشاء تعليمات برمجية لبرنامج بأي لغة برمجة من اختيارك.

المهمة 1 - الوقت المقترح 3 ساعات

يغطي هذا النشاط هدف التعلم (أ).

دراسة لغات البرمجة

احرص على إعداد عرض تقديمي لتقييم مدى ملاءمة نماذج البرمجة المختلفة. ويحتاج العرض التقديمي إلى تقييم مختلف ميزات نماذج البرمجة بما في ذلك مدى ملاءمتها للبرامج المختلفة ومزايا كل نهج وعيوبه.

ينبغي أن يتضمن العرض التقديمي العناصر التالية:

تتمة هدف التعلم (أ)

1 النماذج القائمة على الأحداث

2 البرمجة الكائنية

3 النماذج الإجرائية.

ينبغي لك تقييم مدى ملاءمة نماذج البرمجة هذه للبرامج التالية:

1 نظام حجز لتنظيف السجاد يحتوي على واجهة مستخدم.

2 برنامج يطرح أسئلة الرياضيات ويضع علامة عليها.

3 لعبة سباق سيارات حيث يتعين على اللاعب منافسة سيارات أخرى يتحكم فيها الحاسوب.

يجب أن يتضمن العرض التقديمي:

1 ميزات كل نموذج برمجة ذي صلة بتطوير كلٍ من البرامج المذكورة أعلاه.

2 مختلف المزايا والعيوب لاستخدام كل نموذج برمجة لتطوير كلٍ من البرامج المذكورة أعلاه.

3 تبرير سبب كون كل نموذج برمجة مناسبًا أو غير مناسب لكل لعبة مدرجة أعلاه.

المهارات

المهارات المعرفية: العمليات والاستراتيجيات المعرفية:

- التحليل
- اتخاذ القرار

معايير التقييم هدف التعلم (أ)

النجاح	التفوق	الامتياز
هدف التعلم (أ): فهم الميزات الرئيسية لمختلف نماذج البرمجة		
A.P1 تحديد الميزات الرئيسية لنماذج البرمجة القائمة على الأحداث والبرمجة الكائنية والإجرائية.	**A.M1** توضيح ميزات نماذج البرمجة القائمة على الأحداث والبرمجة الكائنية والإجرائية ومدى ملاءمتها لأغراض تطوير مختلف البرامج.	**A.D1** تقييم مدى ملاءمة نماذج البرمجة القائمة على الأحداث والبرمجة الكائنية والإجرائية لحل برمجي مقترح.
A.P2 تحديد الأسباب التي تجعل كل نموذج مناسب في مواقف معينة.		

نصائح

- تأكد من تغطية جميع نماذج البرمجة الثلاثة التي تناولتها في هدف التعلم هذا. وتأكد من أنك تتناول بوضوح مدى ملاءمتها لكلٍ من البرامج الثلاثة الموضحة في تقييم الإجراء.
- تأكد من أن العرض التقديمي يقدم تقييمًا واضحًا ومتوازنًا لكل نموذج برمجة. ويعني هذا أنك تُضمِّن مزايا كل نموذج برمجة وعيوبه. كدليل، ينبغي لك محاولة تضمين فائدتين وعيبين على الأقل.

نشاط التقييم | أهداف التعلم (ب) و(ج)

المهمة 2 - الوقت المقترح 12 ساعة

يغطي هذا النشاط هدفي التعلم (ب) و(ج).

تطوير حل برنامج واختباره

حددت شركة البرمجة ثلاثة برامج مختلفة تحتاج إلى تطوير. وأنت مطالب فقط ببرمجة **أحد** هذه البرامج. حيث يتم تطوير البرنامجين الآخرين من قبل مطوري البرامج المبتدئين الآخرين.

البرنامج 1

اكتب برنامجًا يطرح أسئلة رياضية على المستخدم.

قواعد البرنامج:

- اسأل المستخدم عن عدد الأسئلة التي يريد الإجابة عنها.
- اسأل المستخدم هذا العدد من الأسئلة.
- ينبغي أن يستخدم كل سؤال رقمين عشوائيين وعملية حسابية عشوائية مثل + أو - أو/أو *. على سبيل المثال، قد يكون السؤال: 9 + 7 = .
- إذا أجاب المستخدم عن السؤال بشكلٍ صحيح، ينبغي للبرنامج تهنئة المستخدم وإضافة درجة إلى درجاته.
- إذا أجاب المستخدم عن السؤال بشكلٍ غير صحيح، فينبغي للبرنامج عرض الإجابة الصحيحة.
- بمجرد الإجابة عن جميع الأسئلة، ينبغي أن يتم إخراج درجة المستخدم.

البرنامج 2

اكتب لعبة ثنائية اللاعبين تتيح للاعب 1 إدخال كلمة يجب على اللاعب 2 تخمينها.

قواعد البرنامج:

- يُطلَب من المستخدم 1 إدخال كلمة (مثل القاموس)
- بعد ذلك، يتم إخراج القرائن التالية:
  - "عدد الحروف هو x" حيث x هو عدد الحروف في الكلمة.
  - "الحرف الأول هو x" حيث يكون x هو الحرف الأول في الكلمة.
- ينبغي أن يتمكن المستخدم 2 بعد ذلك من تخمين الكلمة.
- إذا خمّن الكلمة بشكلٍ صحيح، فسيخرج البرنامج كلمة "صحيح"، وإلا فسيخرج "غير صحيح، حاول مرة أخرى".

البرنامج 3

أنشيء لعبة X - O. حيث يتناوب لاعبان على وضع O أو X في شبكة 3 × 3. وأول لاعب يحصل على عدد 3 O أو 3 X على التوالي يُعد هو اللاعب الفائز. ويمكن أن يحدث هذا أفقيًا أو رأسيًا أو قطريًا. على سبيل المثال، في هذا السيناريو، اللاعب الذي يضع علامات O هو اللاعب الفائز.

X		O
	O	X
O	X	

المهارات

المهارات المعرفية: العمليات والاستراتيجيات المعرفية:

- التفكير الناقد
- حل المسائل
- التفسير

المهارات

المهارات الشخصية: أخلاقيات العمل/الضمير:

- المبادرة
- المثابرة

تتمة أهداف التعلم (ب) و(ج)

قواعد البرنامج:

- اسمح للمستخدم بإدخال موضع O أو X.
- تحقق مما إذا كان هذا الموضع فارغًا أم لا.
- إذا كان هذا الموضع يحتوي بالفعل على O أو X، فينبغي أن ينتج رسالة خطأ مناسبة.
- إذا كان المركز فارغًا، فينبغي وضع O أو X في هذا الموضع ثم التحقق مما إذا كان اللاعب قد فاز.

المهمة

بالنسبة للبرنامج الذي اخترته، عليك تطوير التعليمات البرمجية واختيارها وتحسينها ومراجعتها باستخدام أي لغة برمجة من اختيارك.

تأكد من تضمين ما يلي:

1 أنشِئ واجهة مستخدم تتيح للمستخدم استخدام البرنامج بسهولة.

ينبغي أن تقوم الواجهة بما يلي:

- السماح للمستخدمين بإدخال البيانات والأوامر
- عرض المخرجات بوضوح مثل الرسائل.

2 نَفِّذ الكود الخاص بالبرنامج، مع ضمان اتباع ممارسات الترميز الجيدة.

ينبغي لك:

- إضافة تعليقات الأكواد لشرح الغرض من التعليمات البرمجية عند الاقتضاء
- إضافة المسافات البادئة لإظهار مختلف مجموعات الأكواد بوضوح ولإظهار الكود الذي ينتمي إلى جملة الاختيار أو التكرار
- استخدام أسماء المتغيرات واصطلاحات التسمية التي تحدد بدقة البيانات التي يتم تخزينها

3 احرص على وضع خطة اختبار وأكملها لاختبار وظائف البرنامج باستخدام مجموعة من بيانات الاختبار، بما في ذلك:

- البيانات العادية
- البيانات غير العادية
- البيانات المتطرفة

4 استخدم نتيجة خطة الاختبار لتحسين حل البرنامج لديك. ثم برّر أي تغييرات تم إجراؤها. وينبغي لك التأكد من توثيق أي تغييرات بوضوح.

5 راجع حل البرنامج. واحرص على تضمين ما يلي في مراجعتك:

- متطلبات المستخدم
- الملاءمة للغرض
- تجربة المستخدم
- القيود
- جودة البرنامج
- نقاط القوة

6 أخيرًا، ينبغي لك تقديم **ثلاث** توصيات على الأقل لمزيد من التحسينات التي ترغب في إجرائها على برنامجك.

معايير التقييم	أهداف التعلم (ب) و(ج)	
هدف التعلم (ب): تطوير حل برنامج واختباره		
B.P3 تطوير حل برمجي لموجز معين.	**B.M2** تحسين البرنامج المطور وتنقيحه مقابل نتائج الاختبار.	**B.D2** تحسين البرنامج المطور مقابل نتائج الاختبار لتبرير التغييرات التي تم إجراؤها على جودة الكود لجعله مناسبًا للغرض وتقديم توصيات لمزيد من التحسين.
B.P4 اختبار حل البرنامج وتحسينه للوظائف والجودة.		
هدف التعلم (ج): مراجعة حل البرنامج		
C.P5 تحديد أسباب التغييرات التي تم إجراؤها في مرحلة الاختبار.	**C.M3** توضيح أسباب التغييرات التي تم إجراؤها في مرحلة الاختبار.	
C.P6 وصف كيفية تلبية حل البرنامج النهائي لمتطلبات المستخدم وملاءمته للغرض.	**C.M4** شرح كيف يلبي حل البرنامج النهائي متطلبات المستخدم وملاءمته للغرض.	

نصائح

- تأكد من اتباع ممارسات البرمجة الجيدة أثناء كتابة التعليمات البرمجية. على سبيل المثال، أضف تعليقات الكود واستخدم المسافة البادئة وأسماء المتغيرات المناسبة. فقد يكون من الصعب جدًا تغييرها أو إضافتها في وقتٍ لاحق.
- تأكد من إكمال أكبر عدد ممكن من الاختبارات واستخدام مجموعة من بيانات الاختبار لاختبار أكبر عدد ممكن من مجموعات مدخلات المستخدم قدر الإمكان.
- تأكد من تحسين التعليمات البرمجية إذا كنت تعتقد أن هناك طريقة أفضل لتلبية متطلبات المستخدمين، لتحقيق غرض البرنامج أو لتحسين تجربة المستخدم.

استكشف المزيد

أنشئ مولّد أرقام عشوائي باستخدام القواعد الموضحة أدناه. يمكنك استخدام أي لغة برمجة من اختيارك.

قواعد البرنامج:

- أنشئ رقمًا عشوائيًا بين واحد و50.
- قبل إخراج الرقم، ينبغي أن يتحقق البرنامج مما إذا كان الرقم قد تم إنشاؤه بالفعل أم لا. فإذا كان الرقم قد تم إنشاؤه بالفعل، ينبغي إنشاء رقم آخر ثم التحقق مرة أخرى.
- في حالة عدم إنشاء رقم بالفعل، ينبغي أن يُخرج البرنامج بعد ذلك هذا الرقم إلى المستخدم.
- ينبغي أن يحتفظ بسجل لكل رقم تم إخراجه.
- ينبغي أن يكرر البرنامج الخطوات المذكورة أعلاه لحين إنشاء جميع الأرقام الخمسين.

مسرد المصطلحات

العوامل الحسابية: تُستخدم لإجراء الحسابات الأساسية مثل عمليات الجمع والطرح والضرب والتقسيمات.

المصفوفة: تُستخدم لتخزين قوائم البيانات التي تحتوي جميعها على نوع البيانات نفسه - يتم إعطاء كل عنصر في المصفوفة رقم فهرس مختلف.

المتغيرات: جزء من البيانات أو القيمة المحفوظة في البرنامج.

النهج التصاعدي: النهج التصاعدي هو استراتيجية تطوير تهدف إلى تقسيم البرنامج إلى وحدات صغيرة قابلة للفهم والتطوير، ثم تكوين هذه الوحدات بشكل تدريجي

لتشكل البرنامج الكامل. هذا يُعرف أيضًا بمفهوم التطوير التدريجي.

الكائن: يحدد السمات والطرق في البرنامج وتعمل كقالب لإنشاء العناصر.

مخطط الكائنات: يُستخدم لتخطيط الكائنات المستخدمة في نموذج موجه للعناصر - يعرض كل مخطط العلاقات مع الكائنات الأخرى في البرنامج نفسه.

التصنيف: عندما يتم وضع البيانات في فئات بناءً على خصائصها أو خصائصها.

تعليقات الكود: التعليقات الإضافية التي تتم إضافتها إلى التعليمات البرمجية لشرح معنى سطر أو مجموعة أسطر بالكود.

ترتيب التعليمات البرمجية لسهولة القراءة للبداية والنهاية: عند وضع علامة تبويب في بداية سطر الكود للإشارة إلى متى يبدأ شيء ما وينتهي.

صيغة الكود: مجموعة من المفردات والقواعد في كل لغة برمجة معينة يجب على المبرمجين اتباعها عند كتابة التعليمات البرمجية.

الثوابت: أجزاء مفردة من البيانات التي تم حفظها في ذاكرة الكمبيوتر والتي لا يمكن تغييرها في التعليمات البرمجية.

الإنشاء: يشير إلى تدفق البرنامج - فهو يشير إلى أسطر الكود التي يتم تشغيلها وترتيب تشغيلها.

تجريد البيانات: يُقصد به إزالة التفاصيل غير الضرورية من مشكلة للسماح للمبرمجين بالتركيز على الجوانب الأساسية لها للبرنامج.

بنية البيانات: تستخدم لتخزين البيانات وتنظيمها في برنامج حاسوب.

نوع البيانات: يحدد هذا كيفية حفظ البيانات في برنامج (على سبيل المثال، كرقم أو نص).

التحقق من صحة البيانات: يضمن أن البيانات التي يتم إدخالها صحيحة ومعقولة وضمن الحدود.

تصحيح الأخطاء: عندما يحاول المبرمج العثور على الأخطاء وإصلاحها في التعليمات البرمجية.

حلقة Do-while: حلقة يتم التحكم فيها بالشرط والتي ستكرر مقطعًا من الكود لحين استيفاء الشرط. وستختبر الشرط في نهاية الحلقة.

التغليف: يُقصد به عندما تصبح السمات خاصة بحيث لا يمكن تغييرها إلا بالطرق الموجودة في الفئة ذاتها.

الحدث: إجراء يحتاج البرنامج إلى الاستجابة له.

النماذج القائمة على الأحداث: إعطاء تعليمات للكمبيوتر بشأن كيفية الاستجابة لمختلف الأحداث.

معالج الأحداث: مجموعة من الأكواد التي سيتم تشغيلها استجابةً لحدث وقع في أحد البرامج.

موزع رسائل الأحداث: يراقب الأحداث التي تجري في برنامج حاسوب وينتظرها.

حلقة الأحداث: تستمع باستمرار للأحداث التي يتم تشغيلها ثم تستدعي معالج الأحداث المناسب عند اكتشاف حدث معين.

مشغّل الحدث: يحدد التعليمات البرمجية الذي سيتم تشغيله بمجرد اكتشاف الحدث.

الحقل: عمود في جدول - يحتوي كل عمود على معلومات عن جانب واحد من الشخص أو المكان أو الشيء أو المفهوم.

الاختبار النهائي: اختبار التعليمات البرمجية عند الانتهاء من الكود.

حلقة for: حلقة يتم التحكم فيها بالعد والتي ستكرر مقطعًا من الكود لعدد مرات ثابت.

النموذج: يتيح للمستخدم إدخال البيانات والأوامر إلى الكمبيوتر باستخدام العناصر الرسومية - غالبًا ما تحتوي النماذج على أزرار ومربعات نصية لمساعدة المستخدم.

عناصر تحكم النموذج: هذه أجزاء مفردة من نموذج إدخال المستخدم تتيح للمستخدمين التفاعل مع البرنامج - وهي تتضمن عناصر مثل مربعات النص والأزرار.

الدوال: الإجراءات الفرعية التي يمكن استدعاؤها لإكمال المهمة - سترجع القيمة مرة أخرى إلى أجزاء أخرى من البرنامج.

المتغيرات العالمية: المتغيرات التي تم إنشاؤها خارج روتين فرعي والتي يمكن الوصول إليها من خلال جميع أجزاء البرنامج.

الإخراج اللمسي: يتعلق بحاسة اللمس لدى المستخدم - النوع الأكثر شيوعًا من الإخراج اللمسي هو الاهتزاز.

أرقام الفهرس: تُستخدم لتحديد موضع كل عنصر في المصفوفة.

الوراثة: عندما تأخذ إحدى الفئات السمات والطرق المحددة بالفعل في فئة أخرى.

أمر الإدخال: يأخذ هذا البيانات التي أدخلها المستخدم في البرنامج.

بيئة التطوير المتكامل (IDE): برنامج يستخدمه المبرمجون لكتابة الأكواد وتصحيحها.

التكرار: عند تكرار سطر واحد أو أكثر من الكود أكثر من مرة.

الاختبار التكراري: اختبار التعليمات البرمجية أثناء كتابة الكود.

المتغيرات المحلية: المتغيرات التي تم إنشاؤها داخل روتين فرعي والتي تكون مرئية فقط في هذا الروتين الفرعي.

العوامل المنطقية: تُستخدم لإجراء عمليات حسابية أكثر تعقيدًا مثل مقارنة جزء من البيانات بجزء آخر منها.

خطأ منطقي: عندما تكون قواعد البرنامج أو المنطق غير صحيحة - سيؤدي ذلك إلى قيام البرنامج بشيء غير متوقع.

الطرق (البرمجة): مجموعات الأكواد التي ستنفذ الإجراءات في البرنامج.

اصطلاح التسمية: طريقة متفق عليها لتسمية الأشياء، مثل المتغيرات أو الدوال، في البرنامج.

البرمجة الكائنية (OOPs): إعطاء تعليمات لأجهزة الكمبيوتر استنادًا إلى عناصر العالم الحقيقي التي لها خصائص مشتركة. وتتفاعل هذه العناصر مع بعضها بعضًا لحل مشكلة.

الدوال: مكونات مستقلة في برنامج يمكن استخدامه لتمثيل الأشخاص أو الأماكن أو الأشياء أو المفاهيم.

المصفوفة أحادية البعد: تُخزّن البيانات بطريقة خطية في صف واحد، حيث يتم تخزين البيانات في مواقع متتالية.

أمر الإخراج: يعرض هذا شيئًا من التعليمات البرمجية للمستخدم.

المعاملات والوسائط: هي أجزاء من البيانات توضع في إجراء أو دالة للسماح لها بإكمال مهمتها.

تعددية الأشكال: عندما يمكن أن تعني التقنية أشياءً مختلفة في فئات أو عناصر مختلفة.
و"ستعرف" هذه التقنية تلقائيًا المعنى الصحيح لشيءٍ ما بناءً على سياق الكود.

الدوال المحددة مسبقًا: مجموعات الأكواد المكتوبة مسبقًا والمضمنة في لغة البرمجة - فهي تكمل مهام محددة.

النماذج الإجرائية: أخبر الكمبيوتر بما يجب فعله بناءً على تعليمات تدريجية.

الإجراءات: الإجراءات الفرعية التي يمكن استدعاؤها لإكمال المهمة - لا تُرجِع القيم مرة أخرى إلى أجزاء أخرى من البرنامج.

المبرمج: الشخص الذي يكتب التعليمات البرمجية بلغة برمجة معينة.

التعليمات البرمجية: مجموعة من التعليمات التي يتبعها برنامج الكمبيوتر عند إكمال المهام.

لغة البرمجة: تُستخدم لكتابة التعليمات البرمجية. وتحتوي كل لغة على قواعد بناء معينة يجب على المبرمج اتباعها.

نموذج البرمجة: أسلوب كتابة وهيكلة التعليمات البرمجية؛ تشمل الأمثلة النماذج القائمة على الأحداث والبرمجة الكائنية والنماذج الإجرائية.

السجل: صف في جدول - يحتوي كل صف على عدة حقول بشأن شخص أو مكان أو شيء أو مفهوم.

بنية السجل: تُستخدم لتخزين البيانات بشأن شخص أو مكان أو شيء أو مفهوم - يمكن إعطاء كل حقل في البنية نوع بيانات مختلف.

الكلمة المحجوزة: كلمة لا يمكن للمبرمج استخدامها لتسمية عناصر مثل المتغير. فالكلمة المحجوزة لها معنى خاص في لغة البرمجة المستخدمة.

التحديد: عند مقاطعة تدفق البرنامج واختبار الشرط. بعد ذلك، سيتم تمرير التحكم إلى نقطة أخرى في البرنامج بناءً على نتيجة هذا الشرط.

التسلسل: عند تشغيل كل سطر من التعليمات البرمجية واحدًا تلو الآخر وبترتيب كتابته.

مخطط الهيكل: يقسم المشكلة بصريًا إلى مشكلات أصغر.

الدوال الفرعية: مجموعة مستقلة من الأكواد يمكن للبرنامج استدعاؤها لإكمال مهمة.

خطأ في بناء الجملة: عندما لا يتم اتباع القواعد للغة البرمجة، مما يتسبب في تعطل البرنامج.

بيانات الاختبار: البيانات المحددة للتحقق من عمل كود البرمجة بشكلٍ صحيح.

الاختبار: التحقق من خلو التعليمات البرمجية من الأخطاء وعمله على النحو المنشود وتلبيته لمتطلبات المستخدم.

خطة الاختبار: مستند يسرد جميع الاختبارات المختلفة التي ينبغي إجراؤها على البرنامج لاختبار أن البرنامج يعمل بشكلٍ صحيح.

النهج من أعلى إلى أسفل: عند تقسيم البرامج إلى أقسام أصغر يمكن ترميزها وتحسينها بشكلٍ منفصل.

المصفوفة ثنائية الأبعاد: تُخزن البيانات في بنية تشبه الشبكة تتكون من صفوف وأعمدة.

واجهة المستخدم: تتيح للمستخدمين إدخال البيانات والأوامر في برنامج حاسوب، غالبًا باستخدام نموذج إدخال المستخدم.

المتغيرات: أجزاء مفردة من البيانات التي تم حفظها في ذاكرة الكمبيوتر والتي يمكن تغييرها في التعليمات البرمجية.

حلقة while: حلقة يتم التحكم فيها بالشرط والتي ستكرر مقطعًا من الكود لحين استيفاء الشرط. وستختبر الشرط في بداية الحلقة.